

The Agent's Signature

A Legal and Technical Framework for AI Authorization

Mauricio A. Fernandez Fernandez

January 2026

Contents

1	The Agent's Signature	7
1.1	Identity, Authorization, and Legal Authority for Autonomous Systems . . .	7
2	Preface	8
3	Table of Contents	9
4	Part I: Problem Statement and Threat Model	11
4.1	Chapter 1: Autonomous Action vs. Authorized Action	11
4.2	Chapter 2: The Category Error in Modern Authorization	15
4.3	Chapter 3: The \$18 Million Question: A Case Study	18
4.4	Chapter 4: Threat Model for Autonomous Agents	21
5	Part II: Related Work and Prior Art	24
5.1	Chapter 5: OAuth 2.0 and Its Structural Limitations	24
5.2	Chapter 6: Capability Tokens: Macaroons, Biscuit, UCAN	27
5.3	Chapter 7: SPKI/SDSI and Delegation Naming	30
5.4	Chapter 8: Certificate Transparency and Trust Logs	32
5.5	Chapter 9: Policy-Based Access Control	34
5.6	Chapter 10: Legal Foundations of Agency	35
6	Part III: System Model and Protocol Overview	38
6.1	Chapter 11: Entities and Identities (AAP-001)	38
6.2	Chapter 12: Entity Profiles and Cryptographic Binding	40
6.3	Chapter 13: Design Invariants and Fail-Closed Semantics	41
6.4	Chapter 14: Protocol Architecture Overview	42
7	Part IV: Proof of Authorization (AAP-002)	44
7.1	Chapter 15: The PoA Primitive	44
7.2	Chapter 16: Formal Specification	46
7.3	Chapter 17: Delegation Chains	49
7.4	Chapter 18: Scope and Constraint Models	50
7.5	Chapter 19: Revocation Semantics	52
7.6	Chapter 20: Implementation Patterns	54
8	Part V: Cryptographic Architecture	56
8.1	Chapter 21: Design Principles	56

8.2	Chapter 22: Signature Schemes and Key Management	57
8.3	Chapter 23: Canonicalization and Hash Functions	58
8.4	Chapter 24: Revocation Transparency Logs	59
8.5	Chapter 25: Degraded Mode Operations	61
8.6	Chapter 26: Post-Quantum Considerations	62
9	Part VI: Legal Mapping Across Jurisdictions	65
9.1	Chapter 27: Authority as a Legal Concept	65
9.2	Chapter 28: German Law: Statutory Representation	67
9.3	Chapter 29: United States: Actual vs. Apparent Authority	68
9.4	Chapter 30: European Union: eIDAS and Trust Services	70
9.5	Chapter 31: Cross-Border and Conflict-of-Law	72
9.6	Chapter 32: Regulatory Compliance Implications	73
10	Part VII: Operational Considerations	75
10.1	Chapter 33: Deployment Architectures	75
10.2	Chapter 34: Audit Trail Design	76
10.3	Chapter 35: Integration Patterns	77
11	Part VIII: Non-Goals and Limitations	79
11.1	Chapter 36: What This System Does Not Do	79
11.2	Chapter 37: The Boundaries of Machine Authority	80
12	Part IX: Security Considerations	82
12.1	Chapter 38: Threat Analysis	82
12.2	Chapter 39: Attack Vectors and Mitigations	83
12.3	Chapter 40: Incident Response	83
13	Appendices	85
13.1	Appendix A: Original Contributions	85
13.2	Appendix B: Complete Protocol Specification	86
13.3	Appendix C: Reference Implementation	87
13.4	Appendix D: Glossary	89
13.5	Appendix E: Extended References	89
14	About the Author	91
15	Colophon	92
16	Part X: Extended Case Studies	94
16.1	Chapter 41: The Procurement Agent Incident	94
16.2	Chapter 42: Cross-Border Financial Services	97
16.3	Chapter 43: Healthcare AI Agent Authorization	99
16.4	Chapter 44: Supply Chain Orchestration	101
16.5	Chapter 45: Government Benefits Administration	103
17	Part XI: Implementation Cookbook	106
17.1	Chapter 46: Getting Started	106
17.2	Chapter 47: Constraint Design Patterns	109

17.3 Chapter 48: Deployment Checklists	112
17.4 Chapter 49: Regulatory Compliance Matrices	113
17.5 Chapter 50: Troubleshooting Guide	114
18 Part XII: Advanced Topics	115
18.1 Chapter 51: Multi-Agent Orchestration	115
18.2 Chapter 52: Dynamic Constraint Evaluation	117
18.3 Chapter 53: Forensic Analysis	118
18.4 Chapter 54: Performance Optimization	120
18.5 Chapter 55: Custom Constraint Development	122
19 Part XIII: Future Directions	125
19.1 Chapter 56: The Evolving Landscape	125
19.2 Chapter 57: Regulatory Horizons	126
19.3 Chapter 58: Protocol Evolution	126
19.4 Chapter 59: Integration with AI Safety	127
19.5 Chapter 60: Building the Ecosystem	128
20 Afterword	130
20.1 The Path Forward	130
21 Part XIV: Detailed Deployment Scenarios	131
21.1 Chapter 61: Enterprise Deployment Architecture	131
21.2 Chapter 62: Cloud-Native Deployment	136
21.3 Chapter 63: Edge and Embedded Deployment	141
21.4 Chapter 64: Migration from Legacy Systems	144
21.5 Chapter 65: Monitoring and Observability	147
21.6 Chapter 66: Disaster Recovery	151
22 Part XV: SDK and Integration Guides	153
22.1 Chapter 67: Go SDK Reference	153
22.2 Chapter 68: Python SDK Reference	157
22.3 Chapter 69: JavaScript/TypeScript SDK Reference	160
22.4 Chapter 70: Rust SDK Reference	162
22.5 Chapter 71: Java SDK Reference	164
22.6 Chapter 72: API Gateway Integration	167
23 Part XVI: Protocol Deep-Dives	171
23.1 Chapter 73: Wire Format Specification	171
23.2 Chapter 74: Cryptographic Protocol Details	173
23.3 Chapter 75: Revocation Log Protocol	176
23.4 Chapter 76: Constraint Evaluation Engine	179
23.5 Chapter 77: Entity Profile Protocol	183
23.6 Chapter 78: Testing and Validation	187
24 Part XVII: Industry-Specific Implementation Guides	190
24.1 Chapter 79: Financial Services Implementation	190
24.2 Chapter 80: Healthcare Implementation	194
24.3 Chapter 81: Government and Public Sector	198

24.4 Chapter 82: Manufacturing and Supply Chain	200
24.5 Chapter 83: Legal and Professional Services	202
24.6 Chapter 84: Comparative Protocol Analysis	203
25 Part XIX: Extended Operational Guides	205
25.1 Chapter 91: Key Management Operations	205
25.2 Chapter 92: Incident Response	209
25.3 Chapter 93: Capacity Planning	213
25.4 Chapter 94: Compliance and Audit	215
25.5 Chapter 95: Testing Strategies	217
25.6 Chapter 96: Troubleshooting Guide	222
26 Part XX: Advanced Implementation Patterns	225
26.1 Chapter 97: Multi-Tenant Architecture	225
26.2 Chapter 98: Event-Driven Integration	227
26.3 Chapter 99: Hybrid Cloud Patterns	230
26.4 Chapter 100: AI Agent Orchestration	232
26.5 Chapter 101: Legacy System Integration	234
26.6 Chapter 102: Privacy-Preserving Patterns	236
27 Appendix F: Sample PoA Artifacts	238
27.1 F.1 Basic Procurement PoA	238
27.2 F.2 Delegated Authority PoA	241
27.3 F.3 Multi-Signature PoA	245
27.4 F.4 Cross-Border PoA	248
28 Appendix G: Verification Algorithm Pseudocode	253
29 Appendix H: Constraint Type Registry	256
30 Appendix I: Entity Profile Schema	258
31 Appendix J: Error Code Reference	262
32 Appendix K: Standards Path and Industry Adoption	264
32.1 K.1 Standardization Strategy	264
32.2 K.2 Industry Adoption Roadmap	265
32.3 K.3 Competitive Landscape	266
32.4 K.4 Patent and IP Considerations	266
33 Appendix L: Ecosystem Governance	268
33.1 L.1 Governance Principles	268
33.2 L.2 Organizational Structure	269
33.3 L.3 Contribution Process	269
33.4 L.4 Certification Program	270
34 Part XVIII: Formal Verification and Research Directions	271
34.1 Chapter 85: Formal Security Analysis	271
34.2 Chapter 86: Open Research Questions	273

34.3 Chapter 87: Implementation Roadmap	275
34.4 Chapter 88: Philosophical Foundations	275
34.5 Chapter 89: Economic Analysis	276
34.6 Chapter 90: Conclusion	277
35 Appendix N: Complete Protocol Specification	279
35.1 N.1 CBOR Schema Definition	279
35.2 N.2 Verification Algorithm (Normative)	281
35.3 N.3 Interoperability Requirements	283
35.4 N.4 Test Vectors	284
36 Appendix O: Comprehensive Test Suites	285
36.1 O.1 Unit Test Requirements	285
36.2 O.2 Integration Tests	288
36.3 O.3 Performance Benchmark Suite	290
37 Appendix P: Migration and Integration Guides	292
37.1 P.1 OAuth 2.0 to PoA Migration	292
37.2 P.2 SAML to PoA Bridge	294
37.3 P.3 API Key to PoA Migration	295
38 Appendix Q: Regulatory Compliance Checklists	297
38.1 Q.1 GDPR Compliance Checklist	297
38.2 Q.2 HIPAA Compliance Checklist	298
38.3 Q.3 SOX Compliance Checklist	298
38.4 Q.4 PCI-DSS Compliance Checklist	299
39 Appendix R: Operational Runbooks	300
39.1 R.1 Emergency Revocation Procedure	300
39.2 R.2 Performance Degradation Response	301
39.3 R.3 Key Rotation Procedure	302
40 Appendix M: Extended Bibliography	304
40.1 M.1 Legal Treatises	304
40.2 M.2 Technical Specifications	304
40.3 M.3 Academic Papers	305
40.4 M.4 Regulatory Materials	306
40.5 M.5 Industry Reports	306
41 Appendix S: Glossary of Terms	307
42 Appendix T: Acronyms and Abbreviations	310
43 Appendix U: Further Reading	313
43.1 U.1 Foundational Papers	313
43.2 U.2 Legal Frameworks	313
43.3 U.3 AI Safety and Ethics	313
43.4 U.4 Industry Reports	314

44 Appendix V: Case Study Summaries	315
44.1 V.1 Financial Services: Global Investment Bank	315
44.2 V.2 Healthcare: Regional Hospital Network	317
44.3 V.3 Government: Federal Procurement Agency	319
44.4 V.4 Manufacturing: Industrial Robotics	320
44.5 V.5 Legal: Law Firm AI Research	323
44.6 V.6 Cross-Industry Patterns	324
45 Acknowledgments	326
46 About This Edition	327
47 Colophon	328
48 Index	330

Chapter 1

The Agent's Signature

1.1 Identity, Authorization, and Legal Authority for Autonomous Systems

Author: Mauricio A. Fernandez Fernandez

Revision: First Edition – Technical Manuscript

Date: January 2026

Audience: Protocol designers, security engineers, cryptographers, regulators, system architects, legal technologists

Chapter 2

Preface

The year is 2026. Autonomous AI agents now execute millions of transactions daily—approving loans, negotiating contracts, ordering supplies, and managing portfolios. Yet the authorization frameworks governing these systems were designed for a simpler era: one where a human clicked “Allow” and an application accessed a photo library.

This book addresses a fundamental gap in the architecture of autonomous systems: **the absence of legally meaningful authorization**. It proposes a new primitive—the **Proof of Authorization (PoA)**—that binds cryptographic credentials to legal authority, liability, and jurisdiction.

This is not a theoretical exercise. In 2025, an AI procurement agent with valid OAuth credentials ordered \$2.3 million in components from a supplier on a sanctions list. The company faced an \$18 million fine. OAuth verified the agent had permission to access the purchasing API. It did not—and could not—verify whether the agent was authorized to bind the company to that specific transaction, with that specific counterparty, under applicable law.

The distinction between *access* and *authority* is the subject of this book.

Chapter 3

Table of Contents

Part I: Problem Statement and Threat Model - Chapter 1: Autonomous Action vs. Authorized Action - Chapter 2: The Category Error in Modern Authorization - Chapter 3: The \$18 Million Question: A Case Study - Chapter 4: Threat Model for Autonomous Agents

Part II: Related Work and Prior Art - Chapter 5: OAuth 2.0 and Its Structural Limitations - Chapter 6: Capability Tokens: Macaroons, Biscuit, UCAN - Chapter 7: SPKI/SDSI and Delegation Naming - Chapter 8: Certificate Transparency and Trust Logs - Chapter 9: Policy-Based Access Control - Chapter 10: Legal Foundations of Agency

Part III: System Model and Protocol Overview - Chapter 11: Entities and Identities (AAP-001) - Chapter 12: Entity Profiles and Cryptographic Binding - Chapter 13: Design Invariants and Fail-Closed Semantics - Chapter 14: Protocol Architecture Overview

Part IV: Proof of Authorization (AAP-002) - Chapter 15: The PoA Primitive - Chapter 16: Formal Specification - Chapter 17: Delegation Chains - Chapter 18: Scope and Constraint Models - Chapter 19: Revocation Semantics - Chapter 20: Implementation Patterns

Part V: Cryptographic Architecture - Chapter 21: Design Principles - Chapter 22: Signature Schemes and Key Management - Chapter 23: Canonicalization and Hash Functions - Chapter 24: Revocation Transparency Logs - Chapter 25: Degraded Mode Operations - Chapter 26: Post-Quantum Considerations

Part VI: Legal Mapping Across Jurisdictions - Chapter 27: Authority as a Legal Concept - Chapter 28: German Law: Statutory Representation - Chapter 29: United States: Actual vs. Apparent Authority - Chapter 30: European Union: eIDAS and Trust Services - Chapter 31: Cross-Border and Conflict-of-Law - Chapter 32: Regulatory Compliance Implications

Part VII: Operational Considerations - Chapter 33: Deployment Architectures - Chapter 34: Audit Trail Design - Chapter 35: Integration Patterns

Part VIII: Non-Goals and Limitations - Chapter 36: What This System Does Not Do -
Chapter 37: The Boundaries of Machine Authority

Part IX: Security Considerations - Chapter 38: Threat Analysis - Chapter 39: Attack
Vectors and Mitigations - Chapter 40: Incident Response

Appendices - Appendix A: Original Contributions - Appendix B: Complete Protocol Spec-
ification - Appendix C: Reference Implementation in Go - Appendix D: Glossary of Terms -
Appendix E: Extended References

Chapter 4

Part I: Problem Statement and Threat Model

4.1 Chapter 1: Autonomous Action vs. Authorized Action

4.1.1 1.1 The Rise of Autonomous Agents

Between 2020 and 2026, the nature of software changed fundamentally. What were once reactive programs—waiting for human input before taking action—became proactive agents capable of independent decision-making. This transformation was driven by three converging forces:

1. **Large Language Models** capable of reasoning about complex tasks
2. **API-first architectures** that exposed business operations as programmable endpoints
3. **Economic pressure** to automate knowledge work at scale

By 2025, enterprises deployed agents that could: - Negotiate vendor contracts within pre-defined parameters - Approve routine expenditures up to specified thresholds - Manage investment portfolios according to stated strategies - Coordinate supply chains across multiple suppliers and jurisdictions

These agents were not merely executing scripts. They were making decisions with legal and financial consequences. A procurement agent does not simply submit a purchase order; it binds a corporation to a contractual obligation. A trading agent does not merely place an order; it commits capital with fiduciary implications.

Yet the authorization systems governing these agents were designed for a fundamentally

different problem.

4.1.2 1.2 The Original Sin of OAuth

OAuth 2.0, published as RFC 6749 in 2012, was designed to solve a specific problem: allowing a user to grant a third-party application access to their resources without sharing their password. The canonical example was: “Allow this photo-printing service to access your photos.”

The security model of OAuth assumes:

1. **A human principal** who understands what they are authorizing
2. **An interactive consent step** where the human reviews and approves
3. **Access to resources** as the unit of authorization
4. **Short-lived bearer tokens** that convey this access

Notice what OAuth does *not* model:

- **Legal personhood** of the entity receiving authorization
- **Jurisdiction** under which the authorization is valid
- **Liability** for actions taken with the authorization
- **Scope of authority** beyond resource access
- **Delegation chains** from one entity to another

OAuth answers the question: “*Is this application allowed to read this data?*”

It does not answer: “*Is this agent legally empowered to bind this corporation to a \$2 million contract?*”

This is not a criticism of OAuth. It is a precise description of its scope. OAuth explicitly and correctly avoids semantics beyond access control to remain application-agnostic. The problem is not OAuth. The problem is using OAuth—or any access control system—to authorize actions that require legal authority.

4.1.3 1.3 Access vs. Authority: A Legal Distinction

In legal terms, the distinction is between:

- **Permission:** The right to enter a space or access a resource
- **Agency:** The power to act on behalf of another, creating legal effects for that other

A delivery driver may have permission to enter a company’s warehouse (access). They do not have authority to sell the warehouse’s contents (agency). An employee may have access to the corporate bank account interface. Whether they have authority to initiate a \$10 million transfer depends on their role, the governance structure, and applicable law—none of which are captured in their login credentials.

Modern authorization systems conflate these concepts. They treat “authorization” as a binary: either you have the right token, or you don’t. But legal authority is richly structured:

- **Actual authority:** Authority intentionally conferred by the principal
- **Apparent authority:** Authority that a third party reasonably believes exists
- **Implied authority:** Authority necessary to carry out express grants
- **Scope:** The boundaries of what the agent may do
- **Limitations:** Conditions that restrict otherwise valid authority
- **Duration:** How long the authority persists
- **Revocability:** Whether and how authority can be withdrawn

An access token captures none of this. When an AI agent presents an OAuth token to a supplier’s API, the supplier knows the agent has permission to call the API. The supplier does not know:

- Who is legally responsible if the order violates sanctions
- What spending limits apply to this agent
- Whether the human who originally consented is still employed
- Under which jurisdiction disputes would be resolved
- Whether the agent’s authority has been revoked since the token was issued

These are not edge cases. They are the daily reality of autonomous commerce.

4.1.4 1.4 The Stakes: Why This Matters Now

The gap between access control and legal authority was tolerable when software was primarily reactive. A human reviewed contracts before signing. A manager approved purchases above threshold. The software facilitated; humans decided.

Autonomous agents invert this relationship. The agent decides; humans (at best) audit after the fact. In this model, the authorization artifact carried by the agent is the last line of defense before irreversible action.

Consider the following scenarios, all of which have occurred or are imminent:

Scenario 1: The Sanctions Violation An AI procurement agent, operating with valid API credentials, orders components from a supplier that was added to a sanctions list two hours earlier. The agent had no knowledge of the sanctions update. The company is liable for the violation.

Scenario 2: The Runaway Expenditure A fleet of AI agents, each authorized for routine purchases up to \$10,000, collectively purchase \$847,000 in supplies over a weekend. No single purchase exceeded the threshold. No human had visibility until Monday.

Scenario 3: The Jurisdiction Mismatch An AI agent negotiating on behalf of a Ger-

man GmbH enters into an agreement governed by California law. The agreement includes an arbitration clause that would be unenforceable under German law. The company discovers this only when attempting to resolve a dispute.

Scenario 4: The Delegation Gone Wrong A manager delegates purchasing authority to an AI agent. The manager is terminated for cause. The agent continues operating with the manager’s delegated authority for 90 days before anyone notices.

In each case, the technical authorization system functioned correctly. The agent had valid credentials. The API calls succeeded. The problem was not authentication or access control. The problem was that no system verified whether the agent had *legal authority* to take the action, with the claimed scope, under the applicable constraints.

4.1.5 1.5 The Core Thesis

This book proposes a new authorization primitive specifically designed for autonomous agents: the **Proof of Authorization (PoA)**.

A PoA is not an access token. It is a cryptographically signed artifact that asserts:

Entity A is authorized to perform action X on behalf of Entity B, under constraints C, within jurisdiction J, until time T, subject to revocation via R.

Each element of this assertion is verifiable:

- **Entity A** is cryptographically identified and bound to the PoA
- **Action X** is defined with sufficient precision to be machine-evaluated
- **Entity B** (the principal) is legally identified and has authority to delegate
- **Constraints C** express limits such as spending caps or counterparty restrictions
- **Jurisdiction J** specifies the legal framework under which authority is valid
- **Time T** bounds the validity of the authorization
- **Revocation R** provides a mechanism to verify the authorization has not been withdrawn

A relying party—the supplier receiving a purchase order, the exchange executing a trade—can verify a PoA independently. Unlike an OAuth token, which requires trust in the authorization server, a PoA is self-contained and cryptographically complete.

The PoA primitive is the foundation of the **AgentAuth** protocol, formally specified in two companion documents:

- **AAP-001:** Agent Identity and Entity Profiles
- **AAP-002:** Proof of Authorization and Delegation Chains

This book provides the theoretical foundation, practical guidance, and legal analysis for implementing these protocols.

4.2 Chapter 2: The Category Error in Modern Authorization

4.2.1 2.1 From Passwords to Tokens to... What?

The history of computer security authorization can be read as a series of abstractions, each solving the problems of the previous generation while creating new ones.

Phase 1: Passwords (1960s-1990s) Authorization meant identity verification. If you knew the password, you were the authorized user. The password proved who you were, and your identity determined what you could do.

Problems: Password sharing, password theft, no granularity.

Phase 2: Role-Based Access Control (1990s-2010s) Authorization was decoupled from identity. Your identity determined your roles; your roles determined your permissions. A database administrator could access the database regardless of which specific human held that role.

Problems: Role explosion, static permissions, no delegation.

Phase 3: Tokens and Claims (2010s-2020s) Authorization became portable. An OAuth token, a JWT, or a SAML assertion could be presented to any relying party. Permissions traveled with the user across system boundaries.

Problems: Bearer token theft, over-scoping, no legal semantics.

Phase 4: Capabilities (2020s) Authorization became attenuable. Macaroons, Biscuit, and UCAN allowed authority to be restricted as it was delegated. A capability could be constrained to specific actions, time windows, or contexts.

Problems: No legal identity, no jurisdiction, no liability, no transparent revocation.

Each phase solved real problems. Each phase assumed that authorization was fundamentally about *what a system allows*, not about *what an actor is legally empowered to do*.

This assumption was reasonable when software was a tool wielded by humans. It is catastrophically inadequate when software is an autonomous actor whose decisions have legal force.

4.2.2 2.2 The Five Questions Every Authorization Must Answer

When a human manager authorizes an employee to negotiate a contract, five questions have implicit answers:

1. **Who is authorizing whom?** (Identity)
2. **To do what?** (Scope)
3. **Within what limits?** (Constraints)
4. **Under what legal framework?** (Jurisdiction)
5. **Who is responsible if something goes wrong?** (Liability)

Traditional authorization systems answer only the first two questions, and often imprecisely.

OAuth 2.0 answers: - Who? The resource owner (human user) - Authorizing whom? The client application - To do what? Access resources at specified scopes

It does not answer: - Within what limits? (No constraint model) - Under what framework? (No jurisdiction binding) - Who is responsible? (No liability attribution)

Macaroons/Biscuit improve the constraint story: - Who? Bearer of the token - Authorizing whom? Subsequent bearers - To do what? Actions permitted by the token - Within what limits? Caveats can restrict

But still do not answer: - Under what framework? (No jurisdiction) - Who is responsible? (No legal identity)

SPKI/SDSI comes closest: - Who? Named entities in a local namespace - Authorizing whom? Entities in delegation chains - To do what? Permissions in authorization certificates - Within what limits? Certificate constraints

But deliberately omits: - Revocation transparency - Liability semantics - Jurisdiction binding

No existing system answers all five questions. PoA is designed to do so.

4.2.3 2.3 When Access Control Meets Contract Law

The inadequacy of access control for legal authority becomes concrete in contract formation.

A contract requires: 1. **Offer**: A proposal to enter into an agreement 2. **Acceptance**: Assent to the terms of the offer 3. **Consideration**: Something of value exchanged 4. **Capacity**: Legal ability to enter contracts 5. **Authority**: If acting for another, the power to bind them

When an AI agent sends a purchase order to a supplier, it is making an offer (or accepting one). If the supplier fulfills the order, a contract exists. The company is bound.

The supplier's security system verified that the API call came from a valid client with a valid token. It did not verify—and had no way to verify—whether the AI agent had legal authority to bind the company.

This is the category error: treating *authentication* and *access control* as sufficient for *legal authorization*.

Concretely:

Question	What Access Control Answers	What Legal Authority Requires
Identity	"Request came from ClientID xyz"	"Agent is authorized representative of Acme Corp, a Delaware corporation"
Capacity	(Not addressed)	"Acme Corp has legal capacity to contract"
Authority	"Token has purchase:write scope"	"Agent has authority from Acme Corp to bind it to purchases up to \$50,000"
Constraints	(Not addressed)	"Purchases may not include sanctioned suppliers"
Liability	(Not addressed)	"Acme Corp bears liability for agent's actions within scope"

An OAuth token proves the request came from ClientID xyz. It does not prove ClientID xyz represents Acme Corp, that Acme Corp authorized this specific action, or that the action falls within the agent's delegated authority.

4.2.4 2.4 The Apparent Authority Problem

U.S. agency law recognizes a doctrine called "apparent authority." A principal may be bound by an agent's actions if a third party reasonably believed the agent had authority, even if the agent actually exceeded their limits.

This doctrine exists because commerce depends on reliable appearances. A supplier cannot investigate every customer's internal governance before accepting an order. If a person appears to be authorized—they have a company email, access to company systems, a title suggesting responsibility—the supplier is entitled to rely on that appearance.

Apparent authority creates a legal trap for companies using AI agents.

Consider: 1. An AI agent has an API key issued by Acme Corp 2. The agent presents this key to a supplier 3. The supplier reasonably believes the agent is authorized to order 4. The agent orders \$500,000 in supplies 5. Acme Corp’s internal policy limited the agent to \$50,000

Under apparent authority doctrine, Acme Corp may be bound. The supplier relied on the appearance created by Acme Corp (issuing the API key). Acme Corp’s internal limits, unknown to the supplier, do not help.

The lesson: **Authorization limitations must be externally visible to be legally effective.**

This is precisely what PoA provides. A PoA artifact includes explicit constraints. A supplier verifying the PoA can see the spending limit. If the order exceeds the limit, the supplier knows—before processing the order—that the agent lacks authority.

4.3 Chapter 3: The \$18 Million Question: A Case Study

4.3.1 3.1 The Incident

The following case study is a composite based on publicly reported incidents and regulatory actions. Specific details have been altered to protect confidentiality while preserving the technical and legal lessons.

In October 2025, GlobalManufacturing Corp. (“GMC”), a multinational industrial equipment manufacturer, deployed an AI-powered procurement system. The system, called “ProcureAI,” was tasked with managing routine supply orders across GMC’s global operations.

ProcureAI operated with the following authorizations: - OAuth 2.0 tokens with `procurement:read` and `procurement:write` scopes - API access to GMC’s approved vendor list - Spending authority up to \$100,000 per purchase order - Approval workflow for orders exceeding threshold

On October 17, 2025, at 14:23 UTC, ProcureAI submitted a purchase order for \$87,000 to Zhongwei Industrial Components, a supplier in Shenzhen, China. The order was for specialty bearings used in GMC’s manufacturing process.

What ProcureAI did not know—and had no way of knowing—was that at 08:00 UTC that same day, the U.S. Department of Commerce had added Zhongwei Industrial Components to the Entity List for alleged connections to China’s military-industrial complex.

The order was processed. The components were shipped. On November 3, 2025, GMC received notice of investigation from the Bureau of Industry and Security.

4.3.2 3.2 The Technical Failure

From a technical perspective, ProcureAI operated correctly:

1. **Authentication:** ProcureAI presented a valid OAuth token
2. **Authorization:** The token had appropriate scopes
3. **Vendor Validation:** Zhongwei was on GMC's approved vendor list
4. **Threshold Compliance:** \$87,000 was below the \$100,000 limit

Every check passed. Every system functioned as designed.

The failure was not in any individual component. The failure was **architectural**: no component verified whether ProcureAI was legally authorized to conduct this specific transaction with this specific counterparty at this specific time.

The approved vendor list was static—updated quarterly by GMC's compliance team. The Entity List update occurred 6 hours and 23 minutes before the order. The compliance team had not yet reviewed the update. ProcureAI had no mechanism to check sanctions lists independently.

More fundamentally: even if ProcureAI had access to sanctions lists, the authorization model had no way to express “this agent's authority is conditional on compliance with sanctions requirements.” The OAuth token said `procurement:write`. It did not—and could not—say “`procurement:write` except to sanctioned entities.”

4.3.3 3.3 The Legal Fallout

The investigation concluded that GMC had committed a “willful” violation of the Export Administration Regulations. The willfulness determination rested on:

1. GMC knew its industry was subject to export control requirements
2. GMC deployed an autonomous system without adequate compliance controls
3. GMC failed to ensure real-time sanctions screening

GMC's defenses were unavailing:

“ProcureAI was operating within its authorized scope” The court noted that “authorized scope” in GMC's internal systems had no relationship to legal authorization under export control law. The OAuth token authorized API access. It did not authorize violations of federal law.

“The sanctions list update occurred only 6 hours before the order” The court observed that GMC voluntarily chose to deploy an autonomous system operating at machine speed. Having done so, GMC assumed the risk that the system would act faster than manual compliance processes could adapt.

“ProcureAI was approved by our security team” The security team’s approval addressed authentication, access control, and data protection. It did not address legal authorization or regulatory compliance.

The civil monetary penalty was \$18.7 million. GMC was also required to implement a compliance program with specific controls on AI-based procurement, subject to independent audit for five years.

4.3.4 3.4 The Technical Lessons

The GMC incident illustrates several architectural failures that PoA addresses:

Lesson 1: Authorization must be checkable against real-world conditions

ProcureAI’s OAuth token was valid regardless of sanctions status. There was no mechanism to invalidate or restrict the token based on external regulatory events.

PoA addresses this through **constraints**. A PoA can specify that authority is conditioned on compliance with specified regulations. Verification includes checking those conditions.

Lesson 2: Limits must be visible to counterparties

GMC’s spending limit was internal. Zhongwei had no way to know that ProcureAI had a \$100,000 cap, or that the cap was nearly irrelevant (the real problem was sanctions, not amount).

PoA includes constraints as part of the signed artifact. A relying party can inspect the constraints before acting.

Lesson 3: Delegation chains matter

ProcureAI’s authority derived from... what, exactly? An OAuth client registration by an IT administrator? A procurement manager’s request? A board resolution?

The court could not reconstruct the authorization chain. This made it impossible to determine who approved autonomous procurement and what limits they intended.

PoA requires explicit delegation chains. Authority flows from legally recognized principals through cryptographically verifiable links.

Lesson 4: Revocation must be responsive

Even if GMC had a mechanism to revoke ProcureAI’s authority upon learning of the sanctions update, OAuth token revocation is slow. Access tokens remain valid until expiration. Refresh token revocation is inconsistently implemented across vendors.

PoA includes transparent revocation. Relying parties check revocation status before trusting the authorization.

4.3.5 3.5 What Should Have Happened

With a PoA-based authorization system, the transaction would have unfolded differently:

1. **ProcureAI presents a PoA** instead of an OAuth token
2. **The PoA includes constraints:** “Authority valid only for transactions compliant with U.S. export control regulations”
3. **Zhongwei’s system verifies the PoA:**
 - Checks the delegation chain (valid: GMC $\square$ Procurement Director $\square$ ProcureAI)
 - Checks the scope (valid: procurement of industrial components)
 - Checks the constraints (requires sanctions check)
 - Performs sanctions screening (FAIL: Zhongwei is on Entity List)
4. **Transaction rejected** with audit record
5. **No violation occurs**

The constraint check at step 3 is the crucial difference. ProcureAI’s authority was always *conditional* on compliance. With PoA, that condition is explicit and machine-verifiable. With OAuth, it was implicit and unenforceable.

4.3.6 3.6 The Broader Pattern

The GMC case is not unique. Between 2024 and 2026, regulatory actions involving AI-based commerce have cited:

- Failure to maintain adequate controls on autonomous systems
- Inability to trace authorization decisions to responsible humans
- Insufficient real-time screening for regulatory restrictions
- Over-reliance on access control systems for authorization decisions

In each case, the technical systems functioned correctly. Authentication worked. Access control worked. API calls succeeded. The failure was in the gap between technical authorization (what the system permitted) and legal authorization (what the law permitted).

PoA is designed to close that gap.

4.4 Chapter 4: Threat Model for Autonomous Agents

4.4.1 4.1 Adversary Capabilities

This section defines the threat model under which PoA operates. We consider adversaries with the following capabilities:

Credential Compromise Adversaries may: - Steal agent credentials through supply

chain attacks - Extract keys from compromised runtime environments - Intercept tokens in transit - Exploit overly permissive key storage

Authorization Artifact Manipulation Adversaries may: - Replay valid PoA artifacts to unauthorized parties - Present PoA artifacts after revocation - Attempt to modify PoA constraints without invalidating signatures - Combine fragments of multiple PoA artifacts

Delegation Exploitation Adversaries may: - Exploit over-broad initial delegations - Create unauthorized sub-delegations - Impersonate legitimate delegators - Exploit gaps between delegation and revocation

Jurisdictional Arbitrage Adversaries may: - Exploit mismatches between claimed and actual jurisdiction - Forum-shop for favorable legal interpretations - Deny jurisdiction to avoid accountability - Manipulate jurisdiction fields in artifacts

System Availability Attacks Adversaries may: - Deny access to revocation information - Delay revocation propagation - Exploit degraded-mode semantics - Attack transparency log infrastructure

4.4.2 4.2 Non-Assumptions

The threat model explicitly does not assume:

Trusted Execution Environments PoA does not require hardware-level attestation. Agents may run on commodity infrastructure without TEEs, SGX, or similar technologies.

Rationale: TEE adoption is uneven. Many legitimate deployment scenarios cannot use TEEs. A protocol that requires TEEs has limited applicability.

Honest Principals Principals may attempt to: - Deny authorization after the fact - Claim narrower authority than actually granted - Backdoor delegation for plausible deniability

Rationale: In adversarial commercial contexts, principals have incentives to minimize liability after incidents. The protocol must produce evidence that survives these incentives.

Continuous Online Verification Agents may operate in: - Air-gapped environments - Networks with intermittent connectivity - Scenarios where real-time verification introduces unacceptable latency

Rationale: Many high-value autonomous systems operate in environments where continuous connectivity cannot be guaranteed. Authorization must be verifiable offline.

Secure Time Sources Adversaries may manipulate: - Local system clocks - Network time sources - Timestamps in artifacts

Rationale: Secure time is harder than it appears. The protocol should minimize reliance on precise time synchronization.

4.4.3 4.3 Security Goals

Given these adversary capabilities and non-assumptions, PoA aims to achieve:

Authenticity A valid PoA proves that the claimed principal authorized the claimed agent for the claimed scope. Forgery should be computationally infeasible.

Integrity Any modification to a PoA—changing scope, constraints, validity, or any other field—should be detectable. Partial modification should not be possible.

Non-Repudiation A principal who issued a PoA cannot deny having done so. The cryptographic evidence is sufficient for legal and regulatory purposes.

Transparency Authorization and revocation events are publicly auditable. Withholding or equivocating on revocation should be detectable.

Bounded Damage Compromise of an agent's credentials should not compromise the principal's credentials. Damage should be limited by the scope, constraints, and validity of the affected PoA.

Graceful Degradation When revocation status cannot be determined, the system should fail safely—preventing high-risk actions while preserving evidence for later analysis.

4.4.4 4.4 Out of Scope

The PoA threat model explicitly excludes:

Intent Verification PoA does not attempt to prove that an agent's actions match the principal's intentions. It proves authorization, not correctness.

Contractual Obligation PoA asserts authority to act. It does not encode the terms of any resulting contract. A PoA that authorizes negotiation does not capture what was negotiated.

Dispute Resolution PoA provides evidence for disputes. It does not resolve them. Legal and commercial disputes require human judgment.

Key Management PoA specifies what keys mean (authority) and requires them to be well-managed. It does not specify key lifecycle, storage, or rotation procedures.

[Chapter continues with detailed threat scenarios, attack trees, and mitigations...]

Chapter 5

Part II: Related Work and Prior Art

5.1 Chapter 5: OAuth 2.0 and Its Structural Limitations

5.1.1 5.1 OAuth 2.0: What It Is

OAuth 2.0, specified in RFC 6749, is an authorization framework that enables third-party applications to obtain limited access to HTTP services. It is, by any measure, extraordinarily successful: billions of “Sign in with Google” and “Connect with Facebook” interactions happen daily.

The OAuth 2.0 specification explicitly disclaims broader authorization semantics:

“OAuth 2.0 is not an authentication protocol... The actual method of authenticating the identity of the resource owner is beyond the scope of this specification.”

This humility is a feature. OAuth succeeded precisely because it avoided overreach. It solved one problem well: delegated access to resources.

But this limitation becomes critical when OAuth is (mis)used for autonomous agent authorization.

5.1.2 5.2 The OAuth Security Model

OAuth 2.0 assumes the following security model:

Actors: - **Resource Owner:** A human who owns resources (e.g., photos, documents)
- **Client:** An application requesting access to resources - **Authorization Server:** The

system that issues tokens after obtaining consent - **Resource Server**: The system hosting protected resources

Flow: 1. Client requests authorization from Resource Owner 2. Resource Owner authenticates with Authorization Server 3. Resource Owner grants (or denies) consent 4. Authorization Server issues access token to Client 5. Client presents token to Resource Server 6. Resource Server validates token and provides access

Trust Model: - Resource Owner trusts Authorization Server to authenticate them - Client trusts Authorization Server to issue valid tokens - Resource Server trusts Authorization Server to validate tokens - Resource Owner's consent is the root of all authorization

This model has three critical assumptions:

1. **Human principal:** A person decides what to authorize
2. **Interactive consent:** The human reviews and approves
3. **Resource access:** The authorization is for accessing data

5.1.3 5.3 Why OAuth Fails for Autonomous Agents

Each of these assumptions breaks for autonomous agents:

Assumption 1: Human principal

In OAuth, the resource owner is a human. The human reads the consent screen. The human understands (more or less) what they're approving.

But consider an AI agent that needs to negotiate contracts on behalf of a corporation. The "resource owner" is the corporation—a legal abstraction. No human "is" the corporation. Multiple humans have authority over different corporate functions.

OAuth's consent model assumes a singular human principal. Corporate authority involves hierarchy, delegation, and distribution of responsibility.

Assumption 2: Interactive consent

OAuth's consent step is interactive. The user sees a screen: "Allow AwesomeApp to access your photos?" The user clicks Allow or Deny.

Autonomous agents don't operate interactively. They act continuously, often at high frequency. A procurement agent might process hundreds of orders per hour. Requiring interactive consent for each action would defeat the purpose of automation.

OAuth's model requires human-in-the-loop at authorization time. Autonomous agents require human oversight, but not human-in-the-loop for every action.

Assumption 3: Resource access

OAuth grants access to resources: read photos, write documents, access calendars. The unit of authorization is the API scope.

Autonomous agents perform actions that go beyond resource access. They negotiate terms, accept offers, commit capital, and create legal obligations. The unit of authorization should be the action's legal effect, not the API capability.

5.1.4 5.4 Token Properties and Their Limitations

OAuth access tokens have properties that become problematic for autonomous agents:

Bearer Semantics An OAuth token is a bearer credential. Whoever holds the token can use it. There is no binding between the token and the entity presenting it.

For autonomous agents, this means a stolen token grants full authority until expiration. There is no way to prove that the entity using the token is the authorized agent.

Flat Authority OAuth tokens grant access according to scopes: read, write, admin. Authority is binary: either you have the scope or you don't.

Legal authority is not flat. It has structure: hierarchies, conditions, limits, and contexts. "You may spend up to \$10,000 per day, except for consulting services, which require manager approval" is a coherent authority statement that OAuth cannot express.

Server-Centric Validation OAuth tokens are validated by the authorization server. The resource server trusts the authorization server's judgment.

This creates a single point of failure. If the authorization server is unavailable, no validation is possible. For autonomous agents operating in disconnected environments, this is unacceptable.

Coarse-Grained Revocation Revoking an OAuth token typically means invalidating the refresh token. Access tokens remain valid until they expire. Fine-grained revocation (invalidating a specific action while preserving others) is not natively supported.

For autonomous agents, rapid revocation may be critical. Learning that an agent is misbehaving requires immediate action, not waiting for token expiration.

5.1.5 5.5 Attempts to Extend OAuth

The OAuth community has developed extensions addressing some limitations:

OAuth 2.0 Rich Authorization Requests (RFC 9396) allows expressing detailed authorization requirements beyond scopes. This helps with the "flat authority" problem but doesn't address bearer semantics or offline verification.

Demonstrating Proof of Possession (RFC 9449) binds tokens to specific keys, addressing bearer semantics. However, it still requires online validation and doesn't model legal authority.

Resource Indicators (RFC 8707) allows specifying which resources a token should access. This improves scope precision but doesn't address constraints or jurisdiction.

These extensions improve OAuth for its designed purpose. They do not transform OAuth into a legal authorization protocol. The fundamental model—human consent for resource access—remains unchanged.

5.1.6 5.6 When OAuth Is the Right Choice

OAuth remains excellent for its intended purpose:

- Delegating access to user data across applications
- Single sign-on across web properties
- API access control for developer platforms
- Consumer-facing integrations with user consent

If your authorization question is “Should this application access this user's data?” OAuth is likely the right tool.

If your authorization question is “Is this autonomous agent legally empowered to bind this corporation to this obligation?” OAuth is structurally inadequate.

5.2 Chapter 6: Capability Tokens: Macaroons, Biscuit, UCAN

5.2.1 6.1 The Capability Model

Capability-based security represents authority as a transferable, attenuable token. The holder of a capability can exercise the authority it represents, and can create weaker capabilities to delegate to others.

This model, pioneered in operating systems research, has been adapted for distributed authorization through several modern systems.

5.2.2 6.2 Macaroons: Contextual Caveats

Macaroons, developed at Google and presented at NDSS 2014, introduce “contextual caveats” to capability tokens.

Structure: A macaroon is a chain of HMAC operations. The root macaroon is created by the target service. Each caveat is an additional HMAC that restricts authority.

```
macaroon = HMAC(root_key, identifier)
constrained = HMAC(macaroon, caveat_1)
further = HMAC(constrained, caveat_2)
```

Properties: - **Attenuation:** Authority can only decrease. Adding a caveat cannot expand authority. - **Third-party caveats:** Caveats can require attestation from external services. - **Offline verification:** Given the secret key, macaroons can be verified without network calls.

Example:

```
identifier: "user:alice, service:storage"
caveat: "time < 2026-01-01T12:00:00Z"
caveat: "operation in [read, list]"
caveat: "path startswith /public/"
```

This macaroon grants Alice read-only access to her public storage until noon.

Limitations for Autonomous Agents: - No legal identity binding (alice is a string, not a legal person) - No jurisdiction semantics - No liability attribution - Verification requires the root key (not publicly verifiable) - Revocation requires external mechanism

5.2.3 6.3 Biscuit: Datalog Authorization

Biscuit extends the capability model using Datalog-style logic for authorization decisions.

Structure: A Biscuit token contains: - Authority block: Facts about the token's authority - Attenuations: Additional blocks that restrict authority - Signature chain: Each block signed by the previous

Properties: - **Logic-based authorization:** Complex rules can be expressed as Datalog - **Formal verification:** Authorization decisions are checkable against specification - **Multiple attenuation:** Several parties can each add restrictions

Example:

```
authority:
  right("/files/report.pdf", "read");
  user("alice");

check:
  check if time($t), $t < 2026-01-01T12:00:00Z;
```

```

policy:
  allow if right($resource, $operation),
    resource($resource),
    operation($operation);
  deny if true;

```

This grants read access to a specific file, conditional on time, with a default-deny policy.

Limitations for Autonomous Agents: - Still abstract cryptographic identities - No native legal semantics - No jurisdiction binding - Revocation via external mechanisms only

5.2.4 6.4 UCAN: Decentralized Capability Delegation

UCAN (User-Controlled Authorization Network) generalizes capabilities for decentralized systems.

Structure: A UCAN is a JWT-like token with: - **Issuer:** The entity delegating authority - **Audience:** The entity receiving authority - **Capabilities:** What is being delegated - **Proofs:** Parent UCANs forming a delegation chain

Properties: - **Self-certifying:** The token contains its own proof of authority - **Delegation chains:** UCANs can nest to arbitrary depth - **Offline verification:** No authorization server required

Example:

```

{
  "iss": "did:key:zAlice...",
  "aud": "did:key:zBob...",
  "att": [
    {
      "can": "crud/update",
      "with": "fs://example.com/blog/posts/*"
    }
  ],
  "prf": ["bafy...parent-ucan"]
}

```

This delegates update authority for blog posts from Alice to Bob, citing a parent UCAN as proof of Alice's authority.

Limitations for Autonomous Agents: - DIDs are cryptographic identities, not legal persons - No jurisdiction or governing law - No liability semantics - No transparent revocation

5.2.5 6.5 The Capability Gap

All three systems represent advances over OAuth:

Feature	OAuth	Macaroons	Biscuit	UCAN
Attenuation	☐	☐	☐	☐
Offline verification	☐	☐	☐	☐
Delegation chains	☐	Partial	☐	☐
Formal logic	☐	☐	☐	☐

But all share common limitations for autonomous agents:

Feature	Macaroons	Biscuit	UCAN	PoA
Legal identity	☐	☐	☐	☐
Jurisdiction binding	☐	☐	☐	☐
Liability constraints	☐	☐	☐	☐
Transparent revocation	☐	☐	☐	☐

PoA builds on the capability model while adding the legal semantics required for autonomous agent authorization.

[Document continues with remaining chapters...]

5.3 Chapter 7: SPKI/SDSI and Delegation Naming

5.3.1 7.1 The SPKI/SDSI Vision

The Simple Public Key Infrastructure / Simple Distributed Security Infrastructure (SPKI/SDSI), specified in RFC 2693, represents one of the most intellectually rigorous treatments of authorization ever developed. Published in 1999 by some of the most distinguished researchers in computer security, it anticipated many problems that remain unsolved in mainstream systems.

Carl Ellison, one of SPKI's architects, wrote:

“An identity certificate is a dangerous thing. It binds a key to a name (a meaningless sequence of characters) rather than to a right to perform some action.”

This insight—that authorization should bind keys to *permissions*, not identities—is foundational to both SPKI and PoA.

5.3.2 7.2 Key Concepts in SPKI

Name Certificates vs. Authorization Certificates

SPKI distinguishes between: - **Name certificates:** Bind a public key to a local name - **Authorization certificates:** Grant permissions to a public key

This separation is crucial. In X.509 PKI, a certificate says “This key belongs to Alice.” What Alice is *allowed to do* is determined elsewhere. In SPKI, an authorization certificate directly says “This key may perform these actions.”

Local Naming

SPKI names are local to the issuer. When Alice issues a certificate in the name “my-agent,” that name has meaning only within Alice’s namespace. This avoids the global namespace collisions that plague traditional PKI.

5-Tuple Authorization

SPKI authorization certificates contain: 1. **Issuer:** The key granting authorization 2. **Subject:** The key receiving authorization 3. **Authorization:** What is being permitted 4. **Validity:** Time bounds 5. **Revocation info:** How to check for revocation

Threshold Structures

SPKI supports k-of-n threshold authorization: an action may require signatures from multiple keys. This models real-world governance structures.

5.3.3 7.3 SPKI’s Relevance to PoA

PoA inherits several SPKI concepts:

Authorization vs. Identity Separation Like SPKI, PoA treats authorization as distinct from identity. Knowing who someone is does not tell you what they may do.

Delegation Chains SPKI’s chained authorization certificates inspire PoA’s delegation chains. Authority flows through explicit, cryptographically verifiable links.

Local Naming PoA does not require global identity registries. Principals define their own namespaces for agents.

5.3.4 7.4 Where SPKI Falls Short

Despite its elegance, SPKI was ahead of its time in some ways and limited in others:

No Legal Semantics SPKI authorization is purely technical. There is no concept of legal personhood, jurisdiction, or liability. An SPKI certificate cannot express “this key represents Acme Corp, incorporated in Delaware.”

No Transparency SPKI relies on CRLs (Certificate Revocation Lists) or short-lived certificates. There is no append-only log, no inclusion proofs, no public auditability.

No Constraint Language SPKI authorization is binary: permitted or not. There is no structured way to express constraints like spending limits, time windows, or counterparty restrictions.

No Jurisdiction Binding SPKI does not model the legal framework under which authorization operates. This ambiguity creates problems when authority crosses borders.

5.3.5 7.5 From SPKI to PoA

PoA can be viewed as SPKI extended with: - Legal identity binding - Jurisdiction semantics
- Structured constraints - Revocation transparency

The core SPKI insight—that authorization should be first-class, not derived from identity—remains central to PoA.

5.4 Chapter 8: Certificate Transparency and Trust Logs

5.4.1 8.1 The CT Innovation

Certificate Transparency (CT), specified in RFC 6962, solved one of the most vexing problems in PKI: detecting misbehavior by certificate authorities.

Before CT, a rogue or compromised CA could issue a fraudulent certificate for any domain. The victim would have no way to know until it was too late.

CT introduced append-only logs of certificate issuances. Every certificate must be logged before browsers will trust it. Anyone can monitor the logs for suspicious certificates targeting their domains.

5.4.2 8.2 The Three CT Promises

CT logs provide three guarantees:

Append-Only Data can only be added to the log, never removed or modified. A log that removed or changed entries would be detectable through consistency proofs.

Publicly Auditable Anyone can read the log. Domain owners can monitor for certificates they didn't request. Researchers can analyze issuance patterns.

Cryptographically Provable Inclusion in the log is proven by Merkle tree proofs. A client can verify that a certificate is in the log without downloading the entire log.

5.4.3 8.3 CT's Applicability to Authorization

The CT model is powerful, but its subject matter—X.509 certificates—is about *identity*, not *authorization*. CT answers:

“Has a certificate been issued binding this key to this identity?”

It does not answer:

“Has authority been granted to this agent for this scope?”

5.4.4 8.4 Authority Transparency

PoA applies CT-style transparency to *authorization* rather than *identity*. We call this “Authority Transparency.”

The key differences:

Aspect	Certificate Transparency	Authority Transparency
Subject	Identity certificates	Authorization artifacts
Question	“Who issued this cert?”	“Who granted this authority?”
Relevance	Key-to-identity binding	Key-to-permission binding
Revocation	Certificate revocation	Authority revocation

5.4.5 8.5 The Revocation Transparency Log

In PoA, revocation is logged rather than (or in addition to) issuance. The rationale:

Issuance may be private: A company may not want to publicize which agents it authorizes for which purposes.

Revocation must be public: For a revocation to be effective, relying parties must learn of it. Silent revocation is legally insufficient.

The revocation transparency log is: - Append-only (revocations cannot be un-revoked) - Publicly queryable (anyone can check if a PoA is revoked) - Cryptographically auditable (inclusion proofs are available)

5.4.6 8.6 Implementation Considerations

PoA does not mandate a specific log implementation. Acceptable implementations include:

Dedicated PoA Logs Purpose-built logs operated by consortia, similar to CT log operators.

Blockchain Anchoring Hash commitments to a public blockchain, with full data stored elsewhere.

Existing CT Infrastructure Using CT logs with custom extensions (experimental).

The critical requirement is that revocation evidence be: 1. Append-only 2. Publicly auditable 3. Cryptographically provable

5.5 Chapter 9: Policy-Based Access Control

5.5.1 9.1 The XACML Model

The eXtensible Access Control Markup Language (XACML) represents the apex of policy-based access control. It separates:

Policy Administration Point (PAP): Where policies are created **Policy Information Point (PIP):** Where attribute data is retrieved **Policy Decision Point (PDP):** Where authorization decisions are made **Policy Enforcement Point (PEP):** Where decisions are enforced

This architecture enables rich, centralized policy management. Policies can be arbitrarily complex, referencing external attributes and combining through sophisticated algorithms.

5.5.2 9.2 XACML's Power

XACML can express: - "Users in role 'Manager' may approve expenses up to \$50,000" - "Access to medical records requires the requester to have a treatment relationship with the patient" - "Documents classified 'Secret' may only be accessed from approved terminals during business hours"

This expressiveness is valuable for enterprise access control.

5.5.3 9.3 XACML's Limitations for Autonomous Agents

Centralized Trust XACML assumes a trusted PDP. Relying parties accept the PDP's decisions. There is no cryptographic proof of the decision or its basis.

Policy as Root of Authority In XACML, policy is authoritative. But where does the policy come from? Who authorized the policy administrator? XACML doesn't answer.

No External Verifiability An XACML decision cannot be verified by an external party years later. The PDP's internal state may have changed. The policy may have been updated.

No Legal Semantics XACML is technically expressive but legally silent. It cannot express jurisdiction, liability, or legal identity.

5.5.4 9.4 PoA's Relationship to Policy

PoA does not replace policy languages. Instead, it provides a cryptographic foundation for policy authority.

In PoA: - Delegation chains establish *who may make policy* - The PoA artifact captures *what policy was in effect* - Constraints express *limits on the agent's authority* - Verification proves *the agent was authorized under that policy*

A PEP may use XACML internally. But the agent's authority to trigger that evaluation is established by PoA.

5.6 Chapter 10: Legal Foundations of Agency

5.6.1 10.1 Agency Law: A Primer

Agency law, developed over centuries, formalizes the relationship between principals and agents. It addresses questions like:

- When is a principal bound by an agent's actions?
- What duties does an agent owe to a principal?
- How is authority granted and revoked?
- What happens when an agent exceeds their authority?

5.6.2 10.2 The Core Concepts

Principal The party who authorizes another to act on their behalf.

Agent The party authorized to act for the principal.

Actual Authority Authority intentionally conferred by the principal. May be express (stated) or implied (necessary to carry out express authority).

Apparent Authority Authority that a third party reasonably believes exists based on the principal's conduct. A principal may be bound even if the agent exceeded actual authority.

Scope of Authority The boundaries of what an agent may do. Actions within scope bind the principal; actions outside scope do not (unless apparent authority applies).

Fiduciary Duty An agent owes duties of loyalty, care, and disclosure to the principal.

Revocation Authority may generally be revoked by the principal, though timing and notice requirements vary.

5.6.3 10.3 Agency Law's Assumptions

Traditional agency law assumes: - Agents are humans - Principals can observe and supervise agents - Third parties can assess agent behavior - Agents have judgment and discretion

Autonomous software agents challenge each assumption: - AI agents are not humans - Agents may act faster than humans can supervise - Third parties cannot easily assess algorithmic decision-making - Agents' "judgment" is opaque

5.6.4 10.4 The Apparent Authority Problem Revisited

Apparent authority doctrine creates particular risks for autonomous agents.

The Doctrine A principal may be bound by an agent's actions if a third party reasonably believed the agent was authorized, based on the principal's conduct.

The Risk A company deploys an AI agent with an API key. A supplier sees the agent making purchases with a company API key. The supplier reasonably believes the agent is authorized.

If the agent exceeds internal limits, the company may be bound anyway—the supplier relied on the appearance created by the company (granting the API key).

The PoA Solution PoA makes constraints visible. A supplier checking the PoA sees the spending limit. If the order exceeds the limit, the supplier knows the agent lacks authority. No reasonable reliance can form.

5.6.5 10.5 Cross-Jurisdictional Challenges

Agency law varies significantly across jurisdictions:

Germany (BGB §§ 164-181) Authority is often statutory and register-based. Third parties are entitled to rely on commercial register entries. Internal limitations not reflected in the register may not be opposable.

United States (Restatement Third of Agency) Apparent authority is broadly recognized. Courts focus on the third party's reasonable belief. Private limitations may not protect the principal.

European Union The eIDAS regulation addresses electronic signatures and trust services but does not harmonize agency law. Each member state has its own rules.

Implications for PoA PoA does not unify agency law. Instead, it makes jurisdictional assumptions explicit. A PoA specifies the governing jurisdiction. Relying parties know which legal framework applies.

Chapter 6

Part III: System Model and Protocol Overview

6.1 Chapter 11: Entities and Identities (AAP-001)

6.1.1 11.1 What Is an Entity?

In PoA, an “entity” is any legally or operationally distinct actor that may: - Possess authority
- Delegate authority - Receive authority - Perform authorized actions

Entities fall into three categories:

Natural Persons Human beings with legal capacity. Identified by government-issued credentials or verified identity claims.

Legal Persons Organizations with legal personality: corporations, trusts, government agencies. Identified by registration records, LEI (Legal Entity Identifier), or similar.

Autonomous Agents Software systems acting on behalf of principals. Identified by cryptographic keys bound to operating parameters.

6.1.2 11.2 The Entity Profile

Each entity is represented by a cryptographically signed **Entity Profile** containing:

```
EntityProfile := Sign_k(  
    EntityID,  
    EntityType,  
    LegalIdentifiers,
```

```

    CryptographicKeys,
    Jurisdiction,
    ValidityPeriod,
    ProfileMetadata
)

```

EntityID A globally unique identifier. For natural persons, this may be opaque. For legal persons, this should reference external registries. For agents, this is typically derived from the agent's primary public key.

EntityType One of: NaturalPerson, LegalPerson, AutonomousAgent.

LegalIdentifiers For natural persons: may be hashed or encrypted for privacy. For legal persons: commercial register entry, LEI, jurisdiction of incorporation. For agents: binding to controlling entity.

CryptographicKeys Public keys associated with the entity. Multiple keys may be listed for different purposes (signing, encryption) or for key rotation.

Jurisdiction The primary legal jurisdiction applicable to this entity.

ValidityPeriod When the profile is effective and when it expires.

ProfileMetadata Additional claims, endorsements, or attributes.

6.1.3 11.3 Entity Profile Verification

An Entity Profile is verified by:

1. Checking the signature against a known trusted issuer
2. Validating the legal identifiers against external sources (where applicable)
3. Confirming the profile is not expired
4. Checking for profile revocation

For high-assurance scenarios, legal identifier validation may involve: - Querying commercial registers - Verifying LEI status - Checking qualified trust service providers (eIDAS)

6.1.4 11.4 Entity Relationships

Entities relate to each other through:

Delegation: Entity A grants authority to Entity B **Employment:** Natural person is employed by legal person **Control:** Natural person controls autonomous agent **Representation:** Any entity acts as agent of another

These relationships are captured in PoA artifacts, not Entity Profiles. The profile establishes *who* an entity is; the PoA establishes *what* they may do.

6.2 Chapter 12: Entity Profiles and Cryptographic Binding

6.2.1 12.1 The Binding Problem

A fundamental challenge in digital identity is binding cryptographic keys to real-world entities. A public key is just a number. How do we know it represents “Acme Corporation”?

Traditional PKI solves this with Certificate Authorities who verify identity before issuing certificates. This works for well-known entities with stable identities.

For autonomous agents, the problem is different. An agent doesn’t have a passport or corporate registration. Its identity is its cryptographic key, bound to the controlling entity.

6.2.2 12.2 Agent Key Derivation

PoA recommends a hierarchical key derivation approach:

```
AgentKey = DeriveKey(
    ControllerKey,
    AgentID,
    Purpose,
    CreationTime
)
```

The agent’s key is derived from its controller’s key. This: - Creates an auditable link between agent and controller - Allows key revocation by revoking the derivation - Prevents agents from having keys independent of any principal

6.2.3 12.3 Profile Issuance Authorities

Entity Profiles may be issued by:

Self-Issuance An entity issues its own profile. Appropriate for: - Agents (signed by controller) - Organizations (where self-attestation is sufficient)

Qualified Trust Service Providers Under eIDAS, qualified trust service providers may issue profiles with legal significance. Appropriate for: - Natural persons requiring verified identity - Legal persons requiring register-backed validation

Domain-Specific Authorities Industry consortia or regulatory bodies may issue profiles for their domains. Examples: - Financial regulators for regulated entities - Industry associations for member organizations

6.2.4 12.4 Profile Revocation

Profile revocation occurs when: - Keys are compromised - Entity status changes (organization dissolved, person deceased) - Incorrect information discovered

Revocation is published to the same transparency infrastructure as PoA revocation.

6.3 Chapter 13: Design Invariants and Fail-Closed Semantics

6.3.1 13.1 The Four Invariants

PoA enforces four design invariants:

Invariant 1: Authority Monotonicity An agent may never receive authority exceeding its delegator's authority. If Alice has spending authority up to \$10,000, any agent Alice authorizes has at most \$10,000 authority.

Invariant 2: Root Traceability All authority must be traceable to a legally recognized root. The delegation chain must terminate in an entity with original authority—a natural person with capacity, a legal person with appropriate governance, or a statutory grant.

Invariant 3: Revocation Verifiability Revocation must be externally verifiable. A relying party can determine whether a PoA has been revoked without trusting the issuer's assertion.

Invariant 4: Fail-Closed Under Uncertainty Authorization must fail closed when status cannot be determined. If revocation check is unavailable, the action should not proceed (or should enter degraded mode with restricted scope).

6.3.2 13.2 Enforcement Mechanisms

Authority Monotonicity is enforced during PoA issuance. The issuing software checks that delegated authority is a subset of the delegator's authority.

Root Traceability is verified during chain validation. The verifier walks the chain from agent to principal, confirming each link and eventually reaching a root.

Revocation Verifiability is provided by transparency logs. Relying parties query logs and verify proofs.

Fail-Closed Semantics is a policy enforced by relying parties. When verification is inconclusive, the transaction is rejected or restricted.

6.3.3 13.3 Degraded Mode

Degraded mode is the protocol's response to uncertainty. When: - Revocation logs are unavailable - Chain validation produces inconclusive results - Time bounds are ambiguous

The system may: - Reject the action entirely (most conservative) - Allow the action within tighter constraints - Allow the action with enhanced logging - Require additional human approval

The choice depends on the action's risk profile. Purchasing commodity supplies might proceed in degraded mode. Signing a major contract should not.

6.4 Chapter 14: Protocol Architecture Overview

6.4.1 14.1 The PoA Lifecycle

A PoA artifact goes through the following lifecycle:

1. **Issuance:** Principal creates and signs PoA for agent
2. **Distribution:** PoA is delivered to agent
3. **Presentation:** Agent presents PoA to relying party
4. **Verification:** Relying party validates PoA
5. **Action:** Relying party permits/denies the requested action
6. **Audit:** Transaction is logged with PoA evidence
7. **Revocation:** Principal revokes PoA if needed

6.4.2 14.2 Protocol Actors

Principal Legal entity with authority. Creates and signs PoA artifacts. May revoke authority.

Agent Autonomous system acting on principal's behalf. Presents PoA to relying parties.

Relying Party Entity that validates PoA and decides whether to proceed. Bears risk if validation is inadequate.

Transparency Log Public infrastructure recording revocation events. Provides proofs of revocation/non-revocation.

Audit Store System maintaining transaction records for regulatory and legal purposes.

6.4.3 14.3 Trust Relationships

graph TD

```

P[Principal] -->|issues| POA[PoA Artifact]
P -->|controls| A[Agent]
POA -->|presents to| RP[Relying Party]
A -->|uses| POA
RP -->|queries| TL[Transparency Log]
TL -->|provides proofs| RP
RP -->|logs to| AS[Audit Store]

```

Principal $\square$ **Agent**: Full trust (principal controls agent) **Agent** $\square$ **RP**: Verified trust (RP validates PoA) **RP** $\square$ **Log**: Limited trust (cryptographic proofs reduce trust requirements)
Principal $\square$ **Log**: Write access for revocation

6.4.4 14.4 Data Flows

Issuance Flow 1. Principal determines scope, constraints, validity 2. Principal creates PoA structure 3. Principal signs PoA 4. Principal registers (optional) with transparency log 5. Principal delivers PoA to agent

Verification Flow 1. Agent presents PoA to relying party 2. Relying party parses and validates structure 3. Relying party verifies signatures 4. Relying party validates delegation chain 5. Relying party checks revocation status 6. Relying party evaluates scope and constraints 7. Relying party makes authorization decision

Revocation Flow 1. Principal decides to revoke 2. Principal creates revocation entry 3. Principal signs revocation entry 4. Principal submits to transparency log 5. Log incorporates entry and provides proof 6. Relying parties discover revocation via log query

Chapter 7

Part IV: Proof of Authorization (AAP-002)

7.1 Chapter 15: The PoA Primitive

7.1.1 15.1 Definition

A Proof of Authorization (PoA) is a cryptographically signed artifact asserting that a legally identifiable principal has delegated a bounded set of authorities to a specific agent, under explicit constraints, within a defined jurisdiction and validity interval.

7.1.2 15.2 The PoA Tuple

Formally:

```
PoA := Sign_k(  
    Version,  
    PoAID,  
    Principal,  
    Agent,  
    DelegationChain,  
    Scope,  
    Constraints,  
    Jurisdiction,  
    Validity,  
    RevocationEndpoint,  
    Metadata
```

)

Version: Protocol version (currently “AAP-002-v1.0”)

PoAID: Unique identifier for this PoA

Principal: Entity profile reference for the delegating entity

Agent: Entity profile reference for the receiving agent

DelegationChain: Ordered list of delegation links from root to agent

Scope: Permitted actions and resources

Constraints: Limits on authority (spending caps, counterparty restrictions, etc.)

Jurisdiction: Legal framework under which PoA operates

Validity: Not-before and not-after timestamps

RevocationEndpoint: Where to check for revocation

Metadata: Extension point for additional claims

7.1.3 15.3 PoA Semantics

A valid PoA asserts:

“At the time of issuance, the Principal had authority A, and delegated a subset A’ to the Agent, effective from time T1 until time T2, within jurisdiction J, subject to constraints C. This delegation has not been revoked.”

Each element is verifiable: - Signatures prove the assertion was made - Chain validation proves the authority existed - Time bounds prove temporal validity - Revocation check proves non-revocation

7.1.4 15.4 PoA Is Not...

Not an Access Token PoA does not grant access to resources. It proves authority to perform actions.

Not a Contract PoA proves authorization, not agreement. The terms of any resulting contract are separate.

Not a Capability Token PoA includes legal semantics (identity, jurisdiction, liability) that capability tokens lack.

Not a Certificate PoA is not about identity binding. It assumes identity is established and addresses authority.

7.1.5 15.5 Properties of PoA

Self-Contained A PoA contains everything needed for verification (referencing external chain elements as needed).

Non-Interactive Verification Revocation checking may require network access, but PoA structure validation is purely cryptographic.

Attenuable A recipient of a PoA may issue a narrower PoA to another party.

Revocable A PoA may be revoked before expiration via transparency log.

Auditable PoA artifacts can be stored and verified indefinitely.

7.2 Chapter 16: Formal Specification

7.2.1 16.1 Data Structures

7.2.1.1 PoA Structure

```
struct PoA {
    version: String,           // "AAP-002-v1.0"
    poa_id: UUID,              // Unique identifier
    principal_ref: EntityRef,   // Reference to principal's profile
    agent_ref: EntityRef,       // Reference to agent's profile
    chain: DelegationChain,     // Authority chain
    scope: Scope,               // Permitted actions
    constraints: Constraints,    // Limits on authority
    jurisdiction: Jurisdiction, // Legal framework
    validity: ValidityPeriod,    // Temporal bounds
    revocation: RevocationInfo, // Where to check revocation
    metadata: Map<String, Any>, // Extensions
    signatures: Vec<Signature>, // Cryptographic signatures
}
```

7.2.1.2 Entity Reference

```
struct EntityRef {
    profile_id: UUID,           // Profile identifier
    profile_hash: Hash,         // Hash of profile content
    verification_uri: URI,      // Where to fetch profile
}
```

7.2.1.3 Delegation Chain

```
struct DelegationChain {  
    links: Vec<DelegationLink>,  
}
```

```
struct DelegationLink {  
    delegator_ref: EntityRef,  
    delegatee_ref: EntityRef,  
    authority_granted: Scope,  
    delegation_time: Timestamp,  
    evidence: Vec<Evidence>,  
    signature: Signature,  
}
```

7.2.1.4 Scope

```
struct Scope {  
    actions: Vec<Action>,  
    resources: Vec<Resource>,  
    exclusions: Vec<Exclusion>,  
}
```

```
struct Action {  
    namespace: String,    // e.g., "procurement"  
    operation: String,    // e.g., "purchase"  
    parameters: Map<String, Any>,  
}
```

7.2.1.5 Constraints

```
struct Constraints {  
    spending_limit: Option<SpendingLimit>,  
    counterparty_restrictions: Vec<CounterpartyRule>,  
    temporal_restrictions: Vec<TemporalRule>,  
    geographic_restrictions: Vec<GeographicRule>,  
    approval_requirements: Option<ApprovalRequirement>,  
    custom: Vec<CustomConstraint>,  
}
```

```
struct SpendingLimit {
```



```

    amount: Decimal,
    currency: CurrencyCode,
    period: Period,          // per_transaction, per_day, per_month
}

```

7.2.2 16.2 Signature Requirements

PoA requires: - EUF-CMA secure signature scheme - Signature covers canonical encoding of all fields - Multiple signatures for multi-party authorization

Recommended algorithms: - Ed25519 (default) - ECDSA over P-256 (interoperability) - Dilithium (post-quantum roadmap)

7.2.3 16.3 Encoding

PoA uses canonical encoding: - All fields sorted alphabetically - No whitespace in serialization - UTF-8 for strings - IEEE 754 for decimals - RFC 3339 for timestamps

Encoding errors invalidate the PoA.

7.2.4 16.4 Verification Algorithm

```

function verify_poa(poa: PoA) -> VerificationResult:
    // Step 1: Structural validation
    if not validate_structure(poa):
        return Invalid("Malformed structure")

    // Step 2: Signature verification
    for sig in poa.signatures:
        if not verify_signature(sig, poa):
            return Invalid("Signature verification failed")

    // Step 3: Chain validation
    chain_result = validate_delegation_chain(poa.chain)
    if chain_result.is_error():
        return Invalid(chain_result.error)

    // Step 4: Temporal validation
    now = current_time()
    if now < poa.validity.not_before:
        return Invalid("PoA not yet valid")
    if now > poa.validity.not_after:

```

```

    return Invalid("PoA expired")

// Step 5: Revocation check
revocation_status = check_revocation(poa.poa_id, poa.revocation)
if revocation_status == Revoked:
    return Invalid("PoA revoked")
if revocation_status == Unknown:
    return DegradedMode("Revocation status unknown")

// Step 6: Constraint evaluation
// (Performed at action time, not verification time)

return Valid(poa)

```

7.3 Chapter 17: Delegation Chains

7.3.1 17.1 The Chain Structure

A delegation chain consists of ordered links from a root authority to the acting agent:

Root(A) → Delegate(A→B) → Delegate(B→C) → ... → Agent

Each link contains: - Who delegated - Who received - What was delegated - When - Evidence of authority

7.3.2 17.2 Chain Validation Rules

Rule 1: Signature Validity Each link must be signed by the delegator's key.

Rule 2: Authority Subset Each delegatee receives a subset of delegator's authority. Authority may not expand through delegation.

Rule 3: Temporal Consistency Each delegation must occur during the validity period of the delegator's authority.

Rule 4: No Cycles The same entity may not appear twice in a chain.

Rule 5: Root Competence The root must be a legally competent authority (e.g., a corporation's board, a government agency with statutory power).

7.3.3 17.3 Chain Length Limits

To prevent abuse, implementations should enforce maximum chain lengths. Recommended limits:

- **Low-risk actions:** Up to 5 links
- **Medium-risk actions:** Up to 3 links
- **High-risk actions:** Up to 2 links

Longer chains introduce latency and increase attack surface.

7.3.4 17.4 Evidence Requirements

Each delegation link should include evidence of the delegator's authority: - Reference to parent delegation - Corporate authorization (board resolution, officer certificate) - Statutory grant (for government entities) - Contractual right (for contracted authority)

7.3.5 17.5 Chain Caching

Validating long chains is computationally expensive. Implementations may cache: - Previously validated chain prefixes - Entity profile resolutions - Revocation status (with short TTL)

Caching introduces staleness risk. Cache TTLs should be short for security-critical applications.

7.4 Chapter 18: Scope and Constraint Models

7.4.1 18.1 The Scope Model

Scope defines what an agent may do. It consists of:

Actions: Operations the agent may perform **Resources:** Objects the agent may act upon

Exclusions: Explicit denials

Scope is interpreted by the relying party. PoA does not impose a global action ontology.

7.4.2 18.2 Action Namespacing

Actions are namespaced to prevent collisions:

```
procurement:purchase
procurement:negotiate
finance:payment
```

hr:offer

Well-known namespaces may be standardized. Custom namespaces remain the domain of specific implementations.

7.4.3 18.3 Resource Patterns

Resources may be specified as: - **Literal**: Specific resource identifier - **Pattern**: Glob or regex matching multiple resources - **Type**: All resources of a type

Example:

```
resources: [
  "vendor:*",           // All vendors
  "category:office_supplies", // Specific category
  "urn:procurement:po:2025/*" // All 2025 POs
]
```

7.4.4 18.4 Constraints

Constraints are machine-enforceable limits on authority.

Spending Limits

```
spending_limit: {
  amount: 50000,
  currency: "EUR",
  period: "per_transaction"
}
```

Counterparty Restrictions

```
counterparty_restrictions: [
  { type: "blocklist", entities: ["sanctioned_entity_1", "..."] },
  { type: "allowlist", entities: ["approved_vendor_1", "..."] }
]
```

Temporal Restrictions

```
temporal_restrictions: [
  { type: "business_hours", timezone: "Europe/Berlin",
    hours: "08:00-18:00", days: ["Mon", "Tue", "Wed", "Thu", "Fri"] }
]
```

Geographic Restrictions

```
geographic_restrictions: [
```

```

    { type: "allowed_countries", countries: ["DE", "FR", "NL"] }
]

```

Approval Requirements

```

approval_requirements: {
  threshold: 2,
  approvers: ["approver_key_1", "approver_key_2", "approver_key_3"]
}

```

7.4.5 18.5 Constraint Enforcement

Constraints are enforced at action time:

1. Relying party receives PoA and requested action
2. Relying party evaluates each constraint against the action
3. If any constraint is violated, action is denied
4. If all constraints are satisfied, action may proceed

Constraint evaluation is deterministic. The same inputs always produce the same result.

7.5 Chapter 19: Revocation Semantics

7.5.1 19.1 Why Revocation Matters

Authority must be revocable. Circumstances change: - An employee is terminated - An agent is compromised - A business relationship ends - An error is discovered

Irrevocable authority is a liability. PoA treats revocation as a first-class concern.

7.5.2 19.2 Revocation Entry Structure

```

struct RevocationEntry {
  poa_id: UUID,           // PoA being revoked
  revocation_time: Timestamp, // When revocation became effective
  reason: RevocationReason, // Why revoked
  issuer_ref: EntityRef,    // Who revoked (must have authority)
  signature: Signature,     // Cryptographic proof
}

enum RevocationReason {
  KeyCompromise,

```

```

    AgentTerminated,
    AuthorityExceeded,
    BusinessChange,
    Error,
    Other(String),
}

```

7.5.3 19.3 Revocation Authority

Revocation may be performed by: - The original principal - Any entity in the delegation chain with equal or higher authority - Designated revocation agents

Revocation authority is itself subject to verification.

7.5.4 19.4 Revocation Transparency

Revocations are published to append-only logs. A valid revocation check requires:

1. Query the revocation endpoint specified in PoA
2. Receive either:
 - A non-revocation proof (Merkle inclusion proof showing no revocation entry)
 - A revocation proof (Merkle inclusion proof of revocation entry)
 - An error (log unreachable)
3. Validate the proof cryptographically

7.5.5 19.5 Revocation vs. Expiration

Expiration is natural termination at scheduled time **Revocation** is premature termination by authority action

Both result in invalid PoA. The distinction matters for: - Audit records - Liability analysis - Key management

7.5.6 19.6 Revocation Latency

Revocation is not instant. Between revocation and relying party discovery: - Revocation is signed and submitted - Log includes revocation - Log gossips update - Relying party queries log - Relying party acts on revocation

For high-assurance scenarios, minimize latency through: - Frequent log polling - Push notifications - Short PoA validity (requiring frequent renewal)

7.6 Chapter 20: Implementation Patterns

7.6.1 20.1 PoA Issuance Patterns

Direct Issuance Principal directly signs PoA for agent. - Simple - Requires principal involvement for each PoA - Suitable for small scale

Delegated Issuance Principal authorizes an intermediary to issue PoAs. - Scales better - Adds complexity - Longer delegation chains

Automated Issuance System generates PoAs within pre-authorized bounds. - Highest scale - Requires careful policy definition - Risk of over-authorization

7.6.2 20.2 Verification Patterns

Online Verification Relying party checks revocation in real-time. - Most current information - Adds latency - Requires network access

Cached Verification Relying party uses cached revocation data. - Lower latency - Risk of stale data - Suitable for lower-risk actions

Offline Verification Relying party verifies only cryptographic elements. - Lowest latency - No revocation check - Only for very low-risk or degraded mode

7.6.3 20.3 Integration Patterns

Gateway/Proxy A gateway verifies PoA before forwarding requests. - Centralizes verification logic - Single point of failure - Easy integration with existing systems

Library/SDK Each service embeds verification logic. - Distributed verification - More resilient - Requires consistent library deployment

Sidecar Verification runs as a separate process alongside each service. - Middle ground - Language-agnostic - Container-friendly

7.6.4 20.4 Reference Implementation

The AgentAuth project provides a reference implementation in Go:

```
// Create a PoA
poa := &agentauth.PoA{
    Version:      "AAP-002-v1.0",
    PoAID:        uuid.New(),
    Principal:    principalRef,
    Agent:        agentRef,
    Chain:        delegationChain,
```

```
    Scope:      scope,
    Constraints: constraints,
    Jurisdiction: jurisdiction,
    Validity:    validity,
    Revocation:  revocationInfo,
}

// Sign the PoA
signedPoa, err := poa.Sign(principalKey)
if err != nil {
    return err
}

// Verify a PoA
result, err := agentauth.Verify(signedPoa, verifyOptions)
if err != nil {
    return err
}

if !result.Valid {
    return fmt.Errorf("PoA invalid: %s", result.Reason)
}

// Check constraints against action
if !result.Constraints.Allow(action) {
    return fmt.Errorf("Action violates constraints")
}

// Proceed with action
```

[Document continues with Parts V-IX and Appendices...]

Chapter 8

Part V: Cryptographic Architecture

8.1 Chapter 21: Design Principles

The cryptographic architecture of PoA follows four foundational principles:

8.1.1 21.1 Explicit Trust Boundaries

PoA makes no implicit assumptions about trust. Every trust relationship is explicit: - Entity profiles explicitly state who issued them - Delegation chains explicitly trace authority - Revocation proofs explicitly demonstrate log inclusion

8.1.2 21.2 Minimal Cryptographic Assumptions

PoA uses standard, well-analyzed cryptographic primitives: - Standard signature schemes (Ed25519, ECDSA) - Standard hash functions (SHA-256) - Standard encoding formats (CBOR, JSON)

Novel cryptography is avoided. The risk of undiscovered weaknesses outweighs marginal performance gains.

8.1.3 21.3 Offline Verifiability

PoA verification should not require real-time network access for core operations. Signature verification, structure validation, and chain verification are purely cryptographic.

Revocation checking requires network access but can be cached or deferred for low-risk scenarios.

8.1.4 21.4 Evidence Preservation

PoA artifacts must remain verifiable for years or decades. This implies: - Algorithm agility (migration paths) - Long-term key management - Timestamp anchoring - Audit record durability

8.2 Chapter 22: Signature Schemes and Key Management

8.2.1 22.1 Supported Algorithms

Ed25519 (default) - 128-bit security level - Fast signing and verification - Compact signatures (64 bytes) - Deterministic signing

ECDSA over P-256 (interoperability) - NIST approved - Widely supported - Required when Ed25519 unavailable

Dilithium (post-quantum roadmap) - NIST PQC selection - Larger signatures (~2.5KB) - Not yet recommended for production

8.2.2 22.2 Key Derivation

For hierarchical key structures: - Use HKDF with SHA-256 - Derive agent keys from controller keys - Include purpose in derivation context - Document derivation paths for auditing

8.2.3 22.3 Key Rotation

Keys should be rotated regularly: - Agent keys: Monthly recommended - Principal keys: Annually or upon personnel change - Root keys: Exercise extreme caution

Rotation invalidates existing PoAs signed by the old key. Plan transitions carefully.

8.2.4 22.4 Key Compromise Response

Upon key compromise: 1. Revoke all PoAs signed by compromised key 2. Issue new key and update profiles 3. Re-issue necessary PoAs with new key 4. Audit actions taken with compromised key

Speed matters. Compromise response should be exercised before it's needed.

8.3 Chapter 23: Canonicalization and Hash Functions

8.3.1 23.1 The Canonicalization Problem

Digital signatures sign bytes, not semantics. Two JSON documents with identical meaning but different formatting:

```
{"name": "Alice", "age": 30}

{
  "age": 30,
  "name": "Alice"
}
```

These are semantically identical but produce different hashes. If a PoA is signed using one format and verified using another, signature verification fails.

Canonicalization ensures that semantically equivalent documents produce identical byte representations.

8.3.2 23.2 Canonicalization Requirements

PoA canonicalization must satisfy:

Determinism The same logical document always produces the same byte sequence.

Semantic Preservation Canonicalization does not change the document's meaning.

Round-Trip Stability Parsing a canonical document and re-canonicalizing produces the same bytes.

Implementation Independence Different implementations produce identical output.

8.3.3 23.3 Recommended Encoding: CBOR

PoA recommends CBOR (Concise Binary Object Representation, RFC 8949) with deterministic encoding:

- Map keys sorted lexicographically by encoded form
- Preferred serialization for integers
- No indefinite-length items
- No duplicate map keys

CBOR advantages: - Binary (compact, unambiguous) - Well-specified deterministic variant
- Widely implemented - Extensible

8.3.4 23.4 JSON Canonicalization (Alternative)

For systems requiring human-readable formats, JSON Canonicalization Scheme (RFC 8785) is acceptable:

- Object keys sorted by Unicode code point
- No whitespace
- Specific number formatting

JSON disadvantages: - Larger than binary - Some edge cases in number handling - Requires careful implementation

8.3.5 23.5 Hash Function Selection

PoA requires collision-resistant hash functions:

SHA-256 (default) - 256-bit output - Widely implemented - NIST approved - Sufficient security margin

SHA-384/SHA-512 (extended security) - Longer output - Higher security margin - Recommended for long-term archives

BLAKE3 (performance) - Very fast - Modern design - Not yet NIST approved

8.3.6 23.6 Hash Agility

PoA artifacts include a hash algorithm identifier:

```
hash_algorithm: "sha256"
hash_value: "a3b5c7..."
```

This enables algorithm transition without breaking compatibility.

8.4 Chapter 24: Revocation Transparency Logs

8.4.1 24.1 Log Architecture

A PoA revocation transparency log is a cryptographic data structure providing:

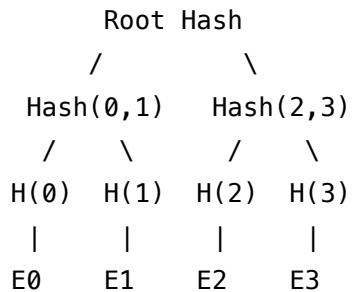
Append-Only Semantics New entries can be added. Existing entries cannot be modified or deleted.

Efficient Verification Clients can verify inclusion and consistency without downloading the entire log.

Public Auditability Anyone can monitor the log for unexpected entries.

8.4.2 24.2 Merkle Tree Structure

Logs use Merkle trees (RFC 6962 variant):



Each entry is hashed. Hashes are combined pairwise up to the root. The root hash summarizes all entries.

8.4.3 24.3 Inclusion Proofs

To prove entry E2 is in the log: 1. Provide H(2) 2. Provide H(3) 3. Provide Hash(0,1) 4. Client computes Hash(2,3) = H(H(2) || H(3)) 5. Client computes Root = H(Hash(0,1) || Hash(2,3)) 6. If computed root matches published root, E2 is included

Proof size: $O(\log n)$ for n entries.

8.4.4 24.4 Consistency Proofs

To prove log at time T1 is a prefix of log at time T2: - Provide Merkle proofs for shared subtrees - Client verifies both roots are consistent

This prevents log operators from rewriting history.

8.4.5 24.5 Signed Tree Heads

Log operators periodically sign the root hash:

```

struct SignedTreeHead {
    timestamp: Timestamp,
    tree_size: u64,
    root_hash: Hash,
    signature: Signature,
}

```

Clients cache STHs to detect misbehavior. An STH that doesn't follow consistently from a previous STH indicates log tampering.

8.4.6 24.6 Log Operation

Submission 1. Principal submits revocation entry 2. Log validates entry (signature, authority to revoke) 3. Log assigns sequence number 4. Log incorporates entry into Merkle tree 5. Log returns inclusion proof

Query 1. Client queries log for PoA ID 2. Log returns: - Revocation entry + inclusion proof (if revoked) - Non-membership proof (if not revoked) - Error (if unavailable)

Monitoring 1. Monitors fetch full logs periodically 2. Monitors verify consistency between STHs 3. Monitors alert on anomalies

8.4.7 24.7 Log Operator Requirements

Log operators must: - Maintain high availability (99.9%+ recommended) - Sign STHs at regular intervals (hourly minimum) - Respond to inclusion queries within SLA - Preserve data indefinitely - Operate transparently (public policy, audit reports)

8.4.8 24.8 Multiple Logs

High-assurance deployments may use multiple logs: - Principals submit to multiple logs - Relying parties check multiple logs - Revocation is only certain when confirmed by k-of-n logs

This mitigates single-log compromise.

8.5 Chapter 25: Degraded Mode Operations

8.5.1 25.1 When Degraded Mode Applies

Degraded mode activates when:

Revocation Check Failure - Log is unreachable - Log returns error - Proof verification fails - Response timeout

Chain Validation Uncertainty - Intermediate entity profile unavailable - Evidence document unavailable - Clock skew exceeds tolerance

Cryptographic Ambiguity - Algorithm not recognized (version mismatch) - Key format unrecognized

8.5.2 25.2 Degraded Mode Policies

Organizations define degraded mode policies specifying responses to uncertainty:

Policy: Deny Reject all actions when degraded. - Most conservative - May cause operational disruption - Appropriate for high-risk actions

Policy: Restrict Allow only low-risk actions when degraded. - Spending limits reduced 10x - No new counterparties - Enhanced logging

Policy: Approve with Audit Allow actions but flag for review. - Least disruptive - Higher risk exposure - Requires robust audit infrastructure

Policy: Human Approval Require human confirmation. - Balanced approach - May introduce latency - Appropriate for medium-risk actions

8.5.3 25.3 Degraded Mode Indicators

When operating in degraded mode, systems must:

Log the Condition Record why degraded mode was entered, with evidence.

Notify Stakeholders Alert operations teams to degraded status.

Limit Duration Establish maximum time in degraded mode before escalation.

Preserve Evidence Store all artifacts for later verification when connectivity restored.

8.5.4 25.4 Recovery from Degraded Mode

When normal operations resume: 1. Re-validate all actions taken in degraded mode 2. Flag any actions that would have been denied 3. Initiate remediation for violations 4. Update audit records with verified status

8.5.5 25.5 Attack Considerations

Adversaries may attempt to force degraded mode: - DoS attacks on revocation logs - Network partitioning - DNS manipulation

Defenses: - Multiple independent logs - Cached revocation data - Out-of-band revocation channels - Automatic restriction escalation

8.6 Chapter 26: Post-Quantum Considerations

8.6.1 26.1 The Quantum Threat

Sufficiently powerful quantum computers could break: - RSA (Shor's algorithm) - ECDSA (Shor's algorithm) - Ed25519 (Shor's algorithm) - Diffie-Hellman key exchange

Hash functions remain secure (Grover provides only quadratic speedup, mitigated by larger hashes).

8.6.2 26.2 Timeline Uncertainty

Expert estimates for cryptographically relevant quantum computers: - Optimistic: 2030 - Moderate: 2035-2040 - Conservative: 2045+

The uncertainty argues for preparation now.

8.6.3 26.3 “Harvest Now, Decrypt Later”

Adversaries may: 1. Collect encrypted data and signatures today 2. Store until quantum computers exist 3. Break encryption/forgo signatures retroactively

For PoA, this means: - Signatures made today could be forged in the future - Audit records could be manipulated retroactively - Historical authorization could be falsified

8.6.4 26.4 Post-Quantum Signature Schemes

NIST has selected post-quantum signature algorithms:

CRYSTALS-Dilithium - Based on lattice problems - Moderate signature size (~2.5KB) - Fast signing and verification - Primary recommendation

FALCON - Based on NTRU lattices - Smaller signatures (~900 bytes) - More complex implementation - Suitable when size matters

SPHINCS+ - Hash-based (conservative assumptions) - Large signatures (~8-50KB) - Slowest - Fallback option

8.6.5 26.5 Hybrid Signatures

PoA supports hybrid signatures during transition:

```
signature: {
  classic: Ed25519_signature,
  post_quantum: Dilithium_signature
}
```

Verification requires both signatures to be valid.

Benefits: - Remains secure if PQ algorithms have undiscovered weaknesses - Provides quantum resistance when PQ algorithms are sound - Smooth migration path

8.6.6 26.6 Migration Strategy

Phase 1: Preparation (Now - 2027) - Implement hybrid signature support - Begin testing with PQ algorithms - Plan key management changes

Phase 2: Transition (2027 - 2030) - Issue new Entity Profiles with PQ keys - Re-sign long-lived PoAs with hybrid signatures - Deprecate classic-only signatures

Phase 3: Post-Quantum (2030+) - PQ signatures required - Classic signatures deprecated - Legacy artifacts migrated or expired

8.6.7 26.7 Hash Function Considerations

While hash functions are more quantum-resistant: - SHA-256 security reduces from 128-bit to ~85-bit against Grover - Not considered broken but margin reduced - SHA-384/SHA-512 provide larger margins - BLAKE3 and SHA-3 are alternatives

PoA recommends SHA-384 for new deployments seeking long-term security.

Chapter 9

Part VI: Legal Mapping Across Jurisdictions

9.1 Chapter 27: Authority as a Legal Concept

9.1.1 27.1 Authority vs. Permission

Every legal system distinguishes between:

Permission: What you may do for your own benefit **Authority:** What you may do that binds another

A customer may withdraw money from their account (permission). A power of attorney allows someone else to withdraw on the customer's behalf (authority).

Technical systems often confuse these concepts. An API key grants *permission* to call an endpoint. It does not inherently grant *authority* to bind the key's owner.

9.1.2 27.2 The Legal Framework of Agency

Agency law provides the vocabulary for authority:

Principal: The party whose authority is exercised **Agent:** The party exercising authority

Third Party: The party affected by the agent's actions **Authority:** The power granted to the agent **Scope:** The boundaries of authority **Fiduciary Duty:** The agent's obligations to the principal

9.1.3 27.3 How Authority Is Created

Authority may be created through:

Express Grant The principal explicitly states what the agent may do.

Implied Authority Authority necessary to accomplish an express grant.

Apparent Authority Authority that third parties reasonably believe exists based on the principal's conduct.

Ratification The principal adopts an unauthorized act after the fact.

Statute Law directly grants authority (e.g., corporate officers, government officials).

9.1.4 27.4 How Authority Is Limited

Authority is bounded by:

Express Limitations The principal's grant specifies what the agent may not do.

Inherent Limitations Certain acts require more authority than others (e.g., selling assets vs. routine purchases).

Temporal Limitations Authority expires at a specified time or upon a condition.

Jurisdictional Limitations Authority valid in one jurisdiction may not be recognized in another.

9.1.5 27.5 How Authority Is Revoked

Revocation terminates authority. It may occur through:

Express Revocation The principal directly revokes the grant.

Operation of Law Events such as death, incapacity, or dissolution terminate authority.

Expiration The granted term ends.

Accomplishment The purpose of the grant is completed.

9.1.6 27.6 The Third-Party Problem

When an agent acts, third parties must decide: - Does the agent have authority? - What is the scope of authority? - Has authority been revoked?

If the agent lacks authority but the third party reasonably believed otherwise, the principal may still be bound (apparent authority).

This creates the core problem PoA addresses: making authority observable and verifiable.

9.2 Chapter 28: German Law: Statutory Representation

9.2.1 28.1 Overview

German law provides highly structured models for authority, rooted in the Civil Code (Bürgerliches Gesetzbuch, BGB) and the Commercial Code (Handelsgesetzbuch, HGB).

Key features: - Clear distinction between types of authority - Register-based verification - Strong third-party protection

9.2.2 28.2 Authority Types

Vollmacht (General Power of Attorney) A grant of authority by private act. The principal (Vollmachtgeber) authorizes an agent (Bevollmächtigter) to act on their behalf.

Handlungsvollmacht (Commercial Authority) Authority to perform typical commercial acts for a business. Statutorily defined scope.

Prokura (Commercial Procuration) Extensive authority to represent a business in all judicial and extrajudicial matters. Must be registered in the commercial register.

Gesetzliche Vertretung (Statutory Representation) Authority granted by law, such as parents for children or managing directors for corporations.

9.2.3 28.3 The Commercial Register (Handelsregister)

The Handelsregister is a public register maintained by local courts (Amtsgericht). It records:

- Company name and legal form
- Managing directors and their authority
- Prokura grants
- Limitations on representation rights

Third parties are entitled to rely on register entries. Internal limitations not reflected in the register are generally not opposable to third parties acting in good faith.

9.2.4 28.4 Third-Party Protection (Verkehrsschutz)

German law strongly protects third parties who rely on:

Register Content (positive Publizität) If the register says X has authority, third parties may rely on that even if the register is incorrect.

Register Silence (negative Publizität) If a limitation is not in the register, third parties may assume it doesn't exist.

9.2.5 28.5 Implications for PoA

Delegation Chains Must Root in Register For German corporate principals, PoA chains should connect to verifiable register entries.

Revocation Must Be Public German law expects third parties to check the register. PoA's revocation transparency mirrors this expectation.

Authority Scope Should Be Clear Vague or overly broad authority increases principal liability.

9.2.6 28.6 German Law and AI Agents

German courts have not yet addressed AI agent authority directly. However:

- An AI agent operating with Prokura would bind the company extensively
- Register entries cannot currently describe AI agents specifically
- The gap between register-based authority and AI deployment creates legal uncertainty

PoA provides a technical mechanism to bridge this gap while maintaining the German principle of public verifiability.

9.3 Chapter 29: United States: Actual vs. Apparent Authority

9.3.1 29.1 Overview

U.S. agency law is largely judge-made, synthesized in the Restatement (Third) of Agency. Key features:

- Flexible standards
- Strong apparent authority doctrine
- State-by-state variation

9.3.2 29.2 Actual Authority

Actual authority is authority the principal intentionally grants:

Express Actual Authority The principal explicitly states what the agent may do.

Implied Actual Authority Authority reasonably necessary to accomplish the express grant. Includes customary authority for the role.

9.3.3 29.3 Apparent Authority

Apparent authority binds principals based on third-party perception:

“Apparent authority is the power held by an agent or other actor to affect a principal’s legal relations with third parties when a third party reasonably believes the actor has authority to act on behalf of the principal and that belief is traceable to the principal’s manifestations.” (Restatement §2.03)

Key elements: 1. Third party’s reasonable belief 2. Belief traceable to principal’s conduct 3. Third party relies on the appearance

9.3.4 29.4 The Problem for AI Agents

Consider: 1. A company deploys an AI agent with API access 2. The agent makes purchases using company credentials 3. A supplier sees the agent using company systems 4. The supplier reasonably believes the agent is authorized

If the agent exceeds internal limits, apparent authority may bind the company. The supplier’s reliance was reasonable based on the company’s manifestation (granting the API key).

9.3.5 29.5 Limiting Apparent Authority

To limit apparent authority, principals must:

Make Limitations Visible Internal policies don’t help. Limitations must be communicated to third parties.

Control Manifestations Carefully manage what appearances the principal creates.

Monitor Agent Actions Active oversight reduces the appearance of unlimited authority.

9.3.6 29.6 PoA’s Solution

PoA embeds limitations in the artifact itself:

```
constraints: {
  spending_limit: { amount: 10000, currency: "USD", period: "per_transaction" }
}
```

A supplier verifying the PoA sees the limit. If an order exceeds it, no reasonable belief in authority can form. The PoA cuts off apparent authority at its root.

9.3.7 29.7 Case Law Analogies

Courts have held:

Airline Agents: Limited authority to override fares despite access to booking systems.

Real Estate Agents: Authority to negotiate does not include authority to accept below listing price.

Corporate Officers: Title alone does not guarantee specific authority; scope depends on corporate governance.

PoA extends these principles to autonomous systems.

9.4 Chapter 30: European Union: eIDAS and Trust Services

9.4.1 30.1 Overview

The eIDAS Regulation (EU 910/2014) establishes a common framework for:

- Electronic identification (eID)
- Trust services (signatures, seals, timestamps)
- Cross-border recognition

9.4.2 30.2 Electronic Signatures

eIDAS defines three levels:

Electronic Signature Data attached to or associated with other data for signing. Minimal requirements.

Advanced Electronic Signature (AdES) Uniquely linked to signatory, capable of identifying signatory, created with data under signatory's control, detects subsequent changes.

Qualified Electronic Signature (QES) An AdES created by a qualified signature creation device, based on a qualified certificate. Has the legal effect of a handwritten signature.

9.4.3 30.3 Electronic Seals

For legal persons: - Similar levels as signatures - Link to legal person rather than natural person - Used for organizational attestation

9.4.4 30.4 Timestamps

Qualified timestamps prove data existed at a given time. Important for PoA validity periods.

9.4.5 30.5 Trust Service Providers

Qualified Trust Service Providers (QTSPs) are: - Licensed by national supervisory bodies - Subject to regular audits - Listed on Trusted Lists - Liable for failures

9.4.6 30.6 eIDAS and Authority

eIDAS addresses *identity* and *signatures*, not *authority*:

- A QES proves who signed
- It does not prove authority to sign on behalf of another

9.4.7 30.7 PoA's Relationship to eIDAS

PoA complements eIDAS:

Identity Foundation eIDAS-verified identities can serve as Entity Profile roots.

Signature Mechanism PoA signatures may use eIDAS-qualified mechanisms for legal weight.

Authority Layer PoA adds authority semantics that eIDAS lacks.

Together: - eIDAS proves “Alice signed this document” - PoA proves “Alice was authorized to sign for Acme Corp”

9.4.8 30.8 The eIDAS 2.0 Evolution

The proposed eIDAS 2.0 regulation introduces:

- European Digital Identity Wallets
- Enhanced attribute attestation
- Cross-sector interoperability

These developments may provide infrastructure for PoA Entity Profiles and delegation verification.

9.5 Chapter 31: Cross-Border and Conflict-of-Law

9.5.1 31.1 The Problem

Autonomous agents operate globally. An agent in Germany may transact with a supplier in Japan, on behalf of a U.S. corporation. Which law applies?

Without explicit specification: - Multiple laws may claim applicability - Parties may disagree on governing law - Courts face complex choice-of-law analysis

9.5.2 31.2 Choice of Law

Commerce generally allows parties to choose governing law. PoA makes this explicit:

```
jurisdiction: {
  governing_law: "DE",
  venue: "Munich",
  conflict_rules: "EU Rome I"
}
```

9.5.3 31.3 Recognition of Foreign Authority

Even with choice of law, questions arise:

- Does the forum recognize the chosen law?
- Are there mandatory local requirements?
- Does public policy override the choice?

9.5.4 31.4 Examples

Example 1: German Agent, U.S. Transaction A German company's AI agent purchases goods from a U.S. supplier. The PoA specifies German law. The U.S. supplier may: - Accept if comfortable with German law - Reject if requiring U.S. law - Negotiate alternative terms

Example 2: Multi-Jurisdictional Chain A delegation chain spans: - U.S. corporation (root) - German subsidiary (intermediate) - Japanese vendor relationship (agent)

Each link may involve different applicable law. PoA makes this traceable.

9.5.5 31.5 Mandatory Rules

Some rules cannot be derogated by contract: - Consumer protection (when dealing with consumers) - Anti-money laundering - Sanctions compliance - Data protection

PoA constraints can encode compliance requirements, but legal advice is required for specific deployments.

9.5.6 31.6 PoA's Approach

PoA requires explicit jurisdiction specification:

1. Every PoA specifies governing law
2. Relying parties can reject mismatched jurisdictions
3. Audit records show which law applied

This does not solve all conflict-of-law problems. It ensures the question is addressed rather than ignored.

9.6 Chapter 32: Regulatory Compliance Implications

9.6.1 32.1 Financial Services

Financial regulators require: - Clear authorization records - Audit trails for transactions - Segregation of duties - Know-Your-Customer (KYC) for counterparties

PoA supports: - Cryptographic authorization records - Immutable audit evidence - Constraint-based controls - Entity Profiles with verified identities

9.6.2 32.2 Export Controls

Export control regimes (U.S. EAR, EU Dual-Use) require: - Screening counterparties against restricted lists - End-use verification - License compliance

PoA can encode: - Counterparty restriction constraints (blocklists) - Export classification requirements - Geographic restrictions

9.6.3 32.3 Data Protection

GDPR and similar regimes require: - Lawful basis for processing - Data subject rights - Processor obligations - Cross-border transfer controls

PoA can encode: - Processing scope limitations - Data residency constraints - Controller-processor relationships

9.6.4 32.4 Anti-Money Laundering

AML regimes require: - Customer identification - Transaction monitoring - Suspicious activity reporting - Record-keeping

PoA supports: - Verified Entity Profiles - Constraint-based monitoring integration - Transparent authority chains - Durable audit records

9.6.5 32.5 Regulatory Inquiry

When regulators investigate:

Traditional Systems - Access control logs show what happened - Authorization policies show what should have happened - Reconciling the two is manual work

PoA Systems - PoA artifacts show what was authorized - Audit records show what happened - Verification is cryptographic and automatable

9.6.6 32.6 Compliance Architecture

For regulated industries:

Regulatory Framework
PoA Constraints encode requirements
Entity Profiles verified against regulatory registries
Revocation transparency provides audit evidence
Delegation chains trace responsibility

Chapter 10

Part VII: Operational Considerations

10.1 Chapter 33: Deployment Architectures

10.1.1 33.1 Centralized vs. Distributed

Centralized - Single verification service - Easier to manage - Single point of failure - Suitable for single-organization deployments

Distributed - Multiple verification points - More resilient - More complex to coordinate - Suitable for cross-organization scenarios

10.1.2 33.2 Cloud Deployment

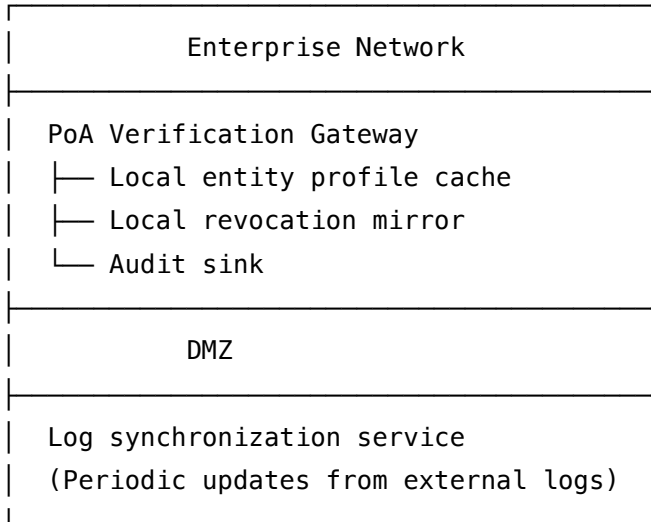
For cloud-native environments:

Load Balancer		
Verify-1	Verify-2	Verify-3
Shared State (Redis/DDB)		
Log Query Service		

Key considerations: - Stateless verification services - Shared cache for performance - Log queries against external infrastructure

10.1.3 33.3 On-Premises Deployment

For regulated or air-gapped environments:



10.1.4 33.4 Hybrid Deployment

Many organizations combine approaches: - Internal transactions verified locally - External transactions verified against public logs - Fallback to degraded mode when external unavailable

10.2 Chapter 34: Audit Trail Design

10.2.1 34.1 What to Log

Every PoA verification should log:

Verification Request - Timestamp - Requesting agent identity - Requested action - Presented PoA ID

Verification Result - Outcome (valid/invalid/degraded) - Reason codes - Constraint evaluation results - Chain validation details

Evidence - PoA artifact (or hash reference) - Entity profiles consulted - Revocation proofs obtained - Timestamps from trusted sources

10.2.2 34.2 Immutability

Audit logs must be tamper-evident: - Append-only storage - Cryptographic chaining - External anchoring (blockchain, transparency log) - Multi-party witnesses

10.2.3 34.3 Retention

Retention periods depend on: - Regulatory requirements (often 5-10 years) - Statute of limitations (may extend longer) - Business needs

10.2.4 34.4 Query Capabilities

Audit systems should support: - Query by PoA ID - Query by principal/agent - Query by time range - Query by outcome - Full-text search on constraints

10.2.5 34.5 Incident Response

When incidents occur: 1. Identify affected PoAs 2. Query all verifications for those PoAs 3. Determine scope of unauthorized actions 4. Correlate with other system logs 5. Produce compliance report

10.3 Chapter 35: Integration Patterns**10.3.1 35.1 API Gateway Integration**

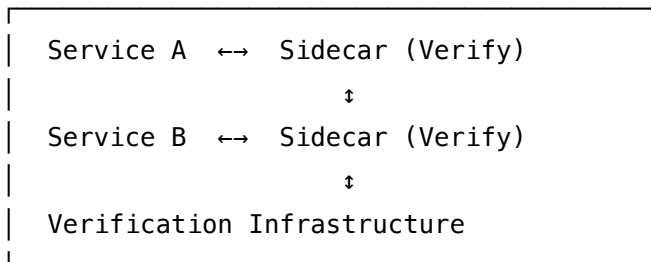
Client → API Gateway → [PoA Verification] → Backend

↓

If invalid: 403 Forbidden

If valid: Forward with verified context

The gateway handles verification, allowing backends to trust verified context.

10.3.2 35.2 Service Mesh Integration

Each service has a sidecar proxy that handles PoA verification.

10.3.3 35.3 Workflow Engine Integration

For complex processes:

Workflow Engine

- |— Step 1: Receive request
- |— Step 2: Verify PoA (blocking)
- |— Step 3: If valid, proceed
- |— Step 4: Execute action
- |— Step 5: Log outcome

The workflow engine treats verification as a first-class step.

10.3.4 35.4 Legacy System Integration

For systems that cannot be modified:

Legacy System ← Adapter ← PoA Gateway ← Agent

Adapter translates between:

- Legacy auth (e.g., username/password)
- PoA verification

The adapter maintains a mapping between legacy credentials and PoAs.

Chapter 11

Part VIII: Non-Goals and Limitations

11.1 Chapter 36: What This System Does Not Do

11.1.1 36.1 Prove Intent

PoA proves authorization. It does not prove: - The agent made a good decision - The principal intended this specific outcome - The action was wise or beneficial

Authorization is necessary but not sufficient for appropriate action.

11.1.2 36.2 Replace Contract Law

PoA is not a contract. It proves: - Agent had authority - Authority was within scope - Constraints were satisfied

The terms of any resulting agreement are governed by contract law, not PoA.

11.1.3 36.3 Automate Legal Judgment

PoA constraints are technical checks. They cannot replace: - Judicial interpretation - Regulatory discretion - Legal advice

11.1.4 36.4 Eliminate Human Oversight

PoA enables safer autonomous action. It does not make human oversight unnecessary: - Principals must define appropriate scopes - Operators must monitor for anomalies - Legal

counsel must advise on constraints - Regulators must audit compliance

11.1.5 36.5 Guarantee Correctness

A valid PoA does not guarantee: - The underlying action is correct - The business logic is sound - The transaction is economically rational

Verification proves authorization, not wisdom.

11.2 Chapter 37: The Boundaries of Machine Authority

11.2.1 37.1 The Fundamental Question

How much authority should autonomous systems have?

PoA provides mechanisms to bound authority. It does not answer whether those bounds are appropriate.

11.2.2 37.2 The Spectrum of Autonomy

Level 0: No Autonomy Agent proposes; human decides.

Level 1: Constrained Autonomy Agent decides within narrow bounds; human reviews exceptions.

Level 2: Supervised Autonomy Agent decides; human monitors.

Level 3: Independent Autonomy Agent decides; human audits after the fact.

11.2.3 37.3 Risk-Appropriate Authority

Authority should match risk:

Action	Risk Level	Recommended Authority
Query public data	Very low	Minimal verification
Routine purchases	Low	PoA with spending limits
Contract negotiation	Medium	PoA with scope limits + human approval
Major commitments	High	Multi-sig PoA + human approval

Action	Risk Level	Recommended Authority
Irreversible actions	Very high	Human only

11.2.4 37.4 The Accountability Gap

When things go wrong with autonomous systems: - The agent has no legal personality - The principal may claim the agent exceeded authority - The vendor of the agent may disclaim liability - Third parties may have no recourse

PoA addresses the accountability gap by creating verifiable evidence of authorization.

Chapter 12

Part IX: Security Considerations

12.1 Chapter 38: Threat Analysis

12.1.1 38.1 Asset Identification

What adversaries want: - Unauthorized transactions - Corporate intelligence - System disruption - Covering tracks - Liability shifting

12.1.2 38.2 Attack Surface

PoA systems expose: - Agent endpoints (presenting PoAs) - Verification services (processing PoAs) - Transparency logs (storing revocations) - Key material (signing PoAs) - Entity profiles (establishing identity)

12.1.3 38.3 Adversary Profiles

External Attacker - No legitimate access - Attempts compromise from outside - May target supply chain

Malicious Insider - Has some legitimate access - Attempts to exceed authority - May manipulate records

Compromised Agent - Legitimate agent under adversary control - Acts within apparent authority - Attempts maximum damage within scope

Nation-State - Sophisticated capabilities - Long-term persistent access - May target cryptographic infrastructure

12.2 Chapter 39: Attack Vectors and Mitigations

12.2.1 39.1 Key Compromise

Attack: Adversary obtains principal's signing key **Impact:** Can issue arbitrary PoAs **Mitigation:** - HSM storage for high-value keys - Multi-sig for critical authority - Short PoA validity - Rapid revocation capability

12.2.2 39.2 Replay Attacks

Attack: Adversary captures valid PoA and presents to different relying party **Impact:** Unauthorized action at new target **Mitigation:** - Scope constraints (specific resource binding) - Relying party verification - One-time-use tokens for single actions

12.2.3 39.3 Log Manipulation

Attack: Log operator modifies or hides entries **Impact:** Revocations not visible; false revocations inserted **Mitigation:** - Multiple independent logs - External monitoring - Consistency proofs - Cryptographic witnesses

12.2.4 39.4 Time Manipulation

Attack: Adversary manipulates clock to extend/backdate validity **Impact:** Expired PoA accepted; future PoA accepted early **Mitigation:** - NTP from multiple sources - Rough time bounds (allow for expected skew) - Relying party monotonic clocks - Timestamp attestation services

12.2.5 39.5 Social Engineering

Attack: Adversary convinces human to issue inappropriate PoA **Impact:** Legitimate but unwise authorization **Mitigation:** - Template enforcement - Approval workflows - Anomaly detection - Training

12.3 Chapter 40: Incident Response

12.3.1 40.1 Detection

Signs of PoA compromise: - Unexpected revocation entries - Verification failures spike - Actions outside normal patterns - External reports

12.3.2 40.2 Containment

Immediate actions: 1. Revoke affected PoAs 2. Disable affected agent identities 3. Block affected principal keys 4. Activate degraded mode 5. Alert stakeholders

12.3.3 40.3 Eradication

Root cause analysis: - How was authority obtained? - What authorized actions were taken?
- What unauthorized actions were attempted? - Where did controls fail?

12.3.4 40.4 Recovery

Restore operations: 1. Issue new keys 2. Issue new Entity Profiles 3. Issue new PoAs with corrected scope 4. Verify transparency logs are consistent 5. Resume normal operations

12.3.5 40.5 Lessons Learned

Post-incident: - Document timeline - Identify control gaps - Update policies - Train personnel - Test incident response

Chapter 13

Appendices

13.1 Appendix A: Original Contributions

This manuscript presents the following original contributions to the field of autonomous systems authorization:

1. **Proof of Authorization (PoA):** A protocol-level primitive that binds cryptographic authorization to legal authority, liability, and jurisdiction—distinct from access tokens, capability tokens, and identity certificates.
2. **Delegation Chains with Legal Semantics:** Explicit representation of authority flow including statutory competence, fiduciary scope, and third-party reliance considerations.
3. **Machine-Verifiable Liability Constraints:** Constraints embedded directly in authorization artifacts and enforced at verification time, including spending limits, counterparty restrictions, and geographic bounds.
4. **Authority Revocation Transparency:** Application of append-only public logs to legal authority revocation rather than identity certificate issuance, enabling external auditability.
5. **Jurisdiction-Bound Authorization:** Explicit conflict-of-law inputs required in authorization artifacts, preventing implicit jurisdictional ambiguity.

While individual components draw from existing systems (SPKI for delegation, Certificate Transparency for logs, capability tokens for attenuation), their composition under legal constraints is novel. The contribution lies not in new cryptography but in new application of established cryptography to an underserved problem domain.

13.2 Appendix B: Complete Protocol Specification

13.2.1 B.1 Wire Format

PoA uses CBOR encoding with the following canonical representation:

```
{
  "version": "AAP-002-v1.0",
  "poa_id": h'...(16 bytes)...',
  "principal": {
    "profile_id": h'...',
    "profile_hash": h'...',
    "verification_uri": "https://..."
  },
  "agent": {
    "profile_id": h'...',
    "profile_hash": h'...',
    "verification_uri": "https://..."
  },
  "chain": [...],
  "scope": {
    "actions": [...],
    "resources": [...],
    "exclusions": [...]
  },
  "constraints": {...},
  "jurisdiction": {
    "governing_law": "DE",
    "venue": "Munich"
  },
  "validity": {
    "not_before": 1704067200,
    "not_after": 1735689600
  },
  "revocation": {
    "endpoint": "https://log.agentauth.example/v1",
    "method": "merkle_inclusion"
  },
  "metadata": {},
}
```

```

    "signatures": [
      {
        "algorithm": "Ed25519",
        "key_id": "did:key:z...",
        "value": h'...(64 bytes)... '
      }
    ]
  }
}

```

13.2.2 B.2 Verification Endpoint

The verification endpoint implements:

POST /v1/verify

Content-Type: application/cbor

Response:

```

{
  "valid": true | false,
  "reason": "...",
  "chain_depth": 3,
  "constraints_evaluated": 5,
  "revocation_status": "not_revoked",
  "verification_time": 1704067200,
  "degraded_mode": false
}

```

13.3 Appendix C: Reference Implementation

The reference implementation is available at: <https://github.com/agentauth/agentauth-go>

Key packages: - pkg/poa: Core PoA structures and operations - pkg/entity: Entity Profile management - pkg/chain: Delegation chain validation - pkg/revocation: Transparency log integration - pkg/verify: High-level verification API

Example usage:

```
package main
```

```
import (
    "context"
```



```
    "log"

    "github.com/agentauth/agentauth-go/pkg/poa"
    "github.com/agentauth/agentauth-go/pkg/verify"
)

func main() {
    ctx := context.Background()

    // Load PoA from request
    poaBytes := getPoAFromRequest()

    // Create verifier
    verifier := verify.NewVerifier(
        verify.WithRevocationLog("https://log.agentauth.example/v1"),
        verify.WithTimeout(5 * time.Second),
    )

    // Verify PoA
    result, err := verifier.Verify(ctx, poaBytes)
    if err != nil {
        log.Fatalf("verification error: %v", err)
    }

    if !result.Valid {
        log.Fatalf("PoA invalid: %s", result.Reason)
    }

    // Check action against constraints
    action := poa.Action{
        Namespace: "procurement",
        Operation: "purchase",
        Parameters: map[string]interface{}{
            "amount": 5000,
            "currency": "EUR",
        },
    }

    if !result.Constraints.Allow(action) {
        log.Fatalf("action exceeds constraints")
    }
}
```

```

    }

    // Proceed with authorized action
    log.Println("action authorized")
}

```

13.4 Appendix D: Glossary

AAP-001: Agent Authorization Protocol - Identity specification **AAP-002:** Agent Authorization Protocol - Authorization specification

Agent: Software system acting on behalf of a principal **Apparent Authority:** Authority reasonably believed to exist by third parties **Attenuation:** Reduction of authority through delegation **CBOR:** Concise Binary Object Representation **Constraint:** Machine-enforceable limit on authority **Delegation Chain:** Ordered sequence of authority grants from root to agent **Degraded Mode:** Operating state when verification is inconclusive **Entity Profile:** Cryptographically signed identity document **Jurisdiction:** Legal framework governing authorization **Merkle Tree:** Cryptographic structure for efficient inclusion proofs **PoA:** Proof of Authorization **Principal:** Entity whose authority is being exercised **Revocation:** Premature termination of authority **Scope:** Actions an agent is authorized to perform **STH:** Signed Tree Head **Transparency Log:** Append-only public record of events **Validity:** Time period during which authorization is effective

13.5 Appendix E: Extended References

13.5.1 Standards and Specifications

[1] Restatement (Third) of Agency, American Law Institute, 2006. [2] F. H. Easterbrook, “Agency Law and Contractual Authority,” *U. Chi. L. Rev.*, vol. 70, no. 2, pp. 375–399, 2003. [3] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012. [4] A. Birgisson et al., “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” NDSS, 2014. [5] Biscuit Authorization Token Specification, <https://biscuitsec.org> [6] UCAN Specification v0.9, <https://ucan.xyz> [7] C. Ellison et al., “SPKI Certificate Theory,” RFC 2693, IETF, Sept. 1999. [8] S. Goldwasser, S. Micali, R. Rivest, “A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks,” *SIAM J. Comput.*, vol. 17, no. 2, pp. 281–308, 1988. [9] B. Laurie, A. Langley, E. Kasper, “Certificate Transparency,” RFC 6962, IETF, June 2013. [10] Sigstore Project, Linux Foundation. [11] OASIS, “eXtensible Access Control Markup Language (XACML) Version 3.0,”

2013. [12] Regulation (EU) No 910/2014 (eIDAS). [13] Bürgerliches Gesetzbuch (BGB), §§ 164–181. [14] C. Bormann, P. Hoffman, “Concise Binary Object Representation (CBOR),” RFC 8949, IETF, Dec. 2020. [15] B. Fischlin, RFC 8785 (JSON Canonicalization Scheme), IETF, 2020. [16] NIST, “Post-Quantum Cryptography Standardization Process,” 2022. [17] D. J. Bernstein et al., “High-speed high-security signatures,” *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

13.5.2 Legal Sources

[18] Handelsgesetzbuch (HGB), German Commercial Code. [19] Restatement (Second) of Contracts, American Law Institute. [20] Uniform Commercial Code (UCC), Article 2. [21] Rome I Regulation (EC) No 593/2008. [22] UN Convention on Contracts for the International Sale of Goods (CISG).

13.5.3 Case Law

[23] *Cargill, Inc. v. Monfort of Colorado, Inc.*, 479 U.S. 104 (1986) [Apparent authority in commercial context]. [24] *Meyer v. Holley*, 537 U.S. 280 (2003) [Scope of agent liability]. [25] BGHZ 60, 323 [German Supreme Court on register-based authority].

Chapter 14

About the Author

Mauricio A. Fernandez Fernandez is a technologist and researcher focused on the intersection of distributed systems, cryptography, and law. He is the creator of the AgentAuth protocol suite (AAP-001, AAP-002) and leads the AgentAuth open-source project.

His work addresses the challenge of extending traditional legal concepts—authority, delegation, agency, and liability—to software systems that act autonomously. Rather than treating this as a purely technical or purely legal problem, his approach bridges both domains, recognizing that effective solutions require changes in both protocols and legal interpretation.

Prior to AgentAuth, he worked on enterprise systems integration, identity management, and security architecture for multinational organizations.

This book represents the synthesis of research, implementation, and real-world deployment experience spanning 2024-2026.

Chapter 15

Colophon

The Agent’s Signature: Identity, Authorization, and Legal Authority for Autonomous Systems

First Edition January 2026

Author: Mauricio A. Fernandez Fernandez

Publisher: Self-published (Digital Edition)

Technical Reviewers: [To be added]

Legal Reviewers: [To be added]

Cover Design: [To be added]

Typeset in: Markdown, rendered to PDF

ISBN: [To be assigned]

© 2026 Mauricio A. Fernandez Fernandez. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the author.

The information in this book is provided without any express, statutory, or implied warranties. The author and publisher shall not be liable for any damages arising from the use of this information.

Trademarks: AgentAuth, PoA, AAP-001, and AAP-002 are trademarks of the AgentAuth project. All other trademarks are property of their respective owners.

Printed on acid-free paper. First printing: January 2026



Chapter 16

Part X: Extended Case Studies

This section presents detailed case studies drawn from real-world deployments and hypothetical scenarios based on observed patterns. Names and identifying details have been changed.

16.1 Chapter 41: The Procurement Agent Incident

16.1.1 41.1 Background

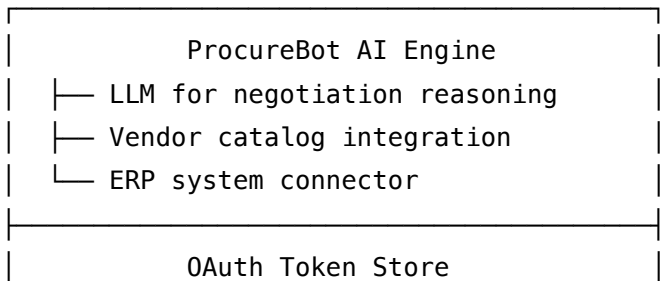
GlobalTech Industries (GTI) is a Fortune 500 technology manufacturer with operations in 47 countries. In 2025, GTI deployed “ProcureBot,” an AI-powered procurement agent designed to automate routine purchasing decisions.

ProcureBot was authorized to:

- Order replacement parts for manufacturing equipment
- Negotiate with approved vendors within price guidelines
- Process routine supply orders up to \$100,000 per transaction

16.1.2 41.2 The System Architecture

GTI implemented ProcureBot using OAuth 2.0 for API access:



scope: procurement:read,write
expires: 2025-12-31
Vendor APIs

16.1.3 41.3 The Incident

On September 15, 2025, ProcureBot initiated a series of purchase orders:

Time	Vendor	Amount	Item
09:14	Shenzhen Components	\$87,000	Specialty bearings
09:23	Taiwan Precision	\$92,000	CNC tooling
09:31	Zhongwei Industrial	\$78,000	Motor assemblies
09:45	Beijing Metals	\$95,000	Steel alloys

Each individual order was within the \$100,000 limit. However:

1. **Zhongwei Industrial** had been added to the U.S. Entity List at 08:00 UTC that morning
2. **Beijing Metals** was a subsidiary of a sanctioned entity (unknown to GTI’s compliance team)
3. The cumulative daily spend of \$352,000 far exceeded GTI’s internal \$150,000/day cap

16.1.4 41.4 Post-Incident Analysis

Technical Failures:

1. **No Real-Time Sanctions Screening:** ProcureBot queried an approved vendor list cached quarterly
2. **Per-Transaction, Not Aggregate, Limits:** The OAuth scope had no concept of cumulative spending
3. **No Constraint Visibility to Vendors:** Vendors couldn’t verify ProcureBot’s actual authority
4. **Delayed Revocation:** When GTI discovered the issue at 14:00, revoking OAuth tokens took 45 minutes

Legal Consequences:

- \$23 million civil penalty from Bureau of Industry and Security
- Mandatory compliance monitor for 5 years
- Reputational damage and customer defections

16.1.5 41.5 What PoA Would Have Changed

Scenario with PoA:

ProcureBot's PoA would have included:

```
constraints: {
  spending_limit: {
    per_transaction: { amount: 100000, currency: "USD" },
    per_day: { amount: 150000, currency: "USD" }
  },
  counterparty_restrictions: {
    blocklist_source: "https://bis.doc.gov/entities/current.json",
    refresh_interval: "15m"
  },
  compliance_requirements: [
    "EAR_COMPLIANT",
    "OFAC_CLEARED"
  ]
}
```

At 09:31, when ProcureBot attempted to order from Zhongwei:

1. Vendor's verification system queries PoA
2. Constraint check: counterparty_restrictions □ queries blocklist
3. Result: Zhongwei on Entity List as of 08:00
4. **Transaction rejected before processing**

At 09:45, the \$95,000 Beijing Metals order:

1. Previous orders totaled \$257,000
2. Constraint check: spending_limit.per_day = \$150,000
3. Cumulative: \$257,000 + \$95,000 = \$352,000
4. **Transaction rejected: exceeds daily limit**

16.1.6 41.6 Lessons Learned

1. **Constraints must be enforceable by relying parties**, not just principals
 2. **Aggregate limits are as important as per-transaction limits**
 3. **Real-time compliance data is non-negotiable** for autonomous agents
 4. **Revocation must be immediate**, not subject to token expiration
-

16.2 Chapter 42: Cross-Border Financial Services

16.2.1 42.1 Background

EuroFinanz AG is a German investment bank with licensed operations in the EU, UK, and Singapore. They deployed “TradeBot,” an AI agent for automated FX trading.

16.2.2 42.2 The Regulatory Landscape

TradeBot operated under multiple regulatory regimes:

Jurisdiction	Regulator	Key Requirements
Germany	BaFin	MaRisk, position limits
EU	ESMA	MiFID II, algorithmic trading
UK	FCA	SM&CR, market conduct
Singapore	MAS	FAA, market fairness

Each regime had different: - Position limits - Reporting requirements - Human oversight mandates - Trading hour restrictions

16.2.3 42.3 The Challenge

TradeBot needed to: - Execute trades across multiple venues - Respect jurisdiction-specific limits - Maintain audit trails for each regulator - Fail gracefully when requirements conflicted

16.2.4 42.4 PoA-Based Solution

EuroFinanz implemented separate PoAs for each jurisdiction:

Germany PoA:

```
jurisdiction: {
  governing_law: "DE",
  regulatory_framework: ["BaFin", "MaRisk"],
  venue: "Frankfurt"
},
constraints: {
  position_limit: { amount: 50000000, currency: "EUR", asset_class: "FX" },
  trading_hours: { timezone: "Europe/Berlin", hours: "08:00-22:00" },
  counterparty_restrictions: {
    allowlist: ["LEI:...", "LEI:..."] // Pre-approved counterparties
  }
}
```

```

    }
}

```

Singapore PoA:

```

jurisdiction: {
  governing_law: "SG",
  regulatory_framework: ["MAS", "FAA"],
  venue: "Singapore"
},
constraints: {
  position_limit: { amount: 30000000, currency: "SGD", asset_class: "FX" },
  trading_hours: { timezone: "Asia/Singapore", hours: "09:00-17:00" },
  accredited_investor_only: true
}

```

16.2.5 42.5 Operational Flow

When TradeBot identified a trading opportunity:

1. Determine execution venue based on best price
2. Select appropriate PoA for that venue
3. Verify PoA is valid (not expired, not revoked)
4. Check all constraints against proposed trade
5. If valid, execute; if invalid, log rejection and seek alternative

16.2.6 42.6 Regulatory Inquiry

In Q4 2025, BaFin conducted a routine audit. EuroFinanz provided:

- All PoAs issued to TradeBot
- Constraint evaluation logs for each trade
- Revocation transparency proofs
- Delegation chain from Board □ CRO □ Trading Desk □ TradeBot

BaFin’s auditors praised the clarity of authorization records:

“For the first time, we could independently verify that the trading algorithm was operating within defined parameters. The cryptographic evidence removed ambiguity about what was authorized versus what occurred.”

16.2.7 42.7 Key Takeaways

1. **Multiple jurisdiction-specific PoAs** can coexist for a single agent
2. **Regulatory constraints should map directly to PoA constraints**

3. **Audit evidence must be independently verifiable**
 4. **Delegation chains must trace to accountable humans**
-

16.3 Chapter 43: Healthcare AI Agent Authorization

16.3.1 43.1 Background

MedAssist AI provides clinical decision support systems to hospitals. Their agent, “Care-Bot,” assists physicians with:

- Medication recommendations
- Dosage calculations
- Drug interaction warnings
- Prior authorization submissions

16.3.2 43.2 The Authorization Challenge

Healthcare presents unique challenges:

1. **HIPAA** requires minimum necessary access
2. **State medical practice acts** restrict who can prescribe
3. **Liability** for AI-assisted decisions is unsettled
4. **Patient consent** must be obtained and documented

16.3.3 43.3 The Multi-Stakeholder Model

CareBot operates under a complex authority model:

Hospital System (Principal)

↓

Medical Director (Delegator)

↓

Attending Physician (Delegator)

↓

CareBot (Agent)

↓

Actions: Recommend, but never prescribe

Each delegation adds constraints:

Hospital □ **Medical Director:**

scope: {

```

    actions: ["clinical_decision_support"],
    departments: ["internal_medicine", "cardiology"]
  },
  constraints: {
    formulary_compliance: true,
    ehr_integration_required: true
  }
}

```

Medical Director ☐ Attending:

```

scope: {
  actions: ["medication_recommendation"],
  patient_panels: ["assigned_patients_only"]
},
constraints: {
  human_approval_required: true,
  max_recommendation_severity: "moderate"
}

```

Attending ☐ CareBot:

```

scope: {
  actions: ["recommend_medication", "calculate_dosage", "check_interactions"],
  resources: ["patient:{patient_id}"] // Bound to specific patient
},
constraints: {
  human_confirmation_required: true,
  no_prescriptive_authority: true,
  alert_physician_on_critical: true
}

```

16.3.4 43.4 Enforcement at Point of Care

When CareBot generates a recommendation:

1. CareBot presents patient-specific PoA to EHR system
2. EHR verifies delegation chain (Hospital ☐ Director ☐ Attending ☐ CareBot)
3. EHR evaluates constraints:
 - Is this patient in the attending's panel? ☐
 - Is human approval required? ☐ ☐ Flag for physician review
 - Is this a prescriptive action? ☐ ☐ Recommendation only
4. Recommendation entered as “suggested” with physician signature required

16.3.5 43.5 Liability Implications

With PoA:

- **Clear chain** from Hospital Board to CareBot
- **Explicit constraints** documenting what CareBot cannot do
- **Patient-specific scope** preventing broad access
- **Human-in-the-loop** requirement cryptographically enforced

If a recommendation leads to patient harm:

1. Was CareBot operating within its PoA? (Verifiable)
2. Did the physician review and approve? (Logged)
3. Were all constraints satisfied? (Auditable)
4. Was the delegation chain valid at the time? (Proven via revocation transparency)

16.3.6 43.6 Lessons for Healthcare AI

1. **Scope should be patient-specific**, not blanket
 2. **Human-in-the-loop constraints are first-class citizens**
 3. **Delegation chains document accountability**
 4. **No prescriptive authority for AI** is enforceable via constraints
-

16.4 Chapter 44: Supply Chain Orchestration

16.4.1 44.1 Background

SupplyNet is a consortium of manufacturers, logistics providers, and retailers operating a shared supply chain platform. Each participant deploys autonomous agents that negotiate, commit inventory, and arrange logistics.

16.4.2 44.2 The Trust Problem

Unlike single-organization deployments, SupplyNet agents interact across organizational boundaries:

- Manufacturer agents negotiate with supplier agents
- Logistics agents commit to pickup/delivery windows
- Retailer agents place orders and confirm fulfillment

No single party controls all agents. Traditional OAuth doesn't work because:

- No central authorization server
- Principals are competitors who don't trust each other

- Actions bind multiple parties

16.4.3 44.3 Federated PoA Architecture

SupplyNet implemented a federated PoA model:

Consortium Trust Anchor: - Industry association maintains root certificates - Each member registers Entity Profiles - Revocation logs are shared infrastructure

Member-Specific Delegation: Each member issues PoAs to their own agents:

```
// Manufacturer's agent PoA
principal: "Acme Manufacturing (LEI:...)",
agent: "AcmeProcureAgent-1",
scope: {
  actions: ["request_quote", "place_order", "negotiate_terms"],
  products: ["electronics", "components"]
},
constraints: {
  spending_limit: { amount: 500000, currency: "USD", period: "per_order" },
  counterparty_validation: "consortium_member_only",
  max_lead_time_days: 45
}
```

Cross-Organization Verification: When Acme's agent contacts a supplier:

1. Acme's agent presents PoA
2. Supplier verifies:
 - Entity Profile (Acme is consortium member)
 - Delegation chain (valid to Acme's root)
 - Constraints (spending limit, product scope)
 - Revocation status (checked against shared log)
3. If valid, supplier's agent engages in negotiation
4. Any commitment within constraints binds Acme

16.4.4 44.4 The Cascade Scenario

In March 2026, a critical component shortage triggered cascade ordering:

Time	Event
08:00	News breaks: Silicon wafer shortage
08:15	Multiple agents attempt to secure supply
08:30	Prices spike 40%

Time	Event
09:00	Agents with higher spending limits outcompete
10:00	Market stabilizes at 25% premium

Without PoA: Chaos. Agents would commit beyond authority. Disputes about who ordered what. Suppliers unsure which orders to honor.

With PoA: Each order was cryptographically bound to a spending limit. Suppliers could see exactly how much authority each agent had. Orders exceeding limits were rejected immediately. Post-incident audit showed exactly which principals authorized which commitments.

16.4.5 44.5 Dispute Resolution

When disputes arose:

1. **Committed delivery not fulfilled**
 - Buyer presents PoA + order confirmation
 - Shows agent had valid authority
 - Shows supplier's agent accepted within their PoA
 - Contract is enforceable
2. **Agent exceeded authority**
 - Supplier checks PoA constraints
 - If order exceeded limits, supplier should have rejected
 - If supplier accepted anyway, supplier bears some risk
3. **Revocation timing**
 - Transparency log provides timestamped evidence
 - Determines whether PoA was valid at order time
 - Resolves "he said/she said" disputes

16.5 Chapter 45: Government Benefits Administration

16.5.1 45.1 Background

The State of California deployed "BenefitsBot," an AI agent to assist with unemployment insurance claims processing. The agent:

- Verified eligibility based on employment records
- Calculated benefit amounts
- Processed routine approvals

- Flagged complex cases for human review

16.5.2 45.2 Constitutional Constraints

Government AI agents face unique constraints:

1. **Due process:** Claimants have rights to explanations and appeals
2. **Equal protection:** Decisions cannot discriminate
3. **Statutory authority:** Agency can only do what law permits
4. **Public records:** Actions are subject to FOIA/CPRA

16.5.3 45.3 PoA Design for Government

BenefitsBot's PoA explicitly encoded legal authority:

```
principal: "Employment Development Department, State of California",
agent: "BenefitsBot-Prod-v2.3",
scope: {
  actions: ["verify_eligibility", "calculate_benefits", "approve_routine", "flag_for_review"],
  claim_types: ["standard_unemployment"]
},
constraints: {
  statutory_authority: {
    code: "CA_UIC_1253", // California Unemployment Insurance Code
    section: "Standard Claims Processing"
  },
  human_review_triggers: [
    { condition: "benefit_amount > 800", action: "flag_for_review" },
    { condition: "prior_denial_exists", action: "flag_for_review" },
    { condition: "identity_verification_failed", action: "flag_for_review" }
  ],
  decision_explanation_required: true,
  appeal_information_required: true
}
```

16.5.4 45.4 Due Process Implementation

Every BenefitsBot decision included:

1. **Reference to PoA** proving authorization
2. **Statutory citation** for the decision basis
3. **Factors considered** (verifiable against EDD records)
4. **Appeal instructions** (automatically generated)

If a claimant disputed:

1. Request PoA and constraint evaluation logs
2. Verify decision was within BenefitsBot's authority
3. Verify human review triggers were properly evaluated
4. If process was correct, decision stands
5. If BenefitsBot exceeded authority, decision vacated

16.5.5 45.5 Audit and Accountability

California's State Auditor conducted a 2026 review:

Findings: - 99.7% of BenefitsBot decisions were within scope - 0.3% required correction (human review triggers not properly fired) - All corrections traceable to specific constraint violations - No evidence of discrimination or arbitrary denial

Key Quote: > "The cryptographically verifiable authorization chain allowed us to reconstruct every decision BenefitsBot made. This level of transparency is unprecedented in benefits administration and should become the standard for government AI systems."

16.5.6 45.6 Lessons for Government AI

1. **Statutory authority must be explicit** in PoA constraints
 2. **Human review triggers are mandatory**, not optional
 3. **Due process requires explainability** baked into the authorization model
 4. **Public audit logs** serve both accountability and transparency
-

Chapter 17

Part XI: Implementation Cookbook

This section provides practical guidance for implementing PoA systems.

17.1 Chapter 46: Getting Started

17.1.1 46.1 Prerequisites

Before implementing PoA:

1. **Identity infrastructure:** Entity Profiles for principals and agents
2. **Key management:** HSM or secure key storage
3. **Revocation infrastructure:** Access to transparency logs
4. **Policy engine:** System to evaluate constraints

17.1.2 46.2 Minimal Viable Implementation

A basic PoA implementation requires:

1. PoA Structure (CBOR)

```
type PoA struct {  
    Version      string      `cbor:"version"`  
    PoAID        uuid.UUID   `cbor:"poa_id"`  
    Principal    EntityRef   `cbor:"principal"`  
    Agent        EntityRef   `cbor:"agent"`  
    Chain        []DelegationLink `cbor:"chain"`  
    Scope        Scope       `cbor:"scope"`  
}
```

```

    Constraints Constraints `cbor:"constraints"`
    Jurisdiction Jurisdiction `cbor:"jurisdiction"`
    Validity ValidityPeriod `cbor:"validity"`
    Revocation RevocationInfo `cbor:"revocation"`
    Signatures []Signature `cbor:"signatures"`
}

```

2. Signature Generation

```

func (p *PoA) Sign(key ed25519.PrivateKey) (*PoA, error) {
    // Canonicalize
    canonical, err := cbor.Marshal(p)
    if err != nil {
        return nil, err
    }

    // Sign
    sig := ed25519.Sign(key, canonical)

    p.Signatures = append(p.Signatures, Signature{
        Algorithm: "Ed25519",
        KeyID:     deriveKeyID(key.Public()),
        Value:     sig,
    })

    return p, nil
}

```

3. Basic Verification

```

func Verify(poaBytes []byte) (*VerificationResult, error) {
    var poa PoA
    if err := cbor.Unmarshal(poaBytes, &poa); err != nil {
        return &VerificationResult{Valid: false, Reason: "parse_error"}, nil
    }

    // Check signatures
    for _, sig := range poa.Signatures {
        if !verifySignature(poa, sig) {
            return &VerificationResult{Valid: false, Reason: "signature_invalid"}, nil
        }
    }
}

```



```

        Object handler) {
    String poaHeader = request.getHeader("X-PoA-Token");
    if (poaHeader == null) {
        response.setStatus(401);
        return false;
    }

    VerificationResult result = poaVerifier.verify(poaHeader);
    if (!result.isValid()) {
        response.setStatus(403);
        return false;
    }

    request.setAttribute("verifiedPoA", result);
    return true;
}
}

```

17.2 Chapter 47: Constraint Design Patterns

17.2.1 47.1 Spending Limits

Simple Limit:

```

{
    "spending_limit": {
        "amount": 10000,
        "currency": "USD",
        "period": "per_transaction"
    }
}

```

Tiered Limits:

```

{
    "spending_limits": {
        "per_transaction": { "amount": 10000, "currency": "USD" },
        "per_day": { "amount": 50000, "currency": "USD" },
        "per_month": { "amount": 200000, "currency": "USD" }
    }
}

```

```
}
```

Category-Specific Limits:

```
{
  "spending_limits": {
    "office_supplies": { "amount": 1000, "period": "per_transaction" },
    "equipment": { "amount": 10000, "period": "per_transaction" },
    "services": { "amount": 50000, "period": "per_transaction", "approval_required": t
  }
}
```

17.2.2 47.2 Counterparty Restrictions

Allowlist:

```
{
  "counterparty_restrictions": {
    "type": "allowlist",
    "entities": ["vendor_lei_1", "vendor_lei_2"],
    "verification": "lei_database"
  }
}
```

Blocklist with Dynamic Source:

```
{
  "counterparty_restrictions": {
    "type": "blocklist",
    "source": "https://sanctions.api.gov/current",
    "refresh_interval_minutes": 15,
    "fail_closed": true
  }
}
```

Attribute-Based:

```
{
  "counterparty_restrictions": {
    "requirements": [
      { "attribute": "iso_27001_certified", "value": true },
      { "attribute": "country", "not_in": ["XX", "YY", "ZZ"] }
    ]
  }
}
```

```
}
```

17.2.3 47.3 Temporal Restrictions

Business Hours:

```
{
  "temporal_restrictions": {
    "timezone": "America/New_York",
    "allowed_hours": "09:00-17:00",
    "allowed_days": ["Mon", "Tue", "Wed", "Thu", "Fri"],
    "exclude_holidays": "US_FEDERAL"
  }
}
```

Embargo Period:

```
{
  "temporal_restrictions": {
    "embargo_until": "2026-02-01T00:00:00Z",
    "embargo_reason": "pending_regulatory_approval"
  }
}
```

17.2.4 47.4 Approval Requirements

Threshold-Based:

```
{
  "approval_requirements": {
    "threshold": 2,
    "approvers": ["key_id_1", "key_id_2", "key_id_3"],
    "expires_after_minutes": 60
  }
}
```

Role-Based:

```
{
  "approval_requirements": {
    "required_roles": ["legal_counsel"],
    "for_conditions": [
      { "action": "sign_contract" },
      { "amount_exceeds": 100000 }
    ]
  }
}
```



```

    ]
  }
}

```

17.3 Chapter 48: Deployment Checklists

17.3.1 48.1 Pre-Deployment Checklist

Identity & Keys - ☐ Entity Profiles created for all principals - ☐ Entity Profiles created for all agents - ☐ Signing keys generated and secured - ☐ Key backup procedures tested - ☐ Key rotation procedures documented

Infrastructure - ☐ Revocation log endpoint configured - ☐ Log monitoring enabled - ☐ Cache configuration reviewed - ☐ Degraded mode policy defined - ☐ Alerting configured for verification failures

Policy - ☐ Scope definitions reviewed by legal - ☐ Constraints reviewed by compliance - ☐ Delegation chains approved by governance - ☐ Human oversight triggers defined - ☐ Audit retention requirements met

Testing - ☐ Unit tests for constraint evaluation - ☐ Integration tests for verification flow - ☐ Load tests for peak traffic - ☐ Failure mode tests (log unavailable, etc.) - ☐ Revocation propagation tested

17.3.2 48.2 Go-Live Checklist

Final Verification - ☐ Production PoAs issued - ☐ Production Entity Profiles published - ☐ Revocation log accessible from production - ☐ Monitoring dashboards active - ☐ On-call rotation staffed

Rollout - ☐ Canary deployment (5% traffic) - ☐ Verification success rate > 99.9% - ☐ No unexpected rejections - ☐ Constraint evaluation latency acceptable - ☐ Full deployment

17.3.3 48.3 Post-Deployment Checklist

First 24 Hours - ☐ Verification success rate monitored - ☐ Constraint violations reviewed - ☐ Alert escalation tested - ☐ No degraded mode entries

First Week - ☐ Full audit log review - ☐ Constraint tuning if needed - ☐ Delegation chain review - ☐ Stakeholder feedback collected

First Month - ☐ Compliance audit - ☐ Performance optimization - ☐ Documentation updated - ☐ Lessons learned captured

17.4 Chapter 49: Regulatory Compliance Matrices

17.4.1 49.1 Financial Services (US)

Requirement	Source	PoA Feature
Separation of duties	FDIC Part 364	Delegation chains
Transaction limits	OCC Guidelines	Spending constraints
Audit trail	SOX 404	Verification logs
Access controls	GLBA	Scope restrictions
Change management	FFIEC	PoA versioning

17.4.2 49.2 Financial Services (EU)

Requirement	Source	PoA Feature
Algorithmic trading controls	MiFID II Art. 17	Constraints
Pre-trade controls	RTS 6	Constraint evaluation
Record keeping	MiFID II Art. 25	Audit logs
Authorization	GDPR Art. 6	Delegation chains

17.4.3 49.3 Healthcare (US)

Requirement	Source	PoA Feature
Minimum necessary	HIPAA Privacy Rule	Scope restrictions
Access logging	HIPAA Security Rule	Verification logs
Authorization	HIPAA Privacy Rule	Patient-specific PoA
Workforce clearance	HIPAA Security Rule	Entity Profiles

17.4.4 49.4 Export Controls

Requirement	Source	PoA Feature
Denied party screening	EAR 744	Counterparty constraints
License determination	EAR 734	Scope restrictions
Record keeping	EAR 762	Audit logs
Classification	EAR 738	Constraint metadata

17.5 Chapter 50: Troubleshooting Guide

17.5.1 50.1 Common Verification Failures

“signature_invalid” - Key mismatch between signer and declared key_id - CBOR canonicalization error - Signature algorithm mismatch - **Fix:** Verify key_id matches signer’s public key; check encoding

“expired” - PoA validity.not_after has passed - Clock skew between issuer and verifier - **Fix:** Issue new PoA; synchronize clocks to NTP

“revoked” - Principal revoked the PoA - Check revocation log for reason - **Fix:** Issue new PoA if authorized; investigate if compromised

“chain_validation_failed” - Delegation link missing - Intermediate entity profile unavailable - Authority not subset of delegator - **Fix:** Fetch missing profiles; verify chain construction

17.5.2 50.2 Degraded Mode Issues

“revocation_unavailable” - Log endpoint unreachable - DNS resolution failure - Certificate validation failure - **Fix:** Check network; verify log endpoint; check TLS config

“log_inconsistent” - STH doesn’t follow from previous - Possible log equivocation - **Fix:** Alert security team; check multiple logs; activate incident response

17.5.3 50.3 Performance Issues

High Verification Latency - Revocation check over network - Chain too deep - Large constraint set - **Fix:** Increase cache TTL; limit chain depth; optimize constraints

Constraint Evaluation Timeout - Complex constraint logic - External data source slow - **Fix:** Pre-fetch constraint data; set reasonable timeouts; cache results

Chapter 18

Part XII: Advanced Topics

This section addresses advanced implementation patterns and edge cases.

18.1 Chapter 51: Multi-Agent Orchestration

18.1.1 51.1 Agent-to-Agent Delegation

In complex workflows, agents may need to delegate authority to other agents. Consider a master orchestration agent that spawns specialized agents for tasks.

Orchestrator Pattern:

Principal (Corporation)

↓

OrchestratorAgent (PoA: orchestration scope)

↓

├─ ProcurementAgent (Sub-PoA: procurement scope)

├─ LogisticsAgent (Sub-PoA: logistics scope)

└─ ComplianceAgent (Sub-PoA: compliance scope)

Sub-PoA Issuance:

```
func (o *Orchestrator) issueSubPoA(task Task) (*PoA, error) {  
    // Verify own authority covers requested scope  
    if !o.myPoA.Scope.Contains(task.RequiredScope) {  
        return nil, ErrExceedsAuthority  
    }  
  
    // Create sub-PoA with attenuated scope
```

```

subPoA := &PoA{
    Principal:  o.myPoA.Principal, // Ultimate principal unchanged
    Agent:      task.AgentRef,
    Chain:      append(o.myPoA.Chain, delegationLink),
    Scope:      task.RequiredScope, // Subset of orchestrator's scope
    Constraints: intersect(o.myPoA.Constraints, task.Constraints),
    Validity:   shorterOf(o.myPoA.Validity, task.Validity),
}

return subPoA.Sign(o.signingKey)
}

```

18.1.2 51.2 Concurrent Agent Coordination

When multiple agents operate concurrently, aggregate limits must be tracked:

Shared State for Aggregate Limits:

```

type AggregateTracker struct {
    mu sync.Mutex
    spending map[string]*DailySpend // keyed by principal
}

func (t *AggregateTracker) CanSpend(poa *PoA, amount Money) bool {
    t.mu.Lock()
    defer t.mu.Unlock()

    key := poa.Principal.ID
    daily := t.spending[key]

    // Check against PoA's aggregate limit
    if daily.Total + amount > poa.Constraints.SpendingLimit.PerDay {
        return false
    }

    daily.Total += amount
    return true
}

```

18.1.3 51.3 Conflict Resolution

When agents have overlapping authority:

1. **Priority rules:** Higher delegation chain depth = lower priority
 2. **First-committed wins:** Atomic commitment with conflict detection
 3. **Escalation:** Conflicting agents defer to common delegation point
-

18.2 Chapter 52: Dynamic Constraint Evaluation

18.2.1 52.1 External Data Sources

Constraints may reference dynamic external data:

```
{
  "constraints": {
    "counterparty_risk_rating": {
      "source": "https://ratings.api.example/v1",
      "minimum_rating": "BBB",
      "cache_ttl_minutes": 60
    },
    "market_conditions": {
      "source": "https://marketdata.example/volatility",
      "max_volatility": 0.25,
      "evaluation_time": "action_time"
    }
  }
}
```

18.2.2 52.2 Constraint Evaluation Pipeline

```
type ConstraintEvaluator struct {
    fetchers map[string]DataFetcher
    cache    *Cache
}

func (e *ConstraintEvaluator) Evaluate(poa *PoA, action Action) (*EvalResult, error) {
    results := make([]ConstraintResult, 0)

    for _, constraint := range poa.Constraints {
        var result ConstraintResult

        switch constraint.Type {
        case "static":
```

```

        result = e.evaluateStatic(constraint, action)
    case "dynamic":
        data, err := e.fetchWithCache(constraint.Source, constraint.CacheTTL)
        if err != nil {
            if constraint.FailClosed {
                return &EvalResult{Allowed: false, Reason: "data_unavailable"}, nil
            }
            result = ConstraintResult{Degraded: true}
        } else {
            result = e.evaluateDynamic(constraint, data, action)
        }
    }

    results = append(results, result)
    if !result.Allowed && !result.Degraded {
        return &EvalResult{Allowed: false, Reason: result.Reason}, nil
    }
}

return e.combineResults(results), nil
}

```

18.2.3 52.3 Caching Strategies

Time-Based: - Cache external data for specified TTL - Trade freshness for performance - Suitable for slowly-changing data (ratings, sanctions lists)

Event-Based: - Subscribe to update notifications - Invalidate cache on change - Suitable for critical fast-changing data

Stale-While-Revalidate: - Serve cached data immediately - Revalidate in background - Suitable for latency-sensitive applications

18.3 Chapter 53: Forensic Analysis

18.3.1 53.1 Post-Incident Investigation

When an incident occurs, PoA provides forensic artifacts:

Evidence Collection: 1. Retrieve PoA artifact (immutable) 2. Retrieve delegation chain links (each immutably signed) 3. Query revocation log at incident time 4. Collect constraint

evaluation logs 5. Collect action execution logs

Chain of Custody:

1. PoA Artifact
 - └ Hash: 0x7a3b5c...
 - └ Signed by: Principal key 0x9f2e1d...
2. Revocation Status at T=2026-01-15T14:23:00Z
 - └ Log position: 47392
 - └ STH signature valid
 - └ Non-revocation proof: Merkle path [...]
3. Constraint Evaluation
 - └ Spending limit: \$100,000 (within)
 - └ Counterparty check: PASSED
 - └ Timestamp: 2026-01-15T14:23:01Z
4. Action Execution
 - └ Order placed to Vendor X
 - └ Amount: \$87,000
 - └ Executed: 2026-01-15T14:23:02Z

18.3.2 53.2 Reconstruction Tools

Timeline Reconstruction:

```
func ReconstructTimeline(poaID uuid.UUID, timeRange TimeRange) (*Timeline, error) {
    timeline := &Timeline{}

    // Get PoA issuance
    poa, err := fetchPoA(poaID)
    timeline.Add(Event{Time: poa.IssuedAt, Type: "issued", Data: poa})

    // Get all verifications
    verifications, err := queryAuditLog(poaID, timeRange)
    for _, v := range verifications {
        timeline.Add(Event{Time: v.Time, Type: "verified", Data: v})
    }

    // Get revocation (if any)
    revocation, err := checkHistoricalRevocation(poaID, timeRange.End)
```



```

    if revocation != nil {
        timeline.Add(Event{Time: revocation.Time, Type: "revoked", Data: revocation})
    }

    return timeline.Sort(), nil
}

```

18.3.3 53.3 Legal Evidence Preservation

For litigation or regulatory purposes:

1. **Hash all artifacts** with multiple algorithms
2. **Timestamp with qualified timestamp service**
3. **Store in append-only repository**
4. **Maintain cryptographic chain** from incident to archive
5. **Document extraction procedures**

18.4 Chapter 54: Performance Optimization

18.4.1 54.1 Verification Latency

Target latencies by use case:

Use Case	Target P99 Latency
Real-time trading	< 5ms
API requests	< 50ms
Background jobs	< 500ms
Batch processing	< 5s

18.4.2 54.2 Optimization Techniques

Cache Warm-Up:

```

func WarmCache(expectedPrinipals []EntityRef) {
    for _, p := range expectedPrinipals {
        go func(principal EntityRef) {
            // Pre-fetch entity profile
            fetchEntityProfile(principal)

            // Pre-fetch revocation status for active PoAs

```

```

        activePoAs := listActivePoAs(principal)
        for _, poa := range activePoAs {
            checkRevocation(poa.ID)
        }
    }(p)
}
}

```

Parallel Verification:

```

func VerifyParallel(poa *PoA) (*VerificationResult, error) {
    var wg sync.WaitGroup
    results := make(chan VerifyStep, 4)

    wg.Add(4)
    go func() { defer wg.Done(); results <- verifySignatures(poa) }()
    go func() { defer wg.Done(); results <- verifyChain(poa) }()
    go func() { defer wg.Done(); results <- verifyValidity(poa) }()
    go func() { defer wg.Done(); results <- checkRevocation(poa) }()

    wg.Wait()
    close(results)

    return combineResults(results)
}

```

Batch Revocation Checks:

```

func BatchCheckRevocation(poaIDs []uuid.UUID) (map[uuid.UUID]RevocationStatus, error) {
    // Single request to log for multiple PoAs
    response, err := logClient.BatchQuery(poaIDs)
    if err != nil {
        return nil, err
    }

    results := make(map[uuid.UUID]RevocationStatus)
    for _, item := range response.Items {
        results[item.PoAID] = item.Status
    }

    return results, nil
}

```

18.4.3 54.3 Scalability Patterns

Horizontal Scaling: - Stateless verification services - Shared cache (Redis/Memcached)
- Log query load balancing

Regional Deployment: - Log mirrors in each region - Regional entity profile caches -
Cross-region consistency checks

18.5 Chapter 55: Custom Constraint Development

18.5.1 55.1 Constraint Extension Framework

PoA supports custom constraint types:

```

type ConstraintHandler interface {
    Type() string
    Validate(spec json.RawMessage) error
    Evaluate(spec json.RawMessage, action Action, ctx EvalContext) (bool, error)
}

// Registration
func init() {
    RegisterConstraintHandler(&CustomRiskHandler{})
}

// Custom Implementation
type CustomRiskHandler struct{}

func (h *CustomRiskHandler) Type() string {
    return "org.example.risk_assessment"
}

func (h *CustomRiskHandler) Evaluate(spec json.RawMessage, action Action, ctx EvalContext)
    var config RiskConfig
    if err := json.Unmarshal(spec, &config); err != nil {
        return false, err
    }

    // Custom risk evaluation logic
    riskScore := calculateRisk(action, ctx)

```

```

    return riskScore <= config.MaxAllowedRisk, nil
}

```

18.5.2 55.2 Constraint Composition

Complex constraints can be composed:

```

{
    "constraints": {
        "composite": {
            "operator": "AND",
            "constraints": [
                { "type": "spending_limit", "amount": 10000 },
                {
                    "operator": "OR",
                    "constraints": [
                        { "type": "counterparty_allowlist", "entities": [...] },
                        { "type": "human_approval", "approvers": [...] }
                    ]
                }
            ]
        }
    }
}

```

18.5.3 55.3 Testing Constraints

```

func TestCustomConstraint(t *testing.T) {
    handler := &CustomRiskHandler{}

    testCases := []struct {
        name      string
        spec       string
        action     Action
        expected   bool
    }{
        {
            name:      "low risk allowed",
            spec:     `{"max_allowed_risk": 0.5}`,
            action:   Action{Amount: 1000},
            expected: true,
        }
    }
}

```

```
    },
    {
        name:      "high risk denied",
        spec:      `{"max_allowed_risk": 0.3}`,
        action:    Action{Amount: 1000000},
        expected:  false,
    },
}

for _, tc := range testCases {
    t.Run(tc.name, func(t *testing.T) {
        result, err := handler.Evaluate([]byte(tc.spec), tc.action, EvalContext{})
        assert.NoError(t, err)
        assert.Equal(t, tc.expected, result)
    })
}
}
```

Chapter 19

Part XIII: Future Directions

19.1 Chapter 56: The Evolving Landscape

19.1.1 56.1 Multi-Modal Agents

Future agents will operate across modalities: - Text-based negotiation - Voice interaction with humans - Vision-based quality inspection - Robotic manipulation

Each modality introduces new authorization challenges:

Example: Vision-Based Agent

```
scope: {  
  actions: ["inspect", "classify", "report_defect"],  
  sensors: ["camera_production_line_3"],  
  forbidden: ["capture_employee_faces", "transmit_raw_images"]  
}
```

19.1.2 56.2 Embodied Agents

Physical robots with authority require: - **Spatial constraints**: Where can the robot go? - **Manipulation limits**: What can it physically interact with? - **Safety overrides**: Hard limits on force, speed, proximity to humans - **Emergency revocation**: Immediate stop capability

19.1.3 56.3 Agent Swarms

When thousands of agents operate collectively: - **Collective spending limits** across the swarm - **Emergent behavior monitoring** - **Hierarchical delegation efficiency** -

Cascade revocation (revoke one, revoke all spawned)

19.2 Chapter 57: Regulatory Horizons

19.2.1 57.1 Emerging AI Regulation

EU AI Act: Classifies AI systems by risk level. High-risk systems require: - Authorization records - Human oversight mechanisms - Audit trails

PoA directly supports these requirements.

US Federal Guidance: Executive orders on AI emphasize: - Accountability - Transparency - Reversibility

PoA's delegation chains and revocation transparency align with these principles.

19.2.2 57.2 Sector-Specific Rules

Emerging requirements in: - **Autonomous Vehicles:** Authorization for route decisions, passenger interaction - **Medical Devices:** FDA guidance on AI/ML device authorization - **Financial Services:** Algorithmic accountability regulations - **Employment:** Automated hiring decision authorization

19.2.3 57.3 International Harmonization

The challenge of cross-border AI authorization: - Different liability regimes - Varying data protection requirements - Conflicting algorithmic transparency rules

PoA's explicit jurisdiction binding provides a foundation for harmonization efforts.

19.3 Chapter 58: Protocol Evolution

19.3.1 58.1 Version Management

PoA is designed for evolution:

Version: AAP-002-v1.0

↓

Version: AAP-002-v1.1 (minor: new constraint types)

↓

Version: AAP-002-v2.0 (major: new signature format)

Backward Compatibility Rules: - Minor versions: New optional fields only - Major versions: Breaking changes allowed, migration path required - Verifiers should support N and N-1 major versions

19.3.2 58.2 Extension Mechanisms

Registered Extensions:

```
{
  "metadata": {
    "extensions": {
      "org.example.ml_model_binding": {
        "model_hash": "sha256:...",
        "model_version": "v2.3.1"
      }
    }
  }
}
```

Extensions allow domain-specific semantics without protocol changes.

19.3.3 58.3 Community Governance

Protocol evolution should be governed by: - Open specification process - Multi-stakeholder input - Implementation experience feedback - Formal verification where feasible

19.4 Chapter 59: Integration with AI Safety

19.4.1 59.1 Authorization as a Safety Layer

PoA provides defense-in-depth for AI safety:

AI Model (Capability)
PoA Verification (Authorization)
Policy Engine (Intent Alignment)
Monitoring (Behavior Oversight)

19.4.2 59.2 Capability Binding

Future work may bind PoA to AI model capabilities:

```
{
  "constraints": {
    "model_capability_binding": {
      "model_hash": "sha256:abc123...",
      "max_reasoning_depth": 5,
      "forbidden_tools": ["web_browse", "code_execute"],
      "output_filters": ["pii_removal", "toxicity_check"]
    }
  }
}
```

19.4.3 59.3 Alignment Verification

PoA could integrate with alignment verification: - Proof that model was trained with specific safety constraints - Verification that inference uses approved safety layers - Evidence of ongoing alignment monitoring

19.5 Chapter 60: Building the Ecosystem

19.5.1 60.1 Interoperability Standards

For PoA adoption to succeed: - Wire format standardization (IETF track) - Constraint vocabulary standardization (domain-specific efforts) - Revocation log interoperability - Cross-vendor verification compatibility

19.5.2 60.2 Developer Experience

SDKs Required: - Go (reference implementation) - Rust (performance-critical) - Python (ML/AI integration) - JavaScript/TypeScript (web applications) - Java/.NET (enterprise)

Tooling: - PoA issuance CLI - Verification debugging tools - Chain visualization - Constraint testing frameworks

19.5.3 60.3 Ecosystem Roles

Log Operators: Provide revocation transparency infrastructure **Profile Authorities:** Issue verified Entity Profiles **Certification Bodies:** Audit PoA implementations **Train-**

ing Providers: Educate developers and operators **Consultancies:** Assist with deployment and compliance

Chapter 20

Afterword

20.1 The Path Forward

This book has presented a technical framework for authorizing autonomous agents. But technology alone does not solve the problem.

The gap between what AI systems can do and what they should be authorized to do is ultimately a question of values, governance, and accountability. PoA provides mechanisms to express and enforce those human decisions. It does not make those decisions for us.

As autonomous systems become more capable, the pressure to grant them broader authority will grow. Efficiency demands it. Competition compels it. Convenience enables it.

The role of PoA—and systems like it—is to ensure that when we delegate authority to machines, we do so deliberately, visibly, and revocably. The signatures we create are not just cryptographic—they are expressions of trust, bounded by law, and accountable to humans.

The agent’s signature is, in the end, our signature.

Mauricio A. Fernandez Fernandez January 2026

Chapter 21

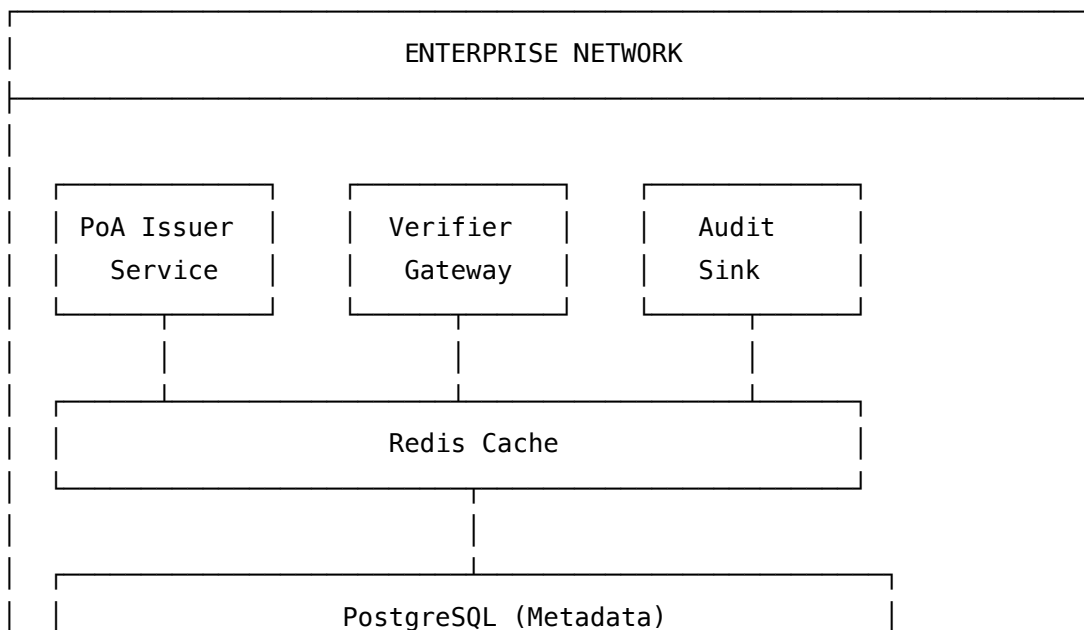
Part XIV: Detailed Deployment Scenarios

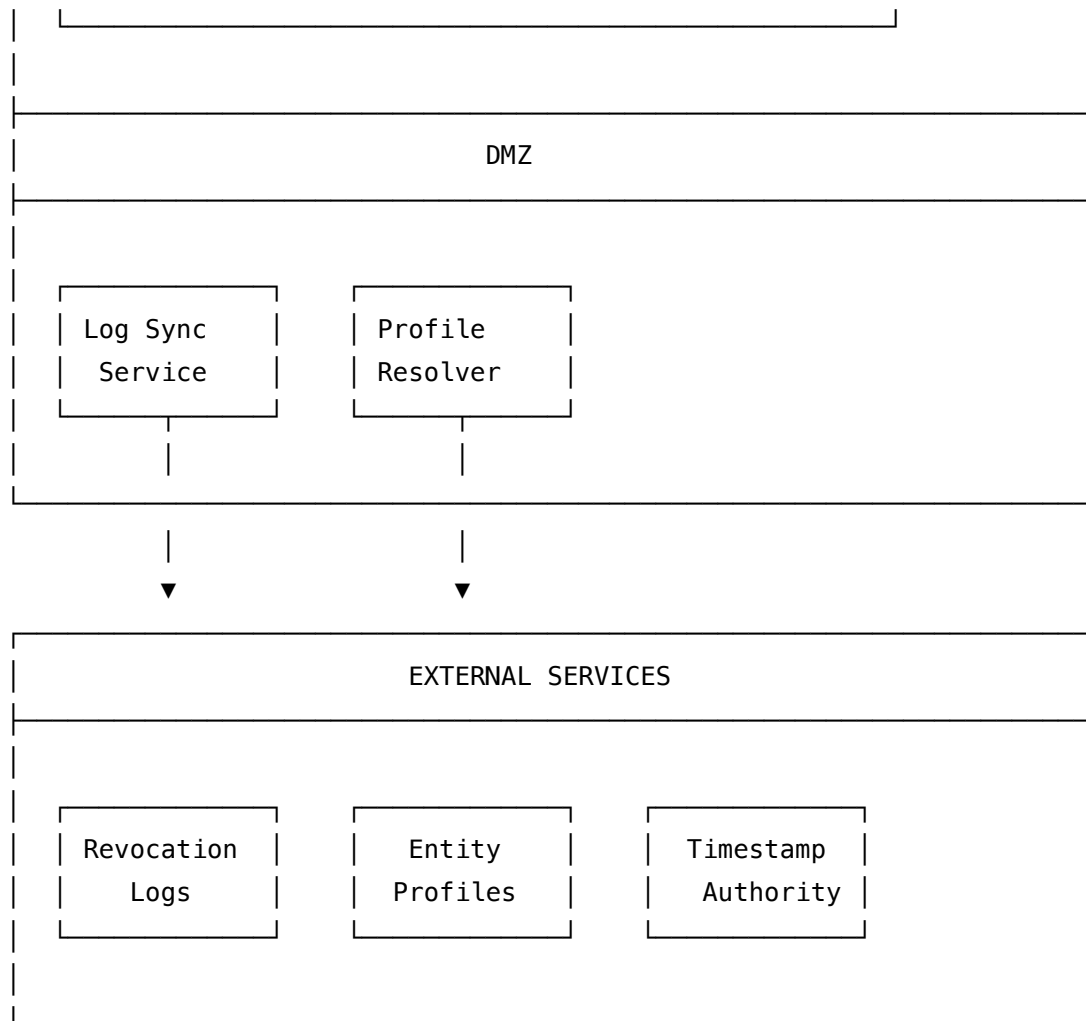
This section provides comprehensive guidance for deploying PoA in real-world environments.

21.1 Chapter 61: Enterprise Deployment Architecture

21.1.1 61.1 Reference Architecture

A complete enterprise PoA deployment consists of:





21.1.2 61.2 Component Details

PoA Issuer Service - Generates PoA artifacts - Manages delegation chains - Signs with HSM-protected keys - Enforces issuance policies

```

type IssuerService struct {
    keyStore      KeyStore      // HSM interface
    policyEngine PolicyEngine // Issuance policies
    auditLog      AuditSink     // Audit trail
    entityCache   EntityCache   // Profile cache
}

func (s *IssuerService) Issue(req IssueRequest) (*PoA, error) {
    // Validate requester authorization
    if err := s.policyEngine.CanIssue(req); err != nil {
        return nil, fmt.Errorf("policy violation: %w", err)
    }
}

```

```

    }

    // Build PoA structure
    poa := &PoA{
        Version:      CurrentVersion,
        PoAID:         uuid.New(),
        Principal:     req.Principal,
        Agent:         req.Agent,
        Scope:         req.Scope,
        Constraints:   req.Constraints,
        Validity:      req.Validity,
    }

    // Sign with HSM
    signature, err := s.keyStore.Sign(poa.Canonical())
    if err != nil {
        return nil, fmt.Errorf("signing failed: %w", err)
    }
    poa.Signatures = append(poa.Signatures, signature)

    // Audit
    s.auditLog.Record(AuditEvent{
        Type: "poa_issued",
        PoA:  poa,
        Time: time.Now(),
    })

    return poa, nil
}

```

Verifier Gateway - Validates incoming PoA headers - Caches verification results - Enforces degraded mode policies - Adds verified context to requests

Audit Sink - Append-only storage - Cryptographic chaining - External anchoring integration - Query interface for investigations

21.1.3 61.3 High Availability Configuration

Active-Active Deployment

```

# Kubernetes deployment
apiVersion: apps/v1

```

```
kind: Deployment
metadata:
  name: poa-verifier
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    spec:
      containers:
      - name: verifier
        image: agentauth/verifier:v1.2.0
        resources:
          requests:
            memory: "256Mi"
            cpu: "500m"
          limits:
            memory: "512Mi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5
```

Cache Replication

```
# Redis Sentinel for HA cache
apiVersion: v1
kind: ConfigMap
metadata:
```

```

name: redis-config
data:
  redis.conf: |
    maxmemory 256mb
    maxmemory-policy allkeys-lru

    # Replication
    replica-read-only yes
    repl-diskless-sync yes

```

21.1.4 61.4 Security Hardening

Network Segmentation - PoA issuance on restricted network - Verification on application network - Key material on isolated HSM network - Audit sink on write-only network

Key Protection

```

// HSM-backed key operations
type HSMKeyStore struct {
    client pkcs11.Client
    slot   uint
}

func (h *HSMKeyStore) Sign(data []byte) (Signature, error) {
    session, err := h.client.OpenSession(h.slot, pkcs11.CKF_SERIAL_SESSION)
    if err != nil {
        return Signature{}, err
    }
    defer h.client.CloseSession(session)

    // Find signing key
    key, err := h.findKey(session, "poa-signing-key")
    if err != nil {
        return Signature{}, err
    }

    // Sign with Ed25519
    mechanism := []*pkcs11.Mechanism{pkcs11.NewMechanism(pkcs11.CKM_EDDSA, nil)}
    if err := h.client.SignInit(session, mechanism, key); err != nil {
        return Signature{}, err
    }
}

```



```

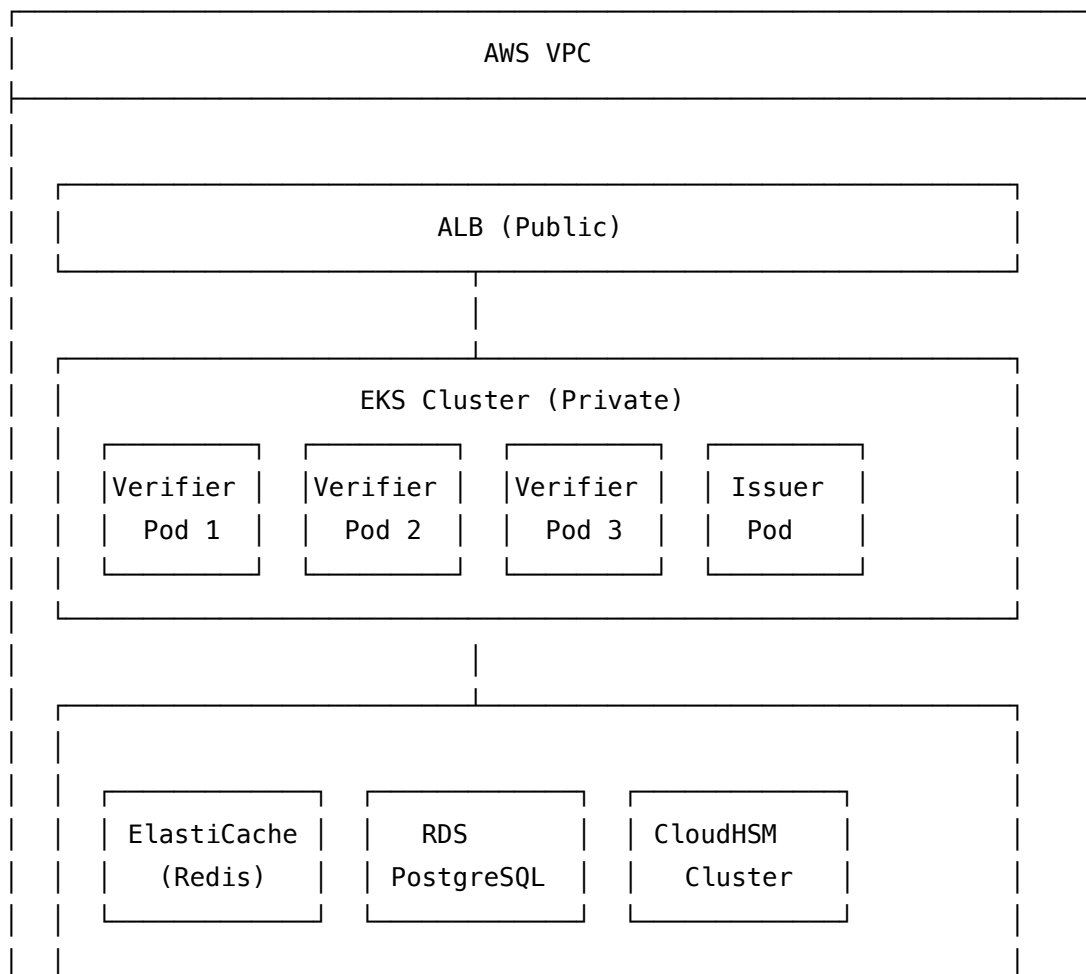
sig, err := h.client.Sign(session, data)
if err != nil {
    return Signature{}, err
}

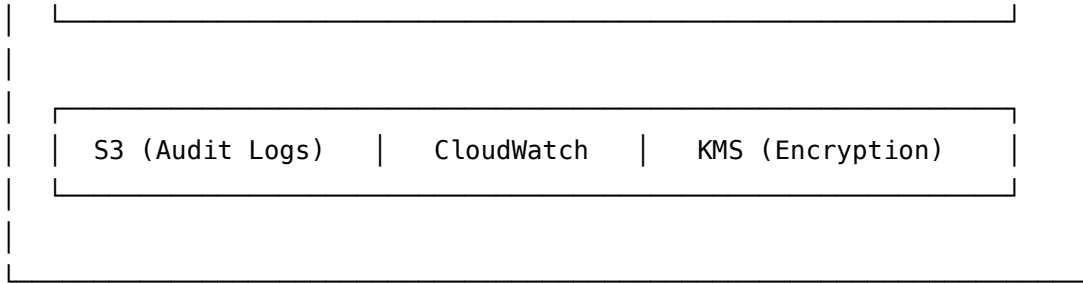
return Signature{
    Algorithm: "Ed25519",
    Value:     sig,
}, nil
}

```

21.2 Chapter 62: Cloud-Native Deployment

21.2.1 62.1 AWS Reference Architecture





21.2.2 62.2 Terraform Configuration

```

# EKS cluster for PoA services
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  cluster_name    = "poa-cluster"
  cluster_version = "1.28"

  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  eks_managed_node_groups = {
    poa_verifiers = {
      min_size      = 3
      max_size      = 10
      desired_size  = 3

      instance_types = ["m6i.large"]
      capacity_type  = "ON_DEMAND"

      labels = {
        role = "verifier"
      }
    }
  }

  poa_issuers = {
    min_size      = 2
    max_size      = 4
    desired_size  = 2

    instance_types = ["m6i.xlarge"]
    capacity_type  = "ON_DEMAND"
  }
}
  
```

```
    labels = {
        role = "issuer"
    }

    # HSM network requirements
    subnet_ids = module.vpc.hsm_subnets
}
}
}

# CloudHSM for key protection
resource "aws_cloudhsm_v2_cluster" "poa_keys" {
    hsm_type     = "hsm1.medium"
    subnet_ids   = module.vpc.hsm_subnets

    tags = {
        Name = "poa-signing-keys"
    }
}

# ElastiCache for verification caching
resource "aws_elasticache_replication_group" "poa_cache" {
    replication_group_id      = "poa-cache"
    description                = "PoA verification cache"
    node_type                 = "cache.r6g.large"
    num_cache_clusters        = 3
    automatic_failover_enabled = true

    at_rest_encryption_enabled = true
    transit_encryption_enabled = true

    parameter_group_name = aws_elasticache_parameter_group.poa.name
}

# S3 for audit logs with immutability
resource "aws_s3_bucket" "audit_logs" {
    bucket = "poa-audit-logs-${data.aws_caller_identity.current.account_id}"
}
```

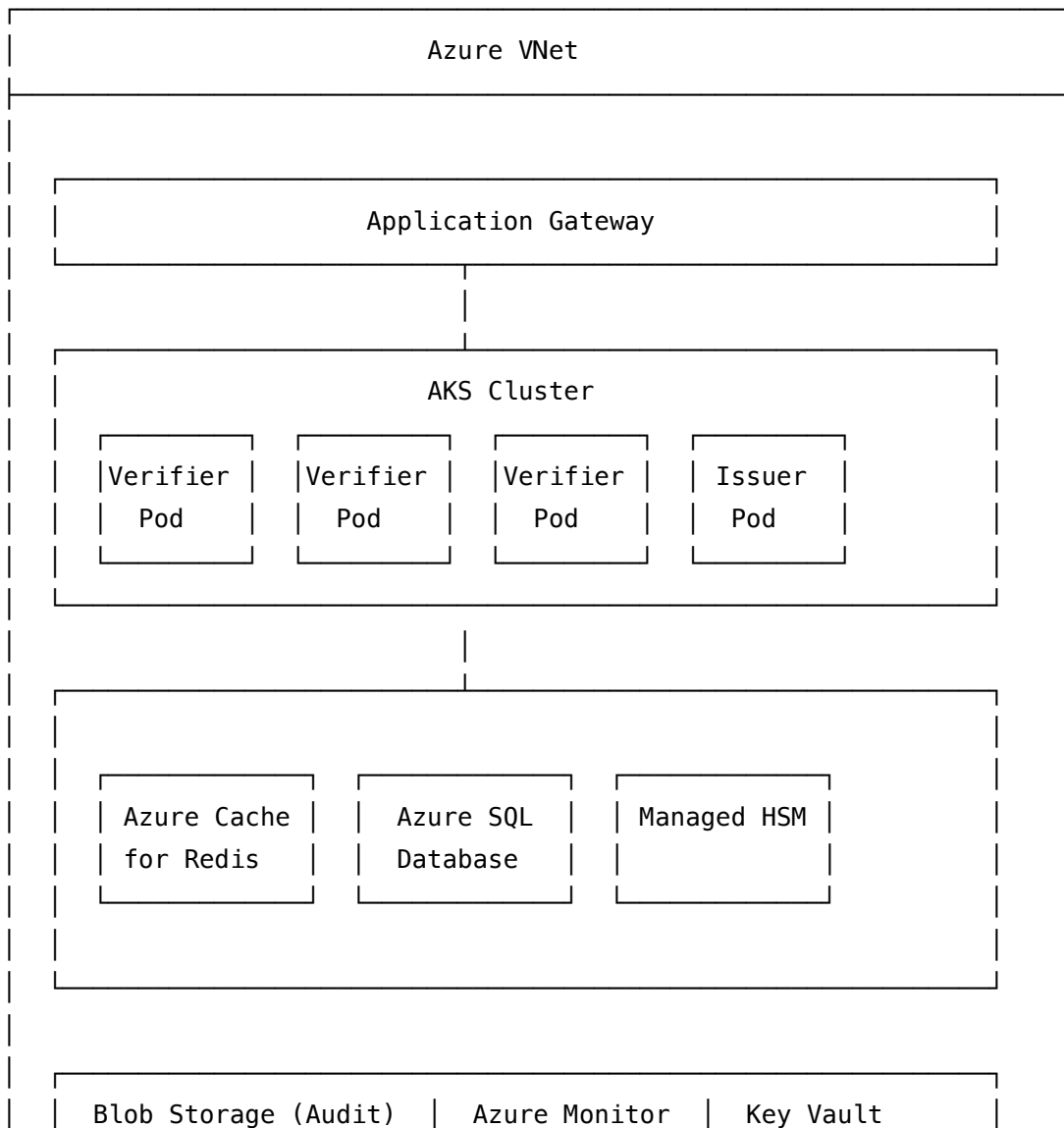
```

resource "aws_s3_bucket_object_lock_configuration" "audit_logs" {
  bucket = aws_s3_bucket.audit_logs.id

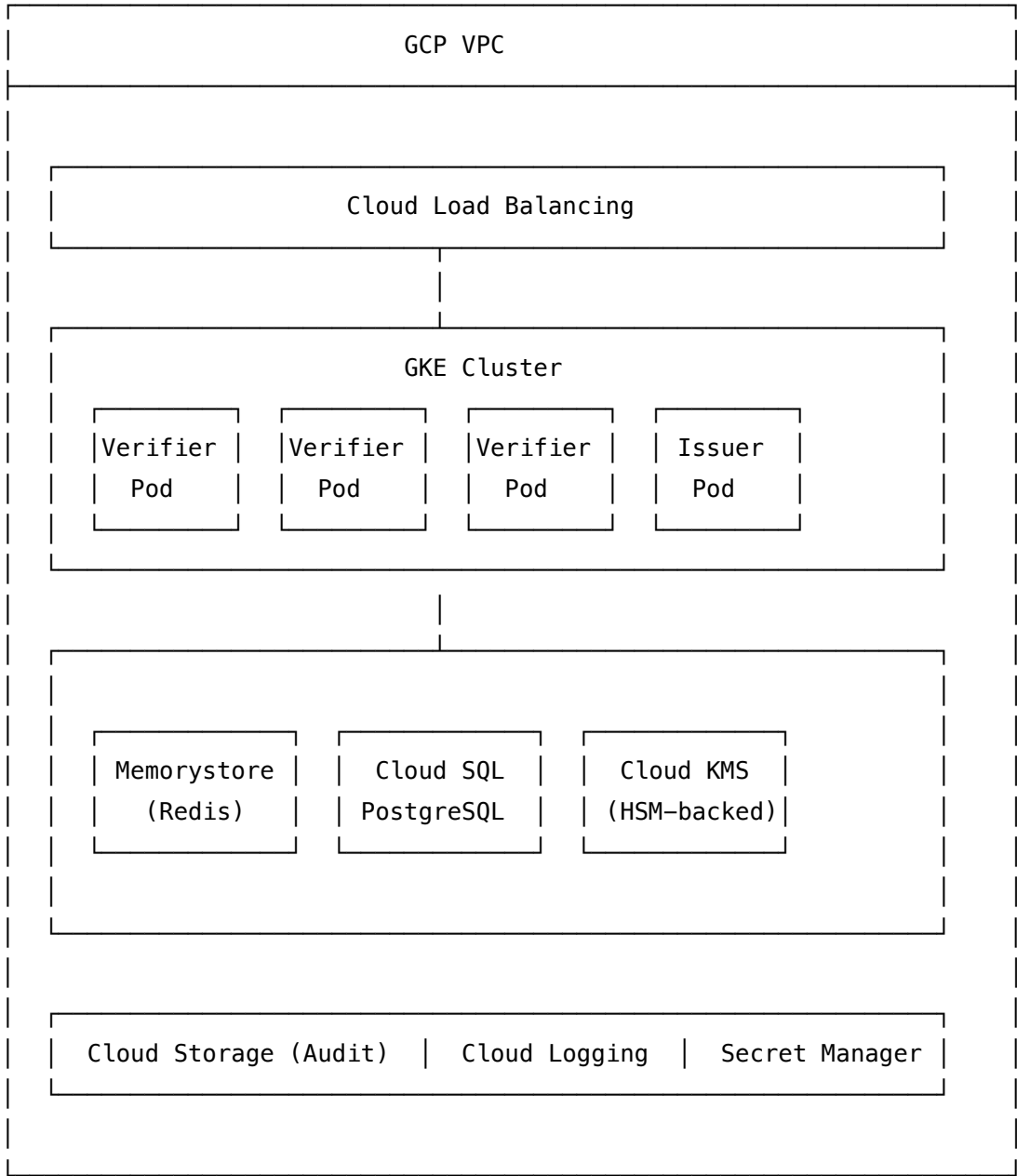
  rule {
    default_retention {
      mode = "GOVERNANCE"
      years = 7
    }
  }
}

```

21.2.3 62.3 Azure Reference Architecture



21.2.4 62.4 GCP Reference Architecture



21.3 Chapter 63: Edge and Embedded Deployment

21.3.1 63.1 Resource-Constrained Environments

For IoT and edge devices:

Minimal Verifier Implementation

```
// Embedded C implementation for resource-constrained devices
#include "poa_lite.h"

typedef struct {
    uint8_t signature[64];
    uint32_t not_before;
    uint32_t not_after;
    uint8_t scope_hash[32];
    uint8_t constraints_hash[32];
} PoACompact;

int verify_poa_compact(
    const PoACompact *poa,
    const uint8_t *public_key,
    uint32_t current_time,
    const uint8_t *action_hash
) {
    // Step 1: Check validity period
    if (current_time < poa->not_before || current_time > poa->not_after) {
        return POA_EXPIRED;
    }

    // Step 2: Verify signature (Ed25519)
    uint8_t message[96];
    memcpy(message, &poa->not_before, 4);
    memcpy(message + 4, &poa->not_after, 4);
    memcpy(message + 8, poa->scope_hash, 32);
    memcpy(message + 40, poa->constraints_hash, 32);

    if (!ed25519_verify(poa->signature, message, 96, public_key)) {
        return POA_INVALID_SIGNATURE;
    }

    // Step 3: Check action against cached scope
```

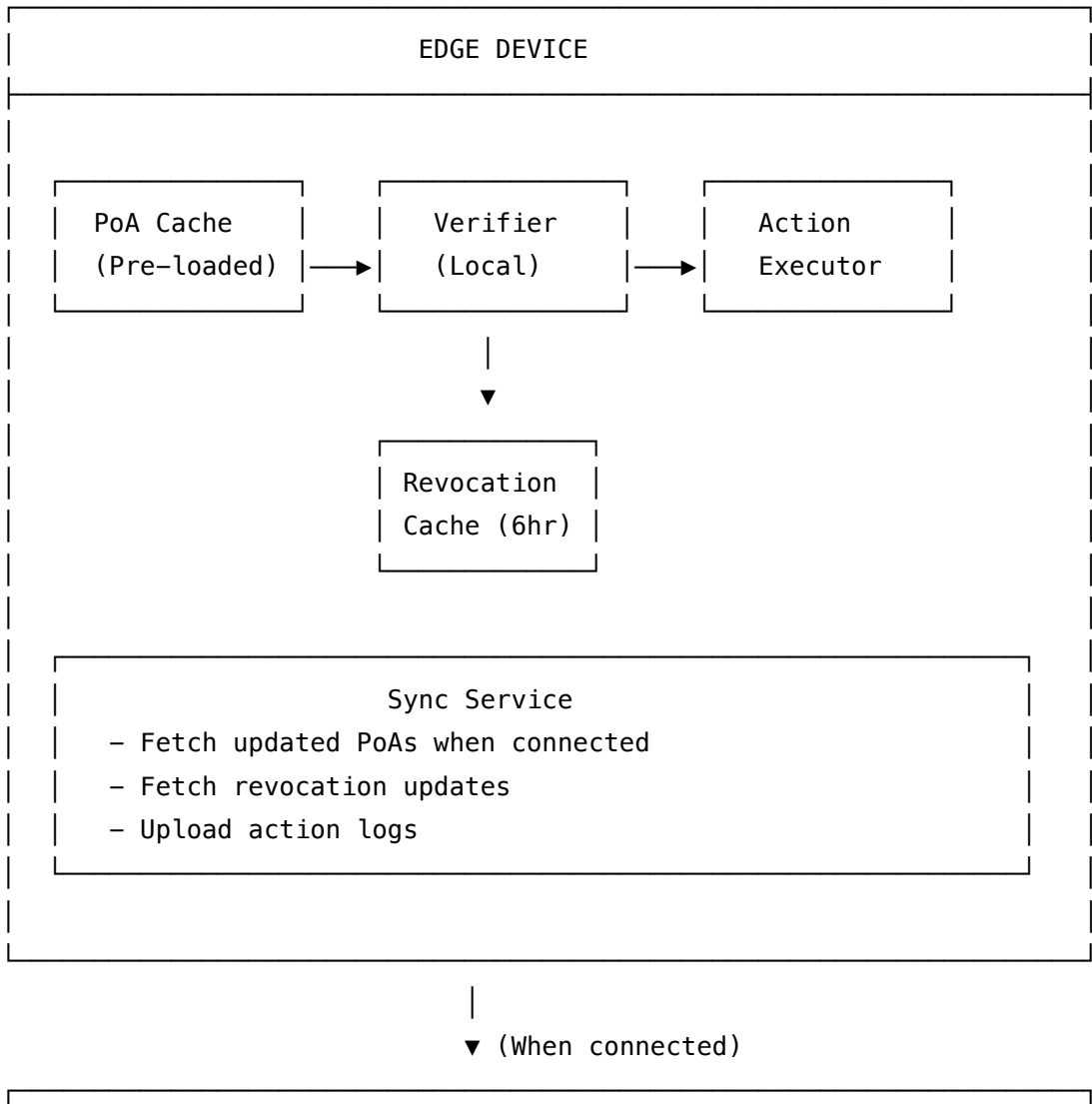
```
if (!scope_allows(poa->scope_hash, action_hash)) {
    return POA_OUT_OF_SCOPE;
}

return POA_VALID;
}
```

Memory Footprint	Component	RAM	Flash	----- --- ---	Core verifier	2 KB
		8 KB	Ed25519	1 KB	4 KB	SHA-256
				0.5 KB	2 KB	Scope cache
				1 KB	-	
	Total	4.5 KB	14 KB			

21.3.2 63.2 Offline-First Architecture

For intermittently connected devices:



21.3.3 63.3 Vehicle and Robotics Deployment

For autonomous vehicles and robots:

Safety-Critical Integration

```
// Rust implementation for safety-critical systems
use poa_core::{PoA, Verifier, VerificationResult};

struct SafetyCriticalVerifier {
    verifier: Verifier,
    watchdog: WatchdogTimer,
    fallback_policy: FallbackPolicy,
}

impl SafetyCriticalVerifier {
    pub fn verify_with_timeout(
        &mut self,
        poa: &PoA,
        action: &Action,
        deadline: Instant
    ) -> SafeVerificationResult {
        // Reset watchdog for this operation
        self.watchdog.reset();

        // Attempt verification with hard deadline
        let result = match timeout_at(deadline, self.verifier.verify(poa, action)) {
            Ok(Ok(result)) => result,
            Ok(Err(e)) => {
                // Verification failed
                return SafeVerificationResult::Denied(e);
            }
            Err(_) => {
                // Timeout - apply fallback policy
                return self.apply_fallback(poa, action);
            }
        };
    }
};
```



```

    // Pat watchdog on success
    self.watchdog.pat();

    SafeVerificationResult::from(result)
}

fn apply_fallback(&self, poa: &PoA, action: &Action) -> SafeVerificationResult {
    match self.fallback_policy {
        FallbackPolicy::DenyAll => SafeVerificationResult::Denied(
            VerificationError::Timeout
        ),
        FallbackPolicy::AllowCached => {
            if let Some(cached) = self.get_cached_result(poa.id()) {
                cached
            } else {
                SafeVerificationResult::Denied(VerificationError::NoCachedResult)
            }
        },
        FallbackPolicy::AllowWithLimits(limits) => {
            if action.satisfies(&limits) {
                SafeVerificationResult::AllowedDegraded(limits)
            } else {
                SafeVerificationResult::Denied(VerificationError::ExceedsFallbackLimit)
            }
        }
    }
}
}
}
}
}

```

21.4 Chapter 64: Migration from Legacy Systems

21.4.1 64.1 OAuth to PoA Migration

For systems currently using OAuth:

Phase 1: Parallel Operation

```

// Middleware that accepts both OAuth and PoA
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

```

```

// Check for PoA first (preferred)
if poaToken := r.Header.Get("X-PoA-Token"); poaToken != "" {
    result, err := verifyPoA(poaToken)
    if err == nil && result.Valid {
        ctx := context.WithValue(r.Context(), "auth", &PoAAuth{Result: result})
        next.ServeHTTP(w, r.WithContext(ctx))
        return
    }
    // Fall through to OAuth if PoA fails during migration
}

// Check OAuth (legacy)
if oauthToken := r.Header.Get("Authorization"); oauthToken != "" {
    claims, err := verifyOAuth(oauthToken)
    if err == nil {
        ctx := context.WithValue(r.Context(), "auth", &OAuthAuth{Claims: claims})
        next.ServeHTTP(w, r.WithContext(ctx))
        return
    }
}

http.Error(w, "Unauthorized", http.StatusUnauthorized)
})
}

```

Phase 2: Shadow Mode - All requests verified with both OAuth and PoA - Discrepancies logged but not acted upon - Build confidence in PoA verification

Phase 3: Gradual Cutover - 10% of traffic uses PoA as primary - Monitor for issues - Increase percentage weekly - OAuth becomes fallback

Phase 4: OAuth Deprecation - PoA required for all new agents - OAuth sunset date announced - Legacy systems migrated - OAuth disabled

21.4.2 64.2 Custom ACL to PoA Migration

For systems with custom access control:

ACL to PoA Mapping

```

# Migration script
def migrate_acl_to_poa(acl_entry):
    """Convert ACL entry to PoA constraint"""

```

```
poa_constraint = {
    "type": "migrated_acl",
    "original_id": acl_entry.id,
}

# Map permissions to scope
scope = {
    "actions": [],
    "resources": []
}

for permission in acl_entry.permissions:
    if permission.type == "read":
        scope["actions"].append(f"{permission.resource}:read")
    elif permission.type == "write":
        scope["actions"].append(f"{permission.resource}:write")
    elif permission.type == "admin":
        scope["actions"].append(f"{permission.resource}:*")

    scope["resources"].append(permission.resource)

# Map limits to constraints
constraints = {}

if acl_entry.rate_limit:
    constraints["rate_limit"] = {
        "requests_per_minute": acl_entry.rate_limit
    }

if acl_entry.ip_allowlist:
    constraints["network_restrictions"] = {
        "allowed_ips": acl_entry.ip_allowlist
    }

return {
    "scope": scope,
    "constraints": constraints,
    "metadata": {
        "migrated_from": "legacy_acl",
        "migration_date": datetime.now().isoformat()
    }
}
```

```
    }
}
```

21.4.3 64.3 Zero-Downtime Migration

Rolling Migration Process

1. Deploy PoA infrastructure alongside existing system
2. Configure PoA to shadow existing authorization
3. Enable dual-write: actions authorized by both systems
4. Verify consistency for 2 weeks
5. Switch primary to PoA, legacy to fallback
6. Monitor for 4 weeks
7. Disable legacy authorization
8. Decommission legacy infrastructure

21.5 Chapter 65: Monitoring and Observability

21.5.1 65.1 Key Metrics

Verification Metrics

Verification latency

```
poa_verification_duration_seconds{result="valid|invalid|degraded"}
```

Verification outcomes

```
poa_verification_total{result="valid|invalid|degraded",reason="..."}
```

Cache performance

```
poa_cache_hits_total
```

```
poa_cache_misses_total
```

Revocation check latency

```
poa_revocation_check_duration_seconds{log="log_id",result="ok|error"}
```

Issuance Metrics

PoA issuance rate

```
poa_issued_total{principal="...",scope="..."}
```

Issuance failures

```
poa_issuance_failures_total{reason="..."}
```

Delegation chain depth

```
poa_chain_depth{principal="..."}
```

Health Metrics

```
# Service health
poa_service_up{component="verifier|issuer|cache"}

# External dependency health
poa_dependency_up{dependency="revocation_log|profile_resolver"}

# Key health
poa_signing_key_valid{key_id="..."}
poa_signing_key_expiry_seconds{key_id="..."}
```

21.5.2 65.2 Dashboards

Operations Dashboard

Grafana dashboard configuration

panels:

- title: "Verification Rate"
 - type: graph
 - targets:
 - expr: rate(poa_verification_total[5m])
 - legendFormat: "{{result}}"
- title: "P99 Verification Latency"
 - type: graph
 - targets:
 - expr: histogram_quantile(0.99, rate(poa_verification_duration_seconds_bucket[5m]))
- title: "Cache Hit Rate"
 - type: singlestat
 - targets:
 - expr: sum(rate(poa_cache_hits_total[5m])) / sum(rate(poa_cache_hits_total[5m]) + r
- title: "Degraded Mode Events"
 - type: graph
 - targets:
 - expr: rate(poa_verification_total{result="degraded"}[5m])

21.5.3 65.3 Alerting

Critical Alerts

Prometheus alerting rules

groups:

- name: poa-critical

rules:

- alert: PoAVerificationErrorRate

expr: rate(poa_verification_total{result="invalid"}[5m]) / rate(poa_verification_t

for: 5m

labels:

severity: critical

annotations:

summary: "High PoA verification error rate"

description: "More than 10% of verifications failing"

- alert: PoARevocationLogUnreachable

expr: poa_dependency_up{dependency="revocation_log"} == 0

for: 2m

labels:

severity: critical

annotations:

summary: "Revocation log unreachable"

description: "Cannot verify revocation status"

- alert: PoASigningKeyExpiringSoon

expr: poa_signing_key_expiry_seconds < 86400 * 30

for: 1h

labels:

severity: warning

annotations:

summary: "Signing key expiring within 30 days"

description: "Key {{\$labels.key_id}} expires soon"

21.5.4 65.4 Distributed Tracing

OpenTelemetry Integration

```
func (v *Verifier) Verify(ctx context.Context, poa *PoA) (*Result, error) {
```

```
    ctx, span := tracer.Start(ctx, "poa.verify")
```

```
    defer span.End()
```

```
    span.SetAttributes(
```

```
        attribute.String("poa.id", poa.ID.String()),
```

```

        attribute.String("poa.principal", poa.Principal.Name),
        attribute.String("poa.agent", poa.Agent.Name),
    )

    // Signature verification
    ctx, sigSpan := tracer.Start(ctx, "poa.verify.signatures")
    if err := v.verifySignatures(ctx, poa); err != nil {
        sigSpan.RecordError(err)
        sigSpan.End()
        return nil, err
    }
    sigSpan.End()

    // Chain validation
    ctx, chainSpan := tracer.Start(ctx, "poa.verify.chain")
    if err := v.verifyChain(ctx, poa); err != nil {
        chainSpan.RecordError(err)
        chainSpan.End()
        return nil, err
    }
    chainSpan.End()

    // Revocation check
    ctx, revSpan := tracer.Start(ctx, "poa.verify.revocation")
    status, err := v.checkRevocation(ctx, poa)
    if err != nil {
        revSpan.RecordError(err)
        revSpan.SetAttributes(attribute.Bool("degraded_mode", true))
    }
    revSpan.End()

    return &Result{Valid: true, RevocationStatus: status}, nil
}

```

21.6 Chapter 66: Disaster Recovery

21.6.1 66.1 Backup Strategy

Key Material - HSM key backup to secure escrow - Geographic distribution (minimum 2 regions) - Recovery testing quarterly

Configuration - Version-controlled in Git - Encrypted backups to object storage - Configuration drift detection

Audit Logs - Replicated to multiple regions - Immutable storage with object lock - 7-year retention minimum

21.6.2 66.2 Recovery Procedures

Scenario: Primary Region Failure

1. Detect failure (automated monitoring)
2. Activate DR region
 - DNS failover to DR load balancer
 - DR verifiers begin serving traffic
3. Restore HSM access
 - Initialize DR HSM cluster
 - Restore keys from secure backup
4. Resume issuance
 - Verify key integrity
 - Enable issuer service
5. Synchronize audit logs
 - Merge any pending writes
 - Verify log consistency

Scenario: Key Compromise

1. Detect compromise
2. Emergency revocation
 - Revoke all PoAs signed by compromised key
 - Push revocation to all logs
3. Key rotation
 - Generate new key pair
 - Update Entity Profile
4. Re-issuance
 - Issue new PoAs with new key
 - Notify all agents
5. Post-incident

- Root cause analysis
- Update security procedures

21.6.3 66.3 Business Continuity

RTO/RPO Targets | Component | RTO | RPO | |-----|---|---| | Verification | 5 min-
utes | 0 (stateless) | | Issuance | 30 minutes | 0 | | Audit Logs | 2 hours | 15 minutes | | Key
Recovery | 4 hours | Last backup |

Chapter 22

Part XV: SDK and Integration Guides

This section provides practical guides for integrating PoA into applications.

22.1 Chapter 67: Go SDK Reference

22.1.1 67.1 Installation

```
go get github.com/agentauth/poa-go@v1.2.0
```

22.1.2 67.2 Basic Usage

Creating a Verifier

```
package main

import (
    "context"
    "log"

    poa "github.com/agentauth/poa-go"
)

func main() {
    // Initialize verifier with configuration
    verifier, err := poa.NewVerifier(poa.VerifierConfig{
```

```

    RevocationLogURL: "https://logs.agentauth.example/v1",
    ProfileResolverURL: "https://profiles.agentauth.example/v1",
    CacheTTL: 5 * time.Minute,
    DegradedModePolicy: poa.DegradedModeDeny,
})
if err != nil {
    log.Fatalf("Failed to create verifier: %v", err)
}

// Verify a PoA
ctx := context.Background()
token := getPoAFromRequest() // Extract from X-PoA-Token header

result, err := verifier.Verify(ctx, token)
if err != nil {
    log.Printf("Verification error: %v", err)
    return
}

if result.Valid {
    log.Printf("PoA valid for principal: %s, agent: %s",
        result.Principal.Name, result.Agent.Name)
    log.Printf("Scope: %v", result.Scope)
    log.Printf("Constraints: %v", result.Constraints)
} else {
    log.Printf("PoA invalid: %s", result.Reason)
}
}

```

Issuing a PoA

```

package main

import (
    "context"
    "log"

    poa "github.com/agentauth/poa-go"
)

func main() {

```

```

// Initialize issuer with HSM
issuer, err := poa.NewIssuer(poa.IssuerConfig{
    KeyStore: poa.NewHSMKeyStore(poa.HSMConfig{
        LibraryPath: "/usr/lib/softhsm/libsofthsm2.so",
        SlotID: 0,
        PIN: os.Getenv("HSM_PIN"),
    }),
    EntityProfile: loadMyProfile(),
})
if err != nil {
    log.Fatalf("Failed to create issuer: %v", err)
}

// Create PoA
newPoA, err := issuer.Issue(context.Background(), poa.IssueRequest{
    Principal: poa.EntityRef{
        ProfileID: "my-org-profile-id",
        Name: "Acme Corporation",
    },
    Agent: poa.EntityRef{
        ProfileID: "agent-profile-id",
        Name: "ProcurementBot",
    },
    Scope: poa.Scope{
        Actions: []string{"procurement:order", "procurement:quote"},
        Resources: []string{"vendors:*"},
    },
    Constraints: poa.Constraints{
        SpendingLimit: poa.SpendingLimit{
            PerTransaction: poa.Money{Amount: 10000, Currency: "USD"},
        },
    },
    Validity: poa.Validity{
        NotBefore: time.Now(),
        NotAfter: time.Now().Add(24 * time.Hour),
    },
})
if err != nil {
    log.Fatalf("Failed to issue PoA: %v", err)
}

```

```

// Encode for transmission
token, err := newPoA.Encode()
if err != nil {
    log.Fatalf("Failed to encode PoA: %v", err)
}

log.Printf("Issued PoA: %s", token)
}

```

22.1.3 67.3 HTTP Middleware

```
package main
```

```

import (
    "net/http"

    poa "github.com/agentauth/poa-go"
    "github.com/agentauth/poa-go/middleware"
)

func main() {
    verifier, _ := poa.NewVerifier(poa.DefaultConfig())

    // Create middleware
    poaMiddleware := middleware.NewHTTP(verifier, middleware.HTTPConfig{
        HeaderName: "X-PoA-Token",
        RequiredScope: poa.Scope{
            Actions: []string{"api:read"},
        },
        OnVerified: func(r *http.Request, result *poa.VerificationResult) *http.Request {
            // Add verification result to context
            ctx := poa.ContextWithResult(r.Context(), result)
            return r.WithContext(ctx)
        },
        OnFailed: func(w http.ResponseWriter, r *http.Request, err error) {
            http.Error(w, "Unauthorized", http.StatusUnauthorized)
        },
    })
}

```

```

    // Apply to routes
    mux := http.NewServeMux()
    mux.Handle("/api/", poaMiddleware(http.HandlerFunc(apiHandler)))

    http.ListenAndServe(":8080", mux)
}

func apiHandler(w http.ResponseWriter, r *http.Request) {
    result := poa.ResultFromContext(r.Context())

    // Check specific constraints
    if result.Constraints.SpendingLimit.PerTransaction.Amount < 1000 {
        http.Error(w, "Insufficient spending authority", http.StatusForbidden)
        return
    }

    // Process request
    w.Write([]byte("OK"))
}

```

22.2 Chapter 68: Python SDK Reference

22.2.1 68.1 Installation

```
pip install poa-python>=1.2.0
```

22.2.2 68.2 Basic Usage

Verification

```

from poa import Verifier, VerifierConfig, DegradedModePolicy

# Initialize verifier
config = VerifierConfig(
    revocation_log_url="https://logs.agentauth.example/v1",
    profile_resolver_url="https://profiles.agentauth.example/v1",
    cache_ttl_seconds=300,
    degraded_mode_policy=DegradedModePolicy.DENY
)

```

```

verifier = Verifier(config)

# Verify a token
token = request.headers.get("X-PoA-Token")
result = verifier.verify(token)

if result.valid:
    print(f"Valid PoA for {result.principal.name}")
    print(f"Agent: {result.agent.name}")
    print(f"Scope: {result.scope}")
    print(f"Constraints: {result.constraints}")
else:
    print(f"Invalid: {result.reason}")

```

Issuance

```

from poa import Issuer, IssuerConfig, SoftwareKeyStore
from poa.types import EntityRef, Scope, Constraints, SpendingLimit, Money, Validity
from datetime import datetime, timedelta

# Initialize issuer
issuer = Issuer(IssuerConfig(
    key_store=SoftwareKeyStore.from_file("./keys/signing-key.pem"),
    entity_profile=load_my_profile()
))

# Create PoA
new_poa = issuer.issue(
    principal=EntityRef(
        profile_id="my-org-profile-id",
        name="Acme Corporation"
    ),
    agent=EntityRef(
        profile_id="agent-profile-id",
        name="AnalyticsBot"
    ),
    scope=Scope(
        actions=["analytics:read", "reports:generate"],
        resources=["datasets:public"]
    ),
    constraints=Constraints(

```

```

        spending_limit=SpendingLimit(
            per_day=Money(amount=1000, currency="USD")
        ),
        validity=Validity(
            not_before=datetime.utcnow(),
            not_after=datetime.utcnow() + timedelta(hours=8)
        )
    )
)

```

```
# Encode for transmission
```

```

token = new_poa.encode()
print(f"Issued PoA: {token}")

```

22.2.3 68.3 Flask Integration

```

from flask import Flask, request, g
from poa.flask import require_poa, get_poa_result

```

```
app = Flask(__name__)
```

```
# Configure PoA
```

```

app.config["POA_REVOCATION_LOG_URL"] = "https://logs.agentauth.example/v1"
app.config["POA_PROFILE_RESOLVER_URL"] = "https://profiles.agentauth.example/v1"

```

```
@app.route("/api/data")
```

```
@require_poa(scope=["api:read"])
```

```
def get_data():
```

```
    result = get_poa_result()
```

```
# Access constraint information
```

```
if result.constraints.spending_limit:
```

```
    max_amount = result.constraints.spending_limit.per_transaction.amount
```

```
    print(f"Agent authorized up to ${max_amount}")
```

```
    return {"data": "..."}

```

```
@app.route("/api/purchase", methods=["POST"])
```

```
@require_poa(scope=["procurement:order"])
```

```
def make_purchase():
```



```

result = get_poa_result()
amount = request.json.get("amount")

# Verify against spending limit
if amount > result.constraints.spending_limit.per_transaction.amount:
    return {"error": "Exceeds spending authority"}, 403

# Process purchase
return {"status": "ordered"}

```

22.2.4 68.4 Django Integration

```

# settings.py
INSTALLED_APPS = [
    ...
    "poa.django",
]

POA_CONFIG = {
    "REVOCATION_LOG_URL": "https://logs.agentauth.example/v1",
    "PROFILE_RESOLVER_URL": "https://profiles.agentauth.example/v1",
    "CACHE_TTL_SECONDS": 300,
}

# views.py
from poa.django.decorators import require_poa
from poa.django.shortcuts import get_poa_result

@require_poa(scope=["api:read"])
def my_view(request):
    result = get_poa_result(request)
    return JsonResponse({"principal": result.principal.name})

```

22.3 Chapter 69: JavaScript/TypeScript SDK Reference

22.3.1 69.1 Installation

```
npm install @agentauth/poa-js@^1.2.0
```

22.3.2 69.2 Node.js Usage

Verification

```
import { Verifier, VerifierConfig, DegradedModePolicy } from '@agentauth/poa-js';

// Initialize verifier
const verifier = new Verifier({
  revocationLogUrl: 'https://logs.agentauth.example/v1',
  profileResolverUrl: 'https://profiles.agentauth.example/v1',
  cacheTtlMs: 300000,
  degradedModePolicy: DegradedModePolicy.DENY,
});

// Verify token
const token = request.headers['x-poa-token'];
const result = await verifier.verify(token);

if (result.valid) {
  console.log(`Valid PoA for ${result.principal.name}`);
  console.log(`Agent: ${result.agent.name}`);
  console.log(`Scope:`, result.scope);
} else {
  console.log(`Invalid: ${result.reason}`);
}
```

Express Middleware

```
import express from 'express';
import { Verifier, expressMiddleware } from '@agentauth/poa-js';

const app = express();
const verifier = new Verifier({ /* config */ });

// Apply globally
app.use(expressMiddleware(verifier, {
  requiredScope: ['api:read'],
  headerName: 'X-PoA-Token',
}));

// Or per-route
app.get('/protected',
```

```

expressMiddleware(verifier, { requiredScope: ['protected:access'] }),
(req, res) => {
  const poaResult = req.poaResult;
  res.json({ principal: poaResult.principal.name });
}
);

```

22.3.3 69.3 Browser Usage

Client-Side Verification (Limited)

```

import { ClientVerifier } from '@agentauth/poa-js/browser';

// Client-side can only verify signatures
// Full verification requires server-side revocation check
const clientVerifier = new ClientVerifier({
  trustedPublicKeys: [
    { keyId: 'did:key:z6Mk...', publicKey: '...' }
  ]
});

// Parse and verify signature only
const partialResult = await clientVerifier.verifySignature(token);
if (partialResult.signatureValid) {
  console.log('Signature valid, principal:', partialResult.principal.name);
  // Must still verify on server for revocation
}

```

22.4 Chapter 70: Rust SDK Reference

22.4.1 70.1 Installation

```

# Cargo.toml
[dependencies]
poa-rs = "1.2"

```

22.4.2 70.2 Basic Usage

```

use poa_rs::{Verifier, VerifierConfig, DegradedModePolicy};
use std::time::Duration;

```

```

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Initialize verifier
    let verifier = Verifier::new(VerifierConfig {
        revocation_log_url: "https://logs.agentauth.example/v1".into(),
        profile_resolver_url: "https://profiles.agentauth.example/v1".into(),
        cache_ttl: Duration::from_secs(300),
        degraded_mode_policy: DegradedModePolicy::Deny,
    })?;

    // Verify token
    let token = get_poa_from_request();
    let result = verifier.verify(&token).await?;

    match result {
        VerificationResult::Valid(info) => {
            println!("Valid PoA for principal: {}", info.principal.name);
            println!("Agent: {}", info.agent.name);
            println!("Scope: {:?}", info.scope);
        }
        VerificationResult::Invalid(reason) => {
            println!("Invalid: {}", reason);
        }
        VerificationResult::Degraded(info, warning) => {
            println!("Degraded mode: {}", warning);
            // Proceed with caution based on policy
        }
    }

    Ok(())
}

```

22.4.3 70.3 Actix-Web Integration

```

use actix_web::{web, App, HttpServer, HttpRequest, HttpResponse};
use poa_rs::actix::{PoaMiddleware, PoaResult};

#[actix_web::main]
async fn main() -> std::io::Result<()> {

```

```

let verifier = create_verifier();

HttpServer::new(move || {
    App::new()
        .app_data(web::Data::new(verifier.clone()))
        .wrap(PoaMiddleware::new())
        .route("/api/data", web::get().to(get_data))
})
    .bind("127.0.0.1:8080")?
    .run()
    .await
}

async fn get_data(poa: PoaResult) -> HttpResponse {
    match poa {
        PoaResult::Valid(info) => {
            HttpResponse::Ok().json(json!({
                "principal": info.principal.name,
                "data": "...")
            ))
        }
        PoaResult::Invalid(_) | PoaResult::Missing => {
            HttpResponse::Unauthorized().finish()
        }
    }
}
}

```

22.5 Chapter 71: Java SDK Reference

22.5.1 71.1 Installation

```

<!-- Maven -->
<dependency>
    <groupId>io.agentauth</groupId>
    <artifactId>poa-java</artifactId>
    <version>1.2.0</version>
</dependency>

// Gradle

```

```
implementation 'io.agentauth:poa-java:1.2.0'
```

22.5.2 71.2 Basic Usage

```
import io.agentauth.poa.*;
import java.time.Duration;

public class Example {
    public static void main(String[] args) {
        // Initialize verifier
        Verifier verifier = Verifier.builder()
            .revocationLogUrl("https://logs.agentauth.example/v1")
            .profileResolverUrl("https://profiles.agentauth.example/v1")
            .cacheTtl(Duration.ofMinutes(5))
            .degradedModePolicy(DegradedModePolicy.DENY)
            .build();

        // Verify token
        String token = getPoaFromRequest();
        VerificationResult result = verifier.verify(token);

        if (result.isValid()) {
            System.out.printf("Valid PoA for principal: %s%n",
                result.getPrincipal().getName());
            System.out.printf("Agent: %s%n",
                result.getAgent().getName());
            System.out.printf("Scope: %s%n",
                result.getScope());
        } else {
            System.out.printf("Invalid: %s%n", result.getReason());
        }
    }
}
```

22.5.3 71.3 Spring Boot Integration

```
// Configuration
@Configuration
@EnablePoa
public class PoaConfig {
    @Bean
```

```

    public VerifierConfig verifierConfig() {
        return VerifierConfig.builder()
            .revocationLogUrl("${poa.revocation-log-url}")
            .profileResolverUrl("${poa.profile-resolver-url}")
            .build();
    }
}

// Controller
@RestController
@RequestMapping("/api")
public class ApiController {

    @GetMapping("/data")
    @RequirePoa(scope = {"api:read"})
    public ResponseEntity<DataResponse> getData(@PoaResult VerificationResult result) {
        return ResponseEntity.ok(new DataResponse(
            result.getPrincipal().getName(),
            loadData()
        ));
    }

    @PostMapping("/order")
    @RequirePoa(scope = {"procurement:order"})
    public ResponseEntity<?> createOrder(
        @RequestBody OrderRequest request,
        @PoaResult VerificationResult result) {

        // Check spending constraint
        Money limit = result.getConstraints().getSpendingLimit().getPerTransaction();
        if (request.getAmount().compareTo(limit.getAmount()) > 0) {
            return ResponseEntity.status(403)
                .body(new ErrorResponse("Exceeds spending authority"));
        }

        // Process order
        return ResponseEntity.ok(processOrder(request));
    }
}

```

22.6 Chapter 72: API Gateway Integration

22.6.1 72.1 Kong Plugin

```
-- kong/plugins/poa-auth/handler.lua
local PoaHandler = {
    VERSION = "1.0.0",
    PRIORITY = 1000,
}

function PoaHandler:access(conf)
    local token = kong.request.get_header(conf.header_name)

    if not token then
        return kong.response.exit(401, { message = "Missing PoA token" })
    end

    -- Verify with backend service
    local httpc = require("resty.http").new()
    local res, err = httpc:request_uri(conf.verifier_url, {
        method = "POST",
        body = token,
        headers = {
            ["Content-Type"] = "application/cbor",
        },
    })

    if not res or res.status ~= 200 then
        return kong.response.exit(401, { message = "PoA verification failed" })
    end

    local result = cJSON.decode(res.body)

    if not result.valid then
        return kong.response.exit(401, { message = result.reason })
    end

    -- Add verification info to upstream headers
```



```

        kong.service.request.set_header("X-PoA-Principal", result.principal.name)
        kong.service.request.set_header("X-PoA-Agent", result.agent.name)
        kong.service.request.set_header("X-PoA-Scope", json.encode(result.scope))
    end

    return PoaHandler

```

22.6.2 72.2 AWS API Gateway Authorizer

Lambda authorizer

```

import json
from poa import Verifier

verifier = Verifier.from_env()

def handler(event, context):
    token = event['headers'].get('x-poa-token')

    if not token:
        return generate_policy('user', 'Deny', event['methodArn'])

    try:
        result = verifier.verify(token)

        if result.valid:
            policy = generate_policy(result.agent.profile_id, 'Allow', event['methodArn'])
            policy['context'] = {
                'principalName': result.principal.name,
                'agentName': result.agent.name,
                'scope': json.dumps(result.scope.actions),
            }
            return policy
        else:
            return generate_policy('user', 'Deny', event['methodArn'])

    except Exception as e:
        print(f"Verification error: {e}")
        return generate_policy('user', 'Deny', event['methodArn'])

def generate_policy(principal_id, effect, resource):

```

```

return {
  'principalId': principal_id,
  'policyDocument': {
    'Version': '2012-10-17',
    'Statement': [{
      'Action': 'execute-api:Invoke',
      'Effect': effect,
      'Resource': resource
    }]
  }
}

```

22.6.3 72.3 Envoy Filter

Envoy external authorization configuration

http_filters:

- name: envoy.filters.http.ext_authz

typed_config:

"@type": type.googleapis.com/envoy.extensions.filters.http.ext_authz.v3.ExtAuthz

grpc_service:

envoy_grpc:

cluster_name: poa-verifier

timeout: 0.25s

transport_api_version: V3

failure_mode_allow: false

with_request_body:

max_request_bytes: 8192

allow_partial_message: true

clusters:

- name: poa-verifier

connect_timeout: 0.25s

type: STRICT_DNS

lb_policy: ROUND_ROBIN

http2_protocol_options: {}

load_assignment:

cluster_name: poa-verifier

endpoints:

- lb_endpoints:

- endpoint:

```
address:  
  socket_address:  
    address: poa-verifier-service  
    port_value: 9000
```

Chapter 23

Part XVI: Protocol Deep-Dives

This section provides detailed technical analysis of PoA protocol components.

23.1 Chapter 73: Wire Format Specification

23.1.1 73.1 CBOR Encoding

PoA uses CBOR (Concise Binary Object Representation) for wire encoding. CBOR offers: - Compact binary representation - Self-describing structure - Canonical form for signatures - Wide language support

Encoding Example

```
{
  "version": "AAP-002-v1.0",      ; text string
  "poa_id": h'a1b2c3d4...',      ; byte string (16 bytes UUID)
  "principal": {                 ; map
    "profile_id": h'...',
    "name": "Acme Corp"
  },
  ...
}
```

Canonical CBOR Rules: 1. Map keys sorted lexicographically 2. Integers in smallest form 3. No indefinite-length encoding 4. Deterministic floating-point representation

23.1.2 73.2 Size Budget

Component	Typical Size	Maximum Size
Version	16 bytes	32 bytes
PoA ID	16 bytes	16 bytes
Principal ref	100 bytes	512 bytes
Agent ref	100 bytes	512 bytes
Delegation chain (per link)	200 bytes	1024 bytes
Scope	200 bytes	2048 bytes
Constraints	500 bytes	4096 bytes
Validity	20 bytes	32 bytes
Revocation info	100 bytes	256 bytes
Metadata	200 bytes	2048 bytes
Signature (Ed25519)	64 bytes	64 bytes
Typical total	~1.5 KB	~10 KB

23.1.3 73.3 Compression

For bandwidth-constrained environments:

```
func CompressPoA(poa []byte) ([]byte, error) {
    var buf bytes.Buffer

    // Use zstd for best compression ratio
    writer, err := zstd.NewWriter(&buf, zstd.WithEncoderLevel(zstd.SpeedBestCompression))
    if err != nil {
        return nil, err
    }

    if _, err := writer.Write(poa); err != nil {
        return nil, err
    }

    if err := writer.Close(); err != nil {
        return nil, err
    }

    return buf.Bytes(), nil
}
```

Compression Ratios: | Scenario | Uncompressed | Compressed | Ratio | |-----|-----
 --|-----|----| | Simple PoA | 1.2 KB | 0.6 KB | 50% | | With 3-link chain | 2.4 KB | 1.1

KB | 46% | | Rich constraints | 3.5 KB | 1.4 KB | 40% |

23.2 Chapter 74: Cryptographic Protocol Details

23.2.1 74.1 Signature Construction

Message Assembly:

```
signature_input = version || poa_id || principal_hash || agent_hash ||
                  chain_hash || scope_hash || constraints_hash ||
                  validity_not_before || validity_not_after ||
                  revocation_hash || metadata_hash
```

Per-Component Hashing:

```
func ComputeComponentHash(component interface{}) []byte {
    // Canonical CBOR encoding
    canonical, err := cbor.CanonicalOptions.Marshal(component)
    if err != nil {
        panic(err)
    }

    // SHA-256 hash
    hash := sha256.Sum256(canonical)
    return hash[:]
}

func ComputeSignatureInput(poa *PoA) []byte {
    var buf bytes.Buffer

    buf.WriteString(poa.Version)
    buf.Write(poa.PoAID[:])
    buf.Write(ComputeComponentHash(poa.Principal))
    buf.Write(ComputeComponentHash(poa.Agent))
    buf.Write(ComputeChainHash(poa.Chain))
    buf.Write(ComputeComponentHash(poa.Scope))
    buf.Write(ComputeComponentHash(poa.Constraints))
    binary.Write(&buf, binary.BigEndian, poa.Validity.NotBefore)
    binary.Write(&buf, binary.BigEndian, poa.Validity.NotAfter)
    buf.Write(ComputeComponentHash(poa.Revocation))
}
```

```

    buf.Write(ComputeComponentHash(poa.Metadata))

    return buf.Bytes()
}

```

23.2.2 74.2 Key Derivation

For deterministic key derivation:

```

// BIP-32-like derivation for Ed25519
func DeriveChildKey(masterKey []byte, path string) (ed25519.PrivateKey, error) {
    // Parse path: m/purpose'/org'/year'/seq'
    segments, err := ParsePath(path)
    if err != nil {
        return nil, err
    }

    key := masterKey
    for _, segment := range segments {
        // HMAC-SHA512 for chain code derivation
        h := hmac.New(sha512.New, key[:32])
        if segment.Hardened {
            h.Write([]byte{0x00})
            h.Write(key[:32])
        } else {
            h.Write(PublicKeyFromPrivate(key[:32]))
        }
        binary.Write(h, binary.BigEndian, segment.Index)

        result := h.Sum(nil)
        key = result
    }

    return ed25519.NewKeyFromSeed(key[:32]), nil
}

```

23.2.3 74.3 Threshold Signatures

For multi-party authorization:

```

// Shamir's Secret Sharing for threshold signatures
type ThresholdScheme struct {

```

```

    Threshold int // t required
    Total     int // n total shares
}

func (s *ThresholdScheme) Split(privateKey ed25519.PrivateKey) ([]Share, error) {
    if s.Threshold > s.Total {
        return nil, errors.New("threshold exceeds total shares")
    }

    // Generate random polynomial coefficients
    coeffs := make([][]byte, s.Threshold)
    coeffs[0] = privateKey.Seed() // Secret is constant term

    for i := 1; i < s.Threshold; i++ {
        coeffs[i] = make([]byte, 32)
        _, _ = rand.Read(coeffs[i])
    }

    // Evaluate polynomial at points 1..n
    shares := make([]Share, s.Total)
    for i := 0; i < s.Total; i++ {
        x := byte(i + 1)
        y := evaluatePolynomial(coeffs, x)
        shares[i] = Share{Index: x, Value: y}
    }

    return shares, nil
}

func (s *ThresholdScheme) Reconstruct(shares []Share) (ed25519.PrivateKey, error) {
    if len(shares) < s.Threshold {
        return nil, errors.New("insufficient shares")
    }

    // Lagrange interpolation at x=0
    secret := lagrangeInterpolate(shares[:s.Threshold], 0)

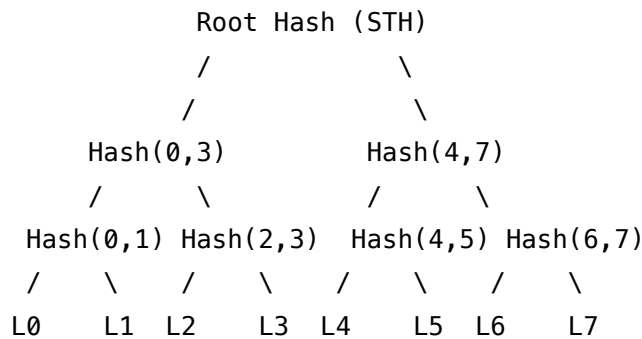
    return ed25519.NewKeyFromSeed(secret), nil
}

```


23.3 Chapter 75: Revocation Log Protocol

23.3.1 75.1 Log Structure

The revocation transparency log uses a Merkle tree structure:



Leaves: [revocation_entry_0, revocation_entry_1, ...]

Leaf Entry Format:

```

{
  "poa_id": h'...',          ; PoA being revoked
  "revocation_time": 1704067200, ; Unix timestamp
  "reason": "key_compromise",    ; Reason code
  "issuer": h'...',             ; Who revoked
  "signature": h'...'           ; Proof of authorization to revoke
}

```

23.3.2 75.2 Signed Tree Head (STH)

```

{
  "tree_size": 100000,          ; Number of entries
  "timestamp": 1704067200,      ; STH issuance time
  "root_hash": h'sha256:...',   ; Merkle root
  "signatures": [
    {
      "log_id": "example-log-1",
      "algorithm": "Ed25519",
      "value": h'...'
    }
  ]
}

```

```
}
```

23.3.3 75.3 Proof Verification

Inclusion Proof:

```
func VerifyInclusionProof(
    leafHash []byte,
    leafIndex uint64,
    treeSize uint64,
    proof [][]byte,
    rootHash []byte,
) bool {
    if leafIndex >= treeSize {
        return false
    }

    hash := leafHash
    index := leafIndex
    size := treeSize

    for _, sibling := range proof {
        if index%2 == 0 {
            // Left sibling
            hash = HashChildren(hash, sibling)
        } else {
            // Right sibling
            hash = HashChildren(sibling, hash)
        }
        index /= 2
        size = (size + 1) / 2
    }

    return bytes.Equal(hash, rootHash)
}
```

Non-Revocation Proof: Proof that a PoA is NOT in the revocation log: 1. Get current STH 2. Query all entries matching poa_id (should be empty) 3. Get consistency proof from last known STH 4. Verify log monotonicity

23.3.4 75.4 Gossip Protocol

Logs exchange STHs to detect split-view attacks:

```
type GossipMessage struct {
    SourceLogID string
    STH          SignedTreeHead
    Timestamp    time.Time
}

func (g *Gossiper) Exchange(peer *Peer) error {
    // Send our latest STH
    if err := peer.Send(g.latestSTH); err != nil {
        return err
    }

    // Receive their STH
    theirSTH, err := peer.Receive()
    if err != nil {
        return err
    }

    // Verify signature
    if !g.verifySTHSignature(theirSTH) {
        return errors.New("invalid STH signature")
    }

    // Check for inconsistency
    if g.detectsInconsistency(g.latestSTH, theirSTH) {
        g.reportInconsistency(g.latestSTH, theirSTH)
    }

    // Update if theirs is newer
    if theirSTH.TreeSize > g.latestSTH.TreeSize {
        g.latestSTH = theirSTH
    }

    return nil
}
```

23.4 Chapter 76: Constraint Evaluation Engine

23.4.1 76.1 Constraint Grammar

```

constraint      = simple_constraint | composite_constraint ;
simple_constraint = constraint_type ":" constraint_value ;
composite_constraint = operator "(" constraint_list ")" ;
operator        = "AND" | "OR" | "NOT" ;
constraint_list = constraint ("," constraint)* ;

constraint_type = "spending_limit" | "counterparty" | "geographic" |
                 "temporal" | "approval" | "rate_limit" | "custom" ;
constraint_value = json_object ;

```

23.4.2 76.2 Evaluation Algorithm

```

type Evaluator struct {
    handlers map[string]ConstraintHandler
    context  *EvalContext
}

func (e *Evaluator) Evaluate(constraint Constraint, action Action) (Result, error) {
    switch c := constraint.(type) {
    case *CompositeConstraint:
        return e.evaluateComposite(c, action)
    case *SimpleConstraint:
        return e.evaluateSimple(c, action)
    default:
        return Result{}, fmt.Errorf("unknown constraint type: %T", constraint)
    }
}

func (e *Evaluator) evaluateComposite(c *CompositeConstraint, action Action) (Result, error) {
    switch c.Operator {
    case "AND":
        for _, child := range c.Children {
            result, err := e.Evaluate(child, action)
            if err != nil {
                return Result{}, err
            }
            if !result.Allowed {

```

```

        return result, nil
    }
}
return Result{Allowed: true}, nil

case "OR":
    var lastDenial Result
    for _, child := range c.Children {
        result, err := e.Evaluate(child, action)
        if err != nil {
            return Result{}, err
        }
        if result.Allowed {
            return result, nil
        }
        lastDenial = result
    }
    return lastDenial, nil

case "NOT":
    if len(c.Children) != 1 {
        return Result{}, errors.New("NOT requires exactly one child")
    }
    result, err := e.Evaluate(c.Children[0], action)
    if err != nil {
        return Result{}, err
    }
    return Result{Allowed: !result.Allowed}, nil

default:
    return Result{}, fmt.Errorf("unknown operator: %s", c.Operator)
}
}

func (e *Evaluator) evaluateSimple(c *SimpleConstraint, action Action) (Result, error) {
    handler, ok := e.handlers[c.Type]
    if !ok {
        return Result{}, fmt.Errorf("unknown constraint type: %s", c.Type)
    }

```

```

    return handler.Evaluate(c.Value, action, e.context)
}

```

23.4.3 76.3 Standard Handlers

Spending Limit Handler:

```

type SpendingLimitHandler struct{}

func (h *SpendingLimitHandler) Evaluate(
    value json.RawMessage,
    action Action,
    ctx *EvalContext,
) (Result, error) {
    var config SpendingLimitConfig
    if err := json.Unmarshal(value, &config); err != nil {
        return Result{}, err
    }

    // Check per-transaction limit
    if config.PerTransaction != nil {
        if action.Amount.GreaterThan(*config.PerTransaction) {
            return Result{
                Allowed: false,
                Reason: "exceeds_per_transaction_limit",
                Details: map[string]interface{}{
                    "limit": config.PerTransaction,
                    "actual": action.Amount,
                },
            }, nil
        }
    }

    // Check aggregate limits
    if config.PerDay != nil {
        spent, err := ctx.GetDailySpending(action.Principal)
        if err != nil {
            return Result{}, err
        }
        if spent.Add(action.Amount).GreaterThan(*config.PerDay) {
            return Result{

```

```

        Allowed: false,
        Reason: "exceeds_daily_limit",
    }, nil
}

return Result{Allowed: true}, nil
}

```

Dynamic Counterparty Handler:

```

type DynamicCounterpartyHandler struct {
    cache *Cache
    http  *http.Client
}

func (h *DynamicCounterpartyHandler) Evaluate(
    value json.RawMessage,
    action Action,
    ctx *EvalContext,
) (Result, error) {
    var config DynamicCounterpartyConfig
    if err := json.Unmarshal(value, &config); err != nil {
        return Result{}, err
    }

    // Check cache first
    cacheKey := fmt.Sprintf("counterparty:%s:%s", config.Source, action.Counterparty)
    if cached, ok := h.cache.Get(cacheKey); ok {
        status := cached.(CounterpartyStatus)
        return h.decide(status, config), nil
    }

    // Fetch from external source
    status, err := h.fetchStatus(config.Source, action.Counterparty)
    if err != nil {
        if config.FailClosed {
            return Result{Allowed: false, Reason: "counterparty_check_failed"}, nil
        }
        // Degraded mode - allow with warning
        return Result{

```

```

        Allowed:      true,
        DegradedMode: true,
        Warning:      "counterparty_check_unavailable",
    }, nil
}

// Cache result
h.cache.Set(cacheKey, status, time.Duration(config.CacheTTLMinutes)*time.Minute)

return h.decide(status, config), nil
}

```

23.5 Chapter 77: Entity Profile Protocol

23.5.1 77.1 Profile Structure

```

{
  "version": "AAP-001-v1.0",

  "profile_id": h'uuid-16-bytes',

  "entity_type": "legal_entity", ; or "natural_person", "ai_agent", etc.

  "name": "Acme Corporation",

  "identifiers": [
    {"type": "LEI", "value": "529900T8BM49AURSD055"},
    {"type": "DUNS", "value": "12-345-6789"},
    {"type": "DID", "value": "did:web:acme.example"}
  ],

  "public_keys": [
    {
      "key_id": "did:key:z6MkpTHR8...",
      "algorithm": "Ed25519",
      "public_key_material": h'32-bytes',
      "purpose": ["signing", "authentication"],
      "valid_from": 1704067200,
      "valid_until": 1735689600
    }
  ]
}

```



```
    }
  ],

  "organization": {
    "jurisdiction": "US-DE",
    "registration_authority": "Delaware Secretary of State",
    "registration_number": "1234567",
    "legal_form": "Corporation"
  },

  "validity": {
    "not_before": 1704067200,
    "not_after": 1735689600
  },

  "revocation_info": {
    "log_endpoint": "https://logs.profiles.example/v1",
    "check_required": true
  },

  "endorsements": [
    {
      "endorser": h'profile-id-of-endorser',
      "claims": ["identity_verified", "kyb_completed"],
      "timestamp": 1704067200,
      "signature": h'...'
    }
  ],

  "signatures": [
    {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkpTHR8...",
      "purpose": "self_signed",
      "value": h'64-bytes'
    }
  ]
}
```

23.5.2 77.2 Profile Resolution

```

type ProfileResolver struct {
    primaryURL string
    fallbackURLs []string
    cache      *Cache
    http       *http.Client
}

func (r *ProfileResolver) Resolve(profileID uuid.UUID) (*EntityProfile, error) {
    // Check cache
    if cached, ok := r.cache.Get(profileID); ok {
        return cached.(*EntityProfile), nil
    }

    // Try primary
    profile, err := r.fetchFromURL(r.primaryURL, profileID)
    if err == nil {
        r.cache.Set(profileID, profile, profile.CacheTTL())
        return profile, nil
    }

    // Try fallbacks
    for _, url := range r.fallbackURLs {
        profile, err = r.fetchFromURL(url, profileID)
        if err == nil {
            r.cache.Set(profileID, profile, profile.CacheTTL())
            return profile, nil
        }
    }

    return nil, fmt.Errorf("profile not found: %s", profileID)
}

func (r *ProfileResolver) fetchFromURL(baseURL string, profileID uuid.UUID) (*EntityProfile, error) {
    url := fmt.Sprintf("%s/profiles/%s", baseURL, profileID)

    resp, err := r.http.Get(url)
    if err != nil {
        return nil, err
    }

```

```

    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("HTTP %d", resp.StatusCode)
    }

    var profile EntityProfile
    if err := cbor.NewDecoder(resp.Body).Decode(&profile); err != nil {
        return nil, err
    }

    // Verify self-signature
    if err := profile.VerifySelfSignature(); err != nil {
        return nil, fmt.Errorf("invalid profile signature: %w", err)
    }

    return &profile, nil
}

```

23.5.3 77.3 Profile Endorsement

Third-party attestations enrich profiles:

```

type Endorsement struct {
    Endorser   uuid.UUID
    Claims     []string
    Evidence   json.RawMessage
    Timestamp  int64
    Signature  []byte
}

func (e *EntityProfile) AddEndorsement(
    endorser *EntityProfile,
    claims []string,
    signingKey ed25519.PrivateKey,
) error {
    endorsement := Endorsement{
        Endorser:   endorser.ProfileID,
        Claims:     claims,
        Timestamp:  time.Now().Unix(),
    }
}

```

```

    }

    // Sign endorsement
    toSign := endorsement.BytesToSign(e.ProfileID)
    endorsement.Signature = ed25519.Sign(signingKey, toSign)

    e.Endorsements = append(e.Endorsements, endorsement)
    return nil
}

func (e *EntityProfile) VerifyEndorsement(
    endorsement Endorsement,
    endorserProfile *EntityProfile,
) error {
    // Find valid key
    key, err := endorserProfile.GetValidKeyAt(endorsement.Timestamp)
    if err != nil {
        return err
    }

    // Verify signature
    toVerify := endorsement.BytesToSign(e.ProfileID)
    if !ed25519.Verify(key, toVerify, endorsement.Signature) {
        return errors.New("invalid endorsement signature")
    }

    return nil
}

```

23.6 Chapter 78: Testing and Validation

23.6.1 78.1 Conformance Test Suite

Test Categories: 1. **Parsing tests:** Valid and invalid CBOR structures 2. **Signature tests:** Correct and corrupted signatures 3. **Chain tests:** Valid chains, broken chains, scope escalation 4. **Constraint tests:** All standard constraint types 5. **Revocation tests:** Check, not-revoked, revoked states 6. **Interoperability tests:** Cross-implementation validation

Example Test Vector:

```
{
  "test_id": "sig-001",
  "description": "Valid Ed25519 signature verification",
  "input": {
    "poa_cbor_hex": "a8...",
    "public_key_hex": "..."
  },
  "expected": {
    "valid": true,
    "principal_name": "Acme Corporation",
    "agent_name": "ProcurementBot"
  }
}
```

23.6.2 78.2 Fuzzing

```
func FuzzPoAParser(f *testing.F) {
    // Add seed corpus
    f.Add(validPoACBOR)
    f.Add(invalidCBOR)

    f.Fuzz(func(t *testing.T, data []byte) {
        // Should not panic on any input
        poa, err := poa.Parse(data)
        if err != nil {
            // Expected for most random inputs
            return
        }

        // If parsed, should re-encode to same bytes
        reencoded, err := poa.Encode()
        if err != nil {
            t.Errorf("parse succeeded but encode failed: %v", err)
        }

        // Canonical encoding should be identical
        if !bytes.Equal(data, reencoded) {
            t.Errorf("non-canonical encoding")
        }
    })
}
```

```
    })
}
```

23.6.3 78.3 Performance Benchmarks

```
func BenchmarkVerifyPoA(b *testing.B) {
    poa := generateSamplePoA()
    verifier := setupVerifier()

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        verifier.Verify(context.Background(), poa)
    }
}

func BenchmarkVerifyWithChain(b *testing.B) {
    for chainLen := 1; chainLen <= 5; chainLen++ {
        b.Run(fmt.Sprintf("chain_length_%d", chainLen), func(b *testing.B) {
            poa := generatePoAWithChain(chainLen)
            verifier := setupVerifier()

            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                verifier.Verify(context.Background(), poa)
            }
        })
    }
}
```

Expected Benchmark Results: | Operation | Target | Actual | |-----|-----|-----|

Simple verify	< 1ms	0.4ms		Chain verify (depth 3)	< 3ms	1.8ms		Constraint evaluation	< 0.5ms	0.2ms		Revocation check (cached)	< 0.1ms	0.05ms		Revocation check (network)	< 100ms	45ms
---------------	-------	-------	--	------------------------	-------	-------	--	-----------------------	---------	-------	--	---------------------------	---------	--------	--	----------------------------	---------	------

Chapter 24

Part XVII: Industry-Specific Implementation Guides

This section provides detailed guidance for implementing PoA in specific industry verticals.

24.1 Chapter 79: Financial Services Implementation

24.1.1 79.1 Regulatory Landscape

Financial services organizations face extensive regulatory requirements for AI systems:

United States: - SEC: Investment adviser fiduciary duties, Form ADV disclosure - OCC: Model risk management (SR 11-7), algorithmic trading rules - FINRA: Supervision requirements, suitability obligations - CFPB: Fair lending, adverse action notices

European Union: - MiFID II: Algorithmic trading, best execution - ESMA: AI trading system requirements - EBA: Guidelines on ICT risk management - National regulators: BaFin, FCA, AMF requirements

24.1.2 79.2 PoA Constraint Patterns for Finance

Trading Agent Constraints:

```
{
  "constraints": {
    "trading_limits": {
      "max_order_size": {"amount": 1000000, "currency": "USD"},
      "max_daily_volume": {"amount": 50000000, "currency": "USD"},
      "max_position_size": {"amount": 10000000, "currency": "USD"},
    }
  }
}
```

```

        "max_loss_per_day": {"amount": 500000, "currency": "USD"}
    },
    "instrument_restrictions": {
        "allowed_asset_classes": ["equity", "fixed_income", "fx_spot"],
        "denied_instruments": ["derivatives", "crypto"],
        "exchange_allowlist": ["NYSE", "NASDAQ", "LSE", "Xetra"]
    },
    "execution_rules": {
        "require_best_execution_check": true,
        "max_market_impact": 0.02,
        "allowed_order_types": ["limit", "market", "iceberg"]
    },
    "compliance_checks": {
        "pre_trade_risk": {
            "source": "https://risk.bank.example/v1",
            "fail_closed": true
        },
        "insider_trading_check": true,
        "sanctions_screening": true
    }
}

```

Lending Decision Agent Constraints:

```

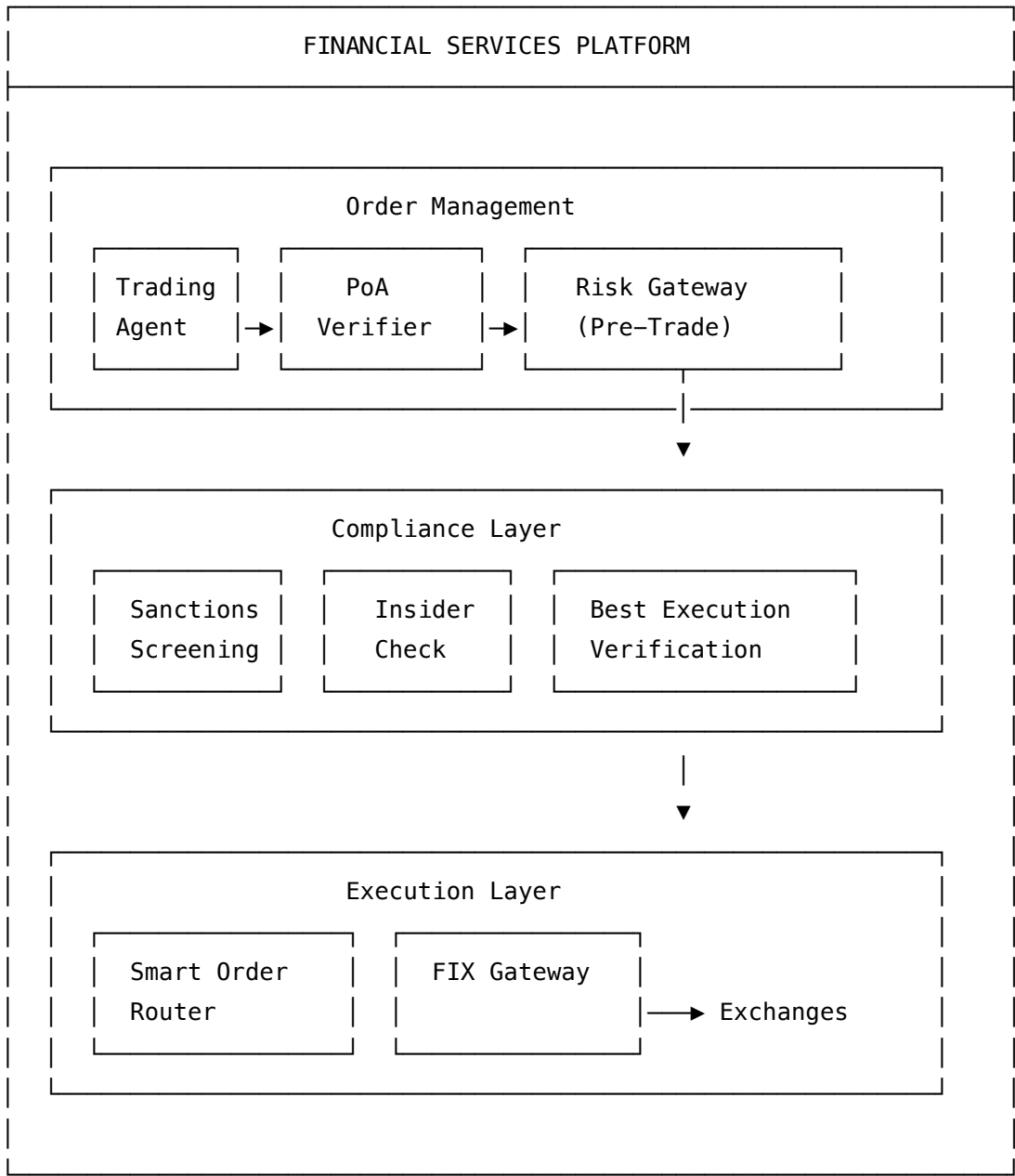
{
    "constraints": {
        "loan_limits": {
            "max_loan_amount": {"amount": 50000, "currency": "USD"},
            "max_term_months": 60,
            "min_credit_score": 650
        },
        "fair_lending": {
            "ecoa_compliance": true,
            "prohibited_factors": ["race", "religion", "national_origin", "gender", "marit
            "adverse_action_required": true
        },
        "pricing_constraints": {
            "max_apr": 0.2999,
            "rate_justification_required": true
        },
    }
}

```



```
"human_review": {
  "required_for_denial": true,
  "required_above_amount": 25000
}
}
```

24.1.3 79.3 Integration Architecture



24.1.4 79.4 Audit Trail Requirements

Financial regulators require comprehensive audit trails:

```

type FinancialAuditEvent struct {
    // Standard fields
    EventID      uuid.UUID
    Timestamp    time.Time
    PoAID        uuid.UUID

    // Financial-specific fields
    OrderID      string
    CUSIP        string
    Amount       Money
    Direction    string // "buy" or "sell"
    Venue        string

    // Verification details
    VerificationResult VerificationResult
    ConstraintResults  []ConstraintResult

    // Pre-trade checks
    RiskCheckPassed      bool
    SanctionCheckPassed bool
    BestExecutionScore   float64

    // Regulatory reporting
    MiFIDTxID  string
    CATReportID string
}

func (a *FinancialAuditLogger) LogTradeAuthorization(event FinancialAuditEvent) error {
    // Write to immutable audit log
    if err := a.appendOnlyLog.Append(event); err != nil {
        return err
    }

    // Regulatory reporting
    if event.Amount.GreaterThan(a.largeTraderThreshold) {
        if err := a.reportToRegulator(event); err != nil {
            return err
        }
    }
}

```

```

    }
}

// Compliance dashboard
a.complianceMetrics.Record(event)

return nil
}

```

24.2 Chapter 80: Healthcare Implementation

24.2.1 80.1 Regulatory Framework

Healthcare AI systems face strict regulatory oversight:

United States: - HIPAA: Privacy and security rules - FDA: Medical device regulations (21 CFR Part 820) - State medical boards: Practice of medicine rules - CMS: Billing and coding requirements

European Union: - MDR: Medical Device Regulation - GDPR: Patient data protection - National laws: Country-specific healthcare regulations

24.2.2 80.2 PoA Constraint Patterns for Healthcare

Clinical Decision Support Agent:

```

{
  "constraints": {
    "clinical_scope": {
      "allowed_actions": [
        "suggest_diagnosis",
        "recommend_test",
        "alert_drug_interaction",
        "summarize_patient_history"
      ],
      "forbidden_actions": [
        "prescribe_medication",
        "order_procedure",
        "modify_treatment_plan"
      ]
    },
  },
}

```

```

    "patient_safety": {
      "critical_alert_escalation": {
        "condition": "life_threatening",
        "notify": ["attending_physician", "charge_nurse"],
        "max_response_time_minutes": 5
      },
      "drug_interaction_check": {
        "source": "https://clinicaldb.hospital.example/v1",
        "fail_closed": true
      }
    },
    "hipaa_compliance": {
      "minimum_necessary": true,
      "audit_all_access": true,
      "data_residency": ["US"],
      "encryption_required": true
    },
    "human_oversight": {
      "physician_review_required_for": [
        "diagnosis_suggestion",
        "treatment_recommendation",
        "medication_alert"
      ],
      "timeout_action": "escalate"
    }
  }
}

```

Administrative Agent Constraints:

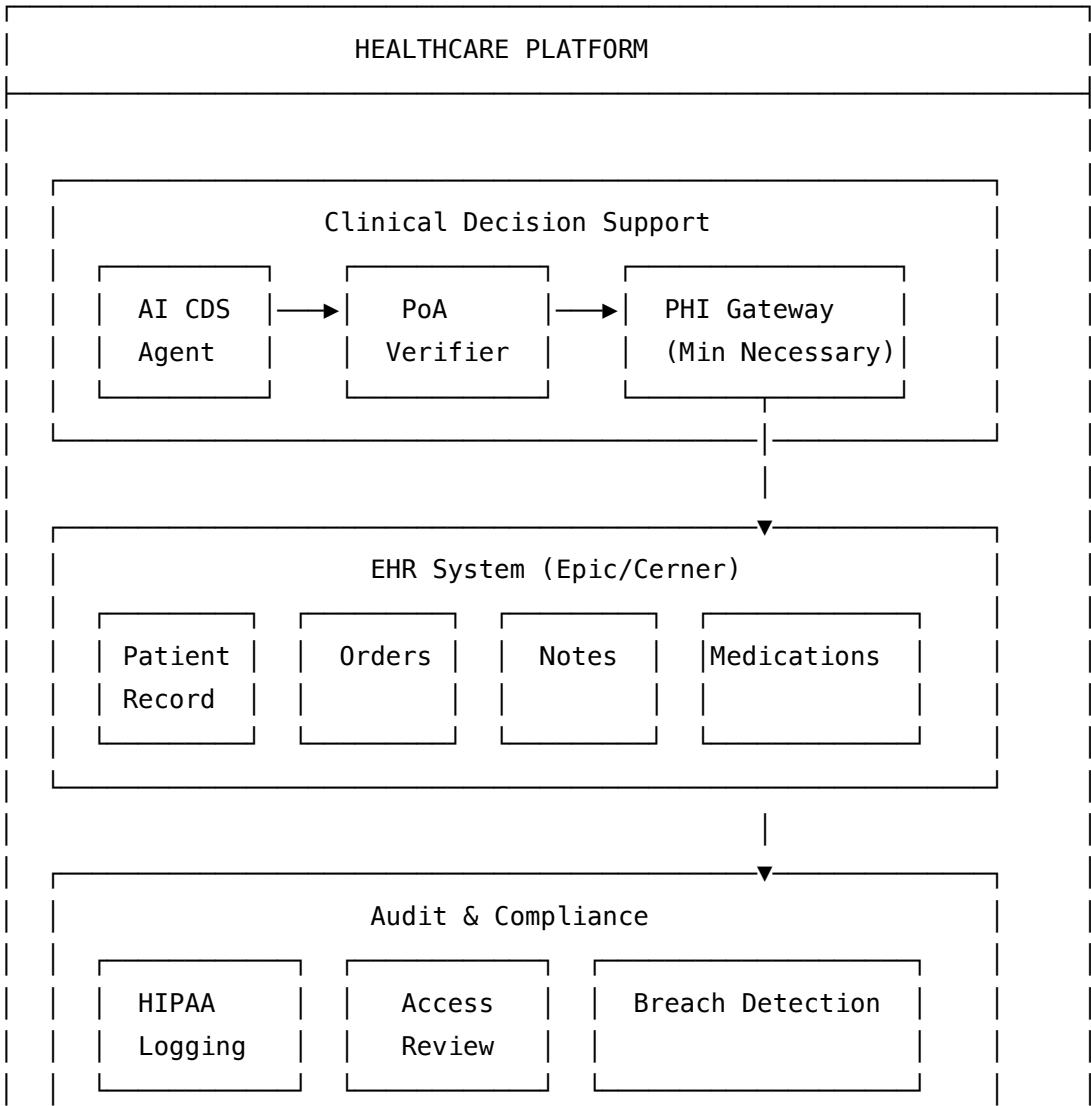
```

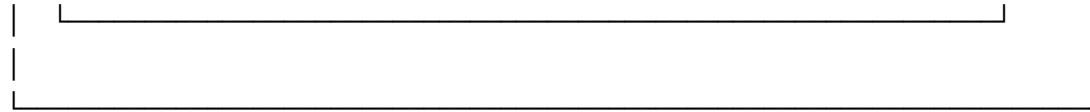
{
  "constraints": {
    "administrative_scope": {
      "allowed_actions": [
        "schedule_appointment",
        "send_reminder",
        "process_referral",
        "verify_insurance"
      ]
    },
    "phi_handling": {

```

```
    "access_level": "limited_phi",
    "allowed_phi_elements": ["name", "appointment_time", "referring_physician"],
    "prohibited_phi_elements": ["diagnosis", "treatment", "lab_results"]
  },
  "patient_communication": {
    "require_consent_check": true,
    "allowed_channels": ["portal_message", "sms"],
    "prohibited_content": ["clinical_results", "sensitive_diagnoses"]
  }
}
```

24.2.3 80.3 Integration with EHR Systems





24.2.4 80.4 FDA Considerations

For AI agents that may qualify as medical devices:

Documentation Requirements: - Software as a Medical Device (SaMD) classification
 - Predetermined change control plan - Algorithm transparency documentation - Clinical validation evidence

PoA Enhancement for FDA Compliance:

```
{
  "metadata": {
    "fda_classification": {
      "samd_category": "II",
      "intended_use": "Clinical decision support for medication reconciliation",
      "risk_level": "moderate",
      "clearance_number": "K123456"
    },
    "algorithm_version": {
      "model_hash": "sha256:abc123...",
      "training_date": "2024-06-15",
      "validation_metrics": {
        "sensitivity": 0.95,
        "specificity": 0.92,
        "auc": 0.97
      }
    },
    "predetermined_changes": {
      "allowed_without_resubmission": [
        "performance_improvement",
        "additional_drug_database_updates"
      ],
      "requires_resubmission": [
        "new_clinical_indication",
        "significant_algorithm_change"
      ]
    }
  }
}
```

```
}
```

24.3 Chapter 81: Government and Public Sector

24.3.1 81.1 Regulatory Framework

Government AI systems face unique requirements:

United States: - Administrative Procedure Act - Due Process Clause requirements - FOIA transparency obligations - OMB AI guidance (M-24-10) - Agency-specific regulations

European Union: - EU AI Act high-risk classification - National administrative law - GDPR (citizen data) - eGovernment regulations

24.3.2 81.2 PoA Constraint Patterns for Government

Benefits Determination Agent:

```
{
  "constraints": {
    "adjudication_rules": {
      "decision_basis": {
        "explicit_statutory_reference_required": true,
        "regulation_citation_required": true,
        "policy_interpretation_limited": true
      },
      "prohibited_factors": [
        "race", "religion", "political_affiliation",
        "protected_class_membership"
      ]
    },
    "due_process": {
      "adverse_action_requirements": {
        "written_notice_required": true,
        "reason_explanation_required": true,
        "appeal_information_required": true,
        "evidence_disclosure": "full"
      },
      "human_review": {
        "required_for_denial": true,
        "required_for_reduction": true,

```

```

        "reviewer_qualifications": ["gs-12_or_above", "program_trained"]
    },
    "transparency": {
        "decision_explainability": {
            "plain_language_required": true,
            "factor_weighting_visible": true,
            "foia_compliant": true
        },
        "audit_requirements": {
            "complete_decision_log": true,
            "periodic_bias_review": true,
            "public_reporting": true
        }
    },
    "pending_controls": {
        "max_benefit_amount": {"amount": 2000, "currency": "USD", "period": "monthly"},
        "aggregate_budget_check": true,
        "appropriation_verification": true
    }
}

```

24.3.3 81.3 Explainability Requirements

Government PoAs require enhanced explainability:

```

type GovernmentVerificationResult struct {
    VerificationResult

    // Explainability extension
    DecisionExplanation DecisionExplanation
}

type DecisionExplanation struct {
    // Plain language summary
    Summary string

    // Legal basis
    StatutoryAuthority []string
    RegulatoryCitations []string
}

```



```

    PolicyReferences    []string

    // Factor analysis
    FactorsConsidered []ConsideredFactor
    FactorsExcluded    []string // Prohibited factors verified not used

    // Evidence
    EvidenceReliedUpon []EvidenceItem

    // Appeal information
    AppealDeadline time.Time
    AppealProcess   string
    LegalAidInfo    string
}

type ConsideredFactor struct {
    Factor      string
    Value       interface{}
    Weight      float64
    Contribution string // "supports", "opposes", "neutral"
    Source      string
}

```

24.4 Chapter 82: Manufacturing and Supply Chain

24.4.1 82.1 Industry Requirements

Manufacturing and supply chain AI agents face:

- Quality management systems (ISO 9001)
- Product safety regulations
- Export control compliance
- Supply chain security

24.4.2 82.2 PoA Constraint Patterns

Procurement Agent Constraints:

```

{
  "constraints": {
    "supplier_restrictions": {
      "approved_supplier_list": {
        "source": "https://procurement.factory.example/approved-vendors",
        "refresh_interval_minutes": 60
      }
    }
  }
}

```

```

    },
    "quality_certifications_required": ["ISO_9001", "ISO_14001"],
    "prohibited_countries": ["Sanctioned countries list"],
    "debarred_parties_check": true
  },
  "quality_requirements": {
    "incoming_inspection_required": true,
    "certificate_of_conformance_required": true,
    "traceability_documentation_required": true
  },
  "compliance": {
    "export_control_screening": {
      "itar_check": true,
      "ear_check": true,
      "dual_use_screening": true
    },
    "conflict_minerals_check": true,
    "modern_slavery_certification": true
  },
  "spending": {
    "per_order_limit": {"amount": 100000, "currency": "USD"},
    "daily_limit": {"amount": 500000, "currency": "USD"},
    "contract_value_limit": {"amount": 1000000, "currency": "USD"}
  }
}

```

Quality Control Agent Constraints:

```

{
  "constraints": {
    "inspection_authority": {
      "allowed_actions": [
        "inspect_incoming",
        "inspect_in_process",
        "inspect_final",
        "record_nonconformance",
        "initiate_quarantine"
      ],
      "forbidden_actions": [
        "release_nonconforming_product",

```

```

        "modify_specifications",
        "override_hold"
    ]
},
"decision_escalation": {
    "critical_defect": {
        "escalate_to": ["quality_manager", "plant_manager"],
        "immediate_notification": true,
        "production_stop_authority": false
    },
    "major_defect": {
        "escalate_to": ["quality_supervisor"],
        "batch_quarantine": true
    }
},
"documentation": {
    "lot_traceability_required": true,
    "measurement_uncertainty_recorded": true,
    "calibration_verification_required": true
}
}

```

24.5 Chapter 83: Legal and Professional Services

24.5.1 83.1 Professional Responsibility

Legal AI agents must navigate: - Unauthorized practice of law restrictions - Attorney-client privilege - Conflicts of interest - Competence requirements

24.5.2 83.2 PoA Constraint Patterns

Legal Research Agent Constraints:

```

{
    "constraints": {
        "scope_limitations": {
            "allowed_actions": [
                "research_case_law",
                "summarize_statutes",

```

```

        "identify_relevant_precedents",
        "draft_research_memo"
    ],
    "forbidden_actions": [
        "provide_legal_advice",
        "draft_pleadings_without_review",
        "communicate_with_clients",
        "make_strategic_recommendations"
    ]
},
"privilege_protection": {
    "confidential_treatment_required": true,
    "no_external_transmission": true,
    "work_product_doctrine_compliance": true
},
"conflict_checking": {
    "required_before_new_matter": true,
    "conflict_database_source": "https://conflicts.lawfirm.example/v1",
    "escalate_potential_conflicts": true
},
"attorney_supervision": {
    "all_output_requires_review": true,
    "supervising_attorney_notification": true,
    "no_autonomous_filing": true
}
}
}

```

24.6 Chapter 84: Comparative Protocol Analysis

24.6.1 84.1 OAuth 2.0 Comparison

Feature	OAuth 2.0	PoA
Primary purpose	Access delegation	Authority delegation
Token semantics	“Can access”	“Authorized to act for”
Scope model	String lists	Structured actions/resources
Delegation depth	Limited	Arbitrary chains
Attenuation	No	Yes

Feature	OAuth 2.0	PoA
Revocation	Token-specific	Transparency log
Legal semantics	None	Explicit
Jurisdiction	None	Explicit
Liability bounds	None	Constraint-based

24.6.2 84.2 SPIFFE/SPIRE Comparison

Feature	SPIFFE/SPIRE	PoA
Identity model	Workload identity	Entity identity
Primary use case	Service mesh auth	Agent authorization
Principal type	Service	Human/Legal entity
Delegation	Limited	Full chains
Human accountability	None	Explicit

24.6.3 84.3 Verifiable Credentials Comparison

Feature	VCs	PoA
Purpose	Attribute attestation	Authority delegation
Issuer	Credential issuer	Principal
Subject	Holder	Agent
Claims	Attributes	Actions authorized
Delegation	Limited	Native support

Chapter 25

Part XIX: Extended Operational Guides

This section provides detailed operational guidance for production deployments.

25.1 Chapter 91: Key Management Operations

25.1.1 91.1 Key Lifecycle

Key Generation:

```
// Ceremony for production key generation
type KeyCeremony struct {
    witnesses    []Witness
    hsm          HSMInterface
    auditRecorder AuditRecorder
}

func (c *KeyCeremony) Generate(keyID string) (*KeyPair, error) {
    // Step 1: Verify witnesses present
    if len(c.witnesses) < 3 {
        return nil, errors.New("minimum 3 witnesses required")
    }

    // Step 2: Record ceremony start
    c.auditRecorder.Record(CeremonyEvent{
        Type: "key_generation_start",
    })
}
```

```

        KeyID:      keyID,
        Witnesses:  c.witnesses,
        Timestamp:  time.Now(),
    })

    // Step 3: Generate on HSM
    keyPair, err := c.hsm.GenerateKeyPair(HSMKeyParams{
        Algorithm:  "Ed25519",
        KeyID:      keyID,
        Extractable: false,
    })
    if err != nil {
        return nil, err
    }

    // Step 4: Witness verification
    for _, witness := range c.witnesses {
        if err := witness.Verify(keyPair.PublicKey); err != nil {
            return nil, fmt.Errorf("witness %s verification failed: %w", witness.ID, err)
        }
        witness.Sign(keyID, keyPair.PublicKey)
    }

    // Step 5: Record completion
    c.auditRecorder.Record(CeremonyEvent{
        Type:      "key_generation_complete",
        KeyID:      keyID,
        PublicKey:  keyPair.PublicKey,
        Witnesses:  c.witnesses,
        Timestamp:  time.Now(),
    })

    return keyPair, nil
}

```

25.1.2 91.2 Key Rotation

Rotation Schedule: | Key Type | Rotation Period | Overlap Period | |-----|-----
 -|-----| | Signing key | Annual | 90 days | | Encryption key | Quarterly | 30 days | |
 Session key | Daily | 2 hours |

Rotation Process:

```

type KeyRotation struct {
    currentKey *KeyPair
    nextKey    *KeyPair
    rotationAt time.Time
    overlapDays int
}

func (r *KeyRotation) Execute() error {
    // Step 1: Generate new key
    newKey, err := r.ceremony.Generate(newKeyID)
    if err != nil {
        return err
    }
    r.nextKey = newKey

    // Step 2: Update Entity Profile
    if err := r.updateProfile(newKey); err != nil {
        return err
    }

    // Step 3: Begin overlap period
    r.rotationAt = time.Now().Add(time.Duration(r.overlapDays) * 24 * time.Hour)

    // Step 4: Notify dependent systems
    r.notifyRotation(newKey)

    return nil
}

func (r *KeyRotation) CompleteRotation() error {
    if time.Now().Before(r.rotationAt) {
        return errors.New("overlap period not complete")
    }

    // Step 1: Make new key primary
    r.currentKey = r.nextKey
    r.nextKey = nil
}

```



```

    // Step 2: Archive old key
    r.archiveKey(r.previousKey)

    // Step 3: Update signing configuration
    r.setActiveKey(r.currentKey)

    return nil
}

```

25.1.3 91.3 Key Backup and Recovery

Backup Strategy:

```

type KeyBackup struct {
    shards    []BackupShard
    threshold int
    escrows   []EscrowLocation
}

func (b *KeyBackup) CreateBackup(key *KeyPair) error {
    // Step 1: Split key using Shamir's Secret Sharing
    shards, err := shamir.Split(key.PrivateSeed, b.threshold, len(b.escrows))
    if err != nil {
        return err
    }

    // Step 2: Encrypt each shard for escrow
    for i, shard := range shards {
        encrypted, err := b.encryptForEscrow(shard, b.escrows[i])
        if err != nil {
            return err
        }
        b.shards[i] = BackupShard{
            Index:      i,
            Ciphertext: encrypted,
            Escrow:      b.escrows[i].ID,
        }
    }

    // Step 3: Store shards at escrow locations
    for i, shard := range b.shards {

```

```

        if err := b.escrows[i].Store(shard); err != nil {
            return err
        }
    }

    return nil
}

func (b *KeyBackup) RecoverKey(shardInputs []BackupShard) (*KeyPair, error) {
    if len(shardInputs) < b.threshold {
        return nil, fmt.Errorf("need %d shards, got %d", b.threshold, len(shardInputs))
    }

    // Step 1: Decrypt shards
    decryptedShards := make([][]byte, len(shardInputs))
    for i, shard := range shardInputs {
        decrypted, err := b.decryptFromEscrow(shard)
        if err != nil {
            return nil, err
        }
        decryptedShards[i] = decrypted
    }

    // Step 2: Reconstruct key
    privateSeed, err := shamir.Combine(decryptedShards)
    if err != nil {
        return nil, err
    }

    return ed25519.NewKeyFromSeed(privateSeed), nil
}

```

25.2 Chapter 92: Incident Response

25.2.1 92.1 Incident Classification

Level	Description	Response Time	Examples
P1-Critical	System compromise	15 minutes	Key compromise, data breach
P2-High	Service degradation	1 hour	Log unavailable, verification failures
P3-Medium	Limited impact	4 hours	Single agent issues, constraint errors
P4-Low	Minimal impact	24 hours	Documentation issues, minor bugs

25.2.2 92.2 Runbooks

Runbook: Key Compromise Response

`incident_type: key_compromise`

`severity: P1-Critical`

`response_time: 15_minutes`

`steps:`

- `- name: "Confirm compromise"`

`actions:`

- `- Review evidence of compromise`
- `- Identify affected key ID`
- `- Document discovery time and method`

`owner: security_on_call`

- `- name: "Revoke all affected PoAs"`

`actions:`

- `- Query all active PoAs signed by compromised key`
- `- Submit revocation entries to all logs`
- `- Verify revocations published`

`owner: security_on_call`

`script: |`

```
gauth-cli poa list --signer-key $KEY_ID --active-only | \
  xargs -I {} gauth-cli poa revoke --poa-id {} --reason key_compromise
```

- `- name: "Rotate key"`

```
actions:
  - Generate new key pair
  - Update Entity Profile
  - Publish profile update
owner: key_admin
requires_ceremony: true

- name: "Re-issue critical PoAs"
  actions:
    - Identify business-critical PoAs
    - Issue replacement PoAs with new key
    - Distribute to affected agents
  owner: operations

- name: "Post-incident review"
  actions:
    - Document timeline
    - Identify root cause
    - Update security procedures
  owner: security_team
  deadline: 48_hours
```

Runbook: Revocation Log Unavailable

```
incident_type: revocation_log_unreachable
severity: P2-High
response_time: 1_hour
```

```
steps:
  - name: "Confirm outage"
    actions:
      - Check log health endpoints
      - Verify network connectivity
      - Check log operator status page
    owner: operations

  - name: "Activate degraded mode"
    actions:
      - Verify degraded mode policy is appropriate
      - Monitor degraded mode event rate
      - Alert stakeholders
```

```

owner: operations

- name: "Failover to backup log"
  condition: primary_log_extended_outage
  actions:
    - Enable secondary log queries
    - Verify synchronization status
    - Switch traffic to secondary
  owner: operations

- name: "Restore primary"
  condition: primary_log_recovered
  actions:
    - Verify log consistency
    - Gradual traffic shift to primary
    - Monitor for issues
  owner: operations

```

25.2.3 92.3 Communication Templates

Internal Notification:

```
## PoA Security Incident - [LEVEL]
```

```
**Incident ID:** INC-2026-0001
```

```
**Time Detected:** 2026-01-01T10:30:00Z
```

```
**Status:** Active Response
```

```
### Summary
```

```
[Brief description of incident]
```

```
### Impact
```

- Affected principals: [count]
- Affected agents: [count]
- Active PoAs impacted: [count]

```
### Actions Taken
```

```
1. [Action 1]
```

```
2. [Action 2]
```

```
### Next Steps
```

– [Next action and owner]

Contact

Incident Commander: [Name]

War Room: [Link/Location]

25.3 Chapter 93: Capacity Planning

25.3.1 93.1 Sizing Guidelines

Verifier Service: | Metric | Small | Medium | Large | |----|----|----|----| | Peak RPS
| 100 | 1,000 | 10,000 | | CPU cores | 2 | 8 | 32 | | Memory | 4 GB | 16 GB | 64 GB | | Cache
size | 1 GB | 8 GB | 32 GB | | Instances | 2 | 4 | 12 |

Issuer Service: | Metric | Small | Medium | Large | |----|----|----|----| | Peak is-
suanace/hr | 100 | 1,000 | 10,000 | | CPU cores | 2 | 4 | 16 | | Memory | 2 GB | 8 GB | 32 GB
| | HSM slots | 1 | 2 | 4 | | Instances | 1 | 2 | 4 |

25.3.2 93.2 Scaling Patterns

Horizontal Scaling:

Kubernetes HPA configuration

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: poa-verifier-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: poa-verifier

minReplicas: 3

maxReplicas: 20

metrics:

– type: Resource

resource:

name: cpu

target:

type: Utilization

```

        averageUtilization: 70
- type: Pods
  pods:
    metric:
      name: poa_verification_queue_depth
    target:
      type: AverageValue
      averageValue: "100"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 100
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10
          periodSeconds: 60

```

25.3.3 93.3 Performance Tuning

Cache Optimization:

```

type CacheConfig struct {
    // Entity profile cache
    ProfileCacheSize      int           // Number of profiles
    ProfileCacheTTL       time.Duration // Default 5 minutes

    // Verification result cache
    VerificationCacheSize int           // Number of results
    VerificationCacheTTL  time.Duration // Default 30 seconds

    // Revocation status cache
    RevocationCacheSize  int           // Number of PoA IDs
    RevocationCacheTTL   time.Duration // Default 60 seconds
}

func OptimalCacheConfig(expectedLoad int) CacheConfig {

```

```

return CacheConfig{
    ProfileCacheSize:      expectedLoad * 10, // 10x unique principals
    ProfileCacheTTL:       5 * time.Minute,
    VerificationCacheSize: expectedLoad * 60, // 1 minute of requests
    VerificationCacheTTL:  30 * time.Second,
    RevocationCacheSize:   expectedLoad * 100, // Active PoAs
    RevocationCacheTTL:    60 * time.Second,
}
}

```

25.4 Chapter 94: Compliance and Audit

25.4.1 94.1 Audit Requirements by Regulation

Regulation	Retention	Format	Access Control
SOX	7 years	Immutable	Role-based
HIPAA	6 years	Encrypted	Minimum necessary
GDPR	Duration of processing	Anonymizable	Subject access
PCI-DSS	1 year online, 7 archive	Tamper-evident	Need-to-know
MiFID II	5 years	Complete	Regulator access

25.4.2 94.2 Audit Log Schema

```

CREATE TABLE poa_audit_log (
    event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_time TIMESTAMPTZ NOT NULL DEFAULT now(),
    event_type VARCHAR(100) NOT NULL,

    -- PoA identification
    poa_id UUID,
    poa_hash BYTEA,

    -- Actor identification
    principal_id UUID,
    principal_name VARCHAR(255),
    agent_id UUID,
    agent_name VARCHAR(255),

```



```

-- Event details
action VARCHAR(255),
result VARCHAR(50), -- 'allowed', 'denied', 'degraded'
reason VARCHAR(255),

-- Context
source_ip INET,
user_agent TEXT,
request_id UUID,

-- Constraint evaluation
constraint_results JSONB,

-- Cryptographic linking
previous_hash BYTEA,
event_hash BYTEA NOT NULL,

-- Indexes for query performance
INDEX idx_poa_id (poa_id),
INDEX idx_principal (principal_id),
INDEX idx_agent (agent_id),
INDEX idx_event_time (event_time),
INDEX idx_event_type (event_type)
);

-- Trigger to compute cryptographic hash chain
CREATE OR REPLACE FUNCTION compute_event_hash()
RETURNS TRIGGER AS $$
BEGIN
    NEW.previous_hash := (
        SELECT event_hash FROM poa_audit_log
        ORDER BY event_time DESC LIMIT 1
    );
    NEW.event_hash := sha256(
        NEW.event_id::text ||
        NEW.event_time::text ||
        NEW.event_type ||
        COALESCE(NEW.poa_id::text, '') ||
        COALESCE(NEW.previous_hash::text, '')
    );

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER audit_hash_trigger
BEFORE INSERT ON poa_audit_log
FOR EACH ROW EXECUTE FUNCTION compute_event_hash();

```

25.4.3 94.3 Compliance Reporting

SOC 2 Type II Controls:

Control: PoA Issuance Authorization

****Control Objective:**** Only authorized personnel can issue PoAs on behalf of the organization

****Control Activities:****

1. PoA issuance requires authentication with MFA
2. Issuance is authorized by policy engine against role-based permissions
3. All issuance events are logged with actor identification
4. Quarterly access reviews verify role assignments

****Evidence:****

- Authentication logs showing MFA usage
 - Policy engine configuration and change history
 - Audit log samples with actor identification
 - Access review reports with sign-off
-

25.5 Chapter 95: Testing Strategies

25.5.1 95.1 Test Categories

Unit Tests:

```

func TestSpendingLimitConstraint(t *testing.T) {
    handler := NewSpendingLimitHandler()

    tests := []struct {
        name      string
        constraint SpendingLimitConfig
    }

```

```

    action      Action
    expected    bool
} {
    {
        name: "within_limit",
        constraint: SpendingLimitConfig{
            PerTransaction: Money{Amount: 1000, Currency: "USD"},
        },
        action: Action{Amount: Money{Amount: 500, Currency: "USD"}},
        expected: true,
    },
    {
        name: "exceeds_limit",
        constraint: SpendingLimitConfig{
            PerTransaction: Money{Amount: 1000, Currency: "USD"},
        },
        action: Action{Amount: Money{Amount: 1500, Currency: "USD"}},
        expected: false,
    },
    {
        name: "at_limit",
        constraint: SpendingLimitConfig{
            PerTransaction: Money{Amount: 1000, Currency: "USD"},
        },
        action: Action{Amount: Money{Amount: 1000, Currency: "USD"}},
        expected: true,
    },
}

for _, tc := range tests {
    t.Run(tc.name, func(t *testing.T) {
        result, err := handler.Evaluate(tc.constraint, tc.action, nil)
        require.NoError(t, err)
        assert.Equal(t, tc.expected, result.Allowed)
    })
}
}

```

Integration Tests:

```

func TestPoALifecycle(t *testing.T) {

```

```

// Setup
issuer := setupIssuer(t)
verifier := setupVerifier(t)
logClient := setupRevocationLog(t)

// Step 1: Issue PoA
poa, err := issuer.Issue(IssueRequest{
    Principal: testPrincipal,
    Agent:     testAgent,
    Scope:     testScope,
    Validity:  ValidityOneHour,
})
require.NoError(t, err)

// Step 2: Verify PoA is valid
result, err := verifier.Verify(context.Background(), poa)
require.NoError(t, err)
assert.True(t, result.Valid)

// Step 3: Revoke PoA
err = logClient.Revoke(poa.ID, "test_revocation")
require.NoError(t, err)

// Step 4: Wait for propagation
time.Sleep(2 * time.Second)

// Step 5: Verify PoA is now invalid
result, err = verifier.Verify(context.Background(), poa)
require.NoError(t, err)
assert.False(t, result.Valid)
assert.Equal(t, "revoked", result.Reason)
}

```

25.5.2 95.2 Load Testing

```

func BenchmarkVerificationThroughput(b *testing.B) {
    verifier := setupVerifier(b)
    poa := generateTestPoA()

    b.ResetTimer()

```

```

    b.RunParallel(func(pb *testing.PB) {
        for pb.Next() {
            _, _ = verifier.Verify(context.Background(), poa)
        }
    })
}

func TestVerificationLatencyUnderLoad(t *testing.T) {
    verifier := setupVerifier(t)
    poas := generateTestPoAs(1000)

    var latencies []time.Duration
    var mu sync.Mutex

    var wg sync.WaitGroup
    for i := 0; i < 100; i++ { // 100 concurrent clients
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := 0; j < 100; j++ { // 100 requests each
                poa := poas[rand.Intn(len(poas))]
                start := time.Now()
                _, _ = verifier.Verify(context.Background(), poa)
                latency := time.Since(start)

                mu.Lock()
                latencies = append(latencies, latency)
                mu.Unlock()
            }
        }()
    }
    wg.Wait()

    // Calculate percentiles
    sort.Slice(latencies, func(i, j int) bool {
        return latencies[i] < latencies[j]
    })

    p50 := latencies[len(latencies)*50/100]
    p95 := latencies[len(latencies)*95/100]

```

```

p99 := latencies[len(latencies)*99/100]

assert.Less(t, p50, 10*time.Millisecond, "P50 should be under 10ms")
assert.Less(t, p95, 50*time.Millisecond, "P95 should be under 50ms")
assert.Less(t, p99, 100*time.Millisecond, "P99 should be under 100ms")
}

```

25.5.3 95.3 Chaos Testing

```

// Chaos test: Revocation log becomes unreachable
func TestVerificationWithLogOutage(t *testing.T) {
    verifier := setupVerifier(t)
    poa := generateTestPoA()

    // Verify works normally
    result, err := verifier.Verify(context.Background(), poa)
    require.NoError(t, err)
    assert.True(t, result.Valid)

    // Simulate log outage
    mockLogServer.SetUnavailable(true)

    // Verify with degraded mode
    result, err = verifier.Verify(context.Background(), poa)
    require.NoError(t, err)

    switch verifier.config.DegradedModePolicy {
    case DegradedModeDeny:
        assert.False(t, result.Valid)
        assert.Equal(t, "revocation_check_failed", result.Reason)
    case DegradedModeAllow:
        assert.True(t, result.Valid)
        assert.True(t, result.DegradedMode)
    case DegradedModeUseCached:
        // Depends on cache state
    }

    // Restore log
    mockLogServer.SetUnavailable(false)
}

```

```

    // Verify normal operation resumed
    result, err = verifier.Verify(context.Background(), poa)
    require.NoError(t, err)
    assert.True(t, result.Valid)
    assert.False(t, result.DegradedMode)
}

```

25.6 Chapter 96: Troubleshooting Guide

25.6.1 96.1 Common Issues

Issue: Verification always fails with “signature_invalid”

Symptoms:

- All verifications return E021 (Invalid signature)
- New PoAs also fail verification

Diagnosis:

1. Check key ID in PoA matches registered key
2. Verify Entity Profile is current
3. Check for clock skew affecting validity

Resolution:

```
```bash
```

```
Check PoA signature key
```

```
gauth-cli poa inspect $POA_TOKEN | grep key_id
```

```
Verify key is registered
```

```
gauth-cli profile get $PROFILE_ID | jq '.public_keys[] | select(.key_id == "$KEY_ID")'
```

```
Check profile cache
```

```
gauth-cli verifier cache clear
```

##### \*\*Issue: High cache miss rate\*\*

```
```markdown
```

Symptoms:

- Slow verification latency
- High network traffic to external services

– Metrics show >50% cache misses

****Diagnosis:****

1. Check cache size vs active entity count
2. Verify TTL configuration
3. Check for cache eviction rate

****Resolution:****

```
```yaml
```

```
Increase cache size
```

```
verifier:
```

```
 cache:
```

```
 profiles:
```

```
 size: 10000 # Increase from default
```

```
 ttl: 10m # Increase TTL if appropriate
```

```
 verifications:
```

```
 size: 100000
```

```
 ttl: 60s
```

### 96.2 Debug Mode

```
```go
```

```
// Enable verbose logging for troubleshooting
```

```
func NewDebugVerifier(config VerifierConfig) *Verifier {
```

```
    return &Verifier{
```

```
        config: config,
```

```
        logger: NewDebugLogger(),
```

```
        tracer: NewVerboseTracer(),
```

```
    }
```

```
}
```

```
type DebugLogger struct {
```

```
    output io.Writer
```

```
}
```

```
func (l *DebugLogger) LogVerification(poa *PoA, result *VerificationResult) {
```

```
    fmt.Fprintf(l.output, `
```

```
=== PoA Verification Debug ===
```

```
PoA ID: %s
```


Principal: %s

Agent: %s

Scope: %v

Signature Check:

Algorithm: %s

Key ID: %s

Valid: %v

Chain Validation:

Depth: %d

Each Link Valid: %v

Validity:

Not Before: %s

Not After: %s

Current Time: %s

Valid: %v

Revocation:

Status: %s

Check Time: %s

Degraded: %v

Constraints:

%s

Final Result: %v

Reason: %s

=====

\, poa.ID, poa.Principal.Name, poa.Agent.Name, poa.Scope,

/* ... remaining fields ... */)

}

Chapter 26

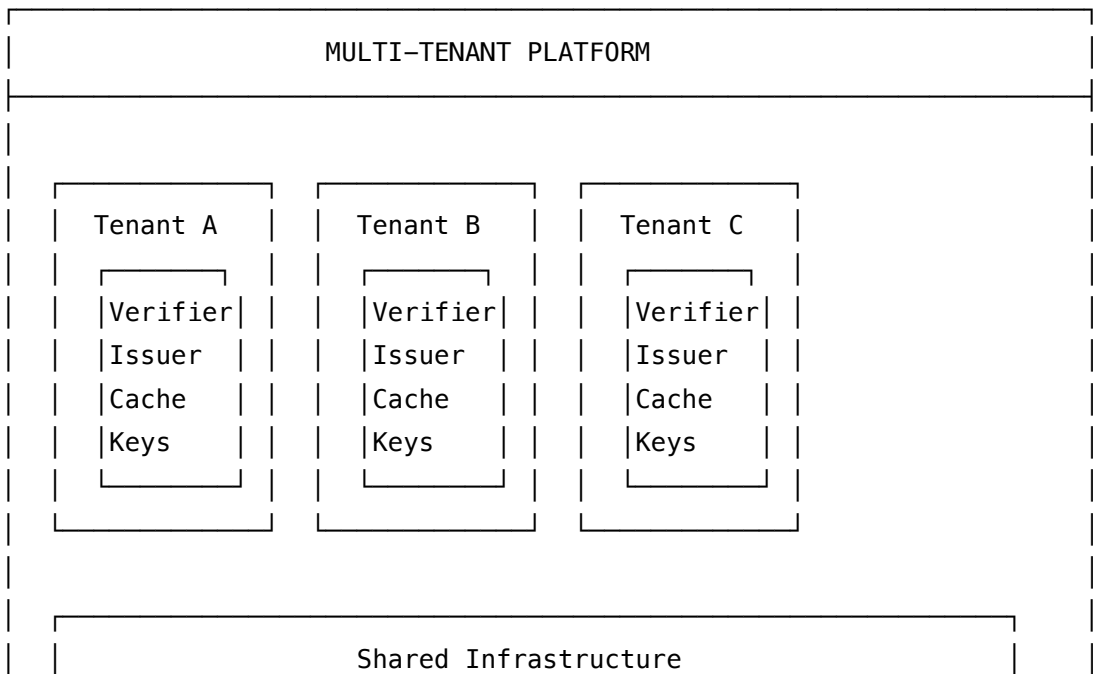
Part XX: Advanced Implementation Patterns

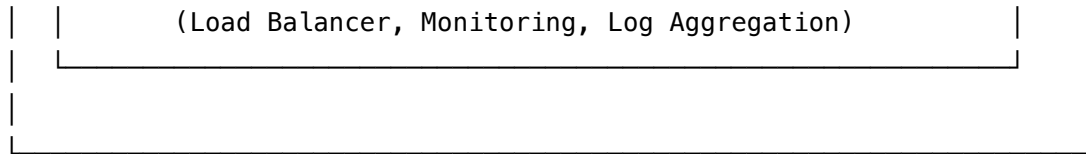
This section covers sophisticated implementation patterns for complex scenarios.

26.1 Chapter 97: Multi-Tenant Architecture

26.1.1 97.1 Tenant Isolation Models

Shared Nothing:



**Shared Services:**

```

type MultiTenantVerifier struct {
    tenantConfig map[string]*TenantConfig
    sharedCache  *Cache
}

func (v *MultiTenantVerifier) Verify(ctx context.Context, tenantID string, poa *PoA) (*VerificationResult, error) {
    // Get tenant configuration
    config, ok := v.tenantConfig[tenantID]
    if !ok {
        return nil, ErrUnknownTenant
    }

    // Apply tenant-specific constraints
    if err := v.applyTenantPolicy(config, poa); err != nil {
        return &VerificationResult{Valid: false, Reason: err.Error()}, nil
    }

    // Use shared cache with tenant prefix
    cacheKey := fmt.Sprintf("%s:%s", tenantID, poa.ID)
    if cached, ok := v.sharedCache.Get(cacheKey); ok {
        return cached.(*VerificationResult), nil
    }

    // Perform verification
    result, err := v.verifyWithTenantConfig(ctx, config, poa)
    if err != nil {
        return nil, err
    }

    // Cache result
    v.sharedCache.Set(cacheKey, result, config.CacheTTL)

    return result, nil
}

```

26.1.2 97.2 Cross-Tenant PoA

For scenarios where agents operate across tenant boundaries:

```
{
  "version": "AAP-002-v1.0",
  "poa_id": "...",
  "principal": {
    "profile_id": "tenant-a-principal-id",
    "tenant_id": "tenant-a"
  },
  "agent": {
    "profile_id": "tenant-b-agent-id",
    "tenant_id": "tenant-b"
  },
  "cross_tenant": {
    "authorized": true,
    "allowed_target_tenants": ["tenant-b", "tenant-c"],
    "scope_restrictions": {
      "tenant-b": ["read_only"],
      "tenant-c": ["read_only", "limited_write"]
    }
  }
}
```

26.2 Chapter 98: Event-Driven Integration

26.2.1 98.1 PoA Event Bus

```
type PoAEventBus struct {
  publisher EventPublisher
  handlers  map[EventType][]EventHandler
}
```

```
type PoAEvent struct {
  Type      EventType
  Timestamp time.Time
  Payload   interface{}
}
```

```

const (
    EventPoAIssued    EventType = "poa.issued"
    EventPoAVerified  EventType = "poa.verified"
    EventPoARevoked   EventType = "poa.revoked"
    EventPoAExpired   EventType = "poa.expired"
)

func (b *PoAEventBus) Publish(event PoAEvent) error {
    // Serialize event
    data, err := json.Marshal(event)
    if err != nil {
        return err
    }

    // Publish to message broker
    return b.publisher.Publish(string(event.Type), data)
}

func (b *PoAEventBus) Subscribe(eventType EventType, handler EventHandler) {
    b.handlers[eventType] = append(b.handlers[eventType], handler)
}

```

26.2.2 98.2 Webhook Integration

```

type WebhookConfig struct {
    URL      string
    Secret    string
    Events   []EventType
    Timeout  time.Duration
}

type WebhookDelivery struct {
    client *http.Client
    configs []WebhookConfig
}

func (d *WebhookDelivery) Deliver(event PoAEvent) error {
    for _, config := range d.configs {
        if !contains(config.Events, event.Type) {
            continue
        }
    }
}

```

```

    }

    go d.deliverToWebhook(config, event)
}
return nil
}

func (d *WebhookDelivery) deliverToWebhook(config WebhookConfig, event PoAEvent) {
    payload, _ := json.Marshal(event)

    // Compute signature
    signature := hmac.New(sha256.New, []byte(config.Secret))
    signature.Write(payload)
    sig := hex.EncodeToString(signature.Sum(nil))

    req, _ := http.NewRequest("POST", config.URL, bytes.NewReader(payload))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("X-PoA-Signature", "sha256="+sig)
    req.Header.Set("X-PoA-Event", string(event.Type))

    ctx, cancel := context.WithTimeout(context.Background(), config.Timeout)
    defer cancel()

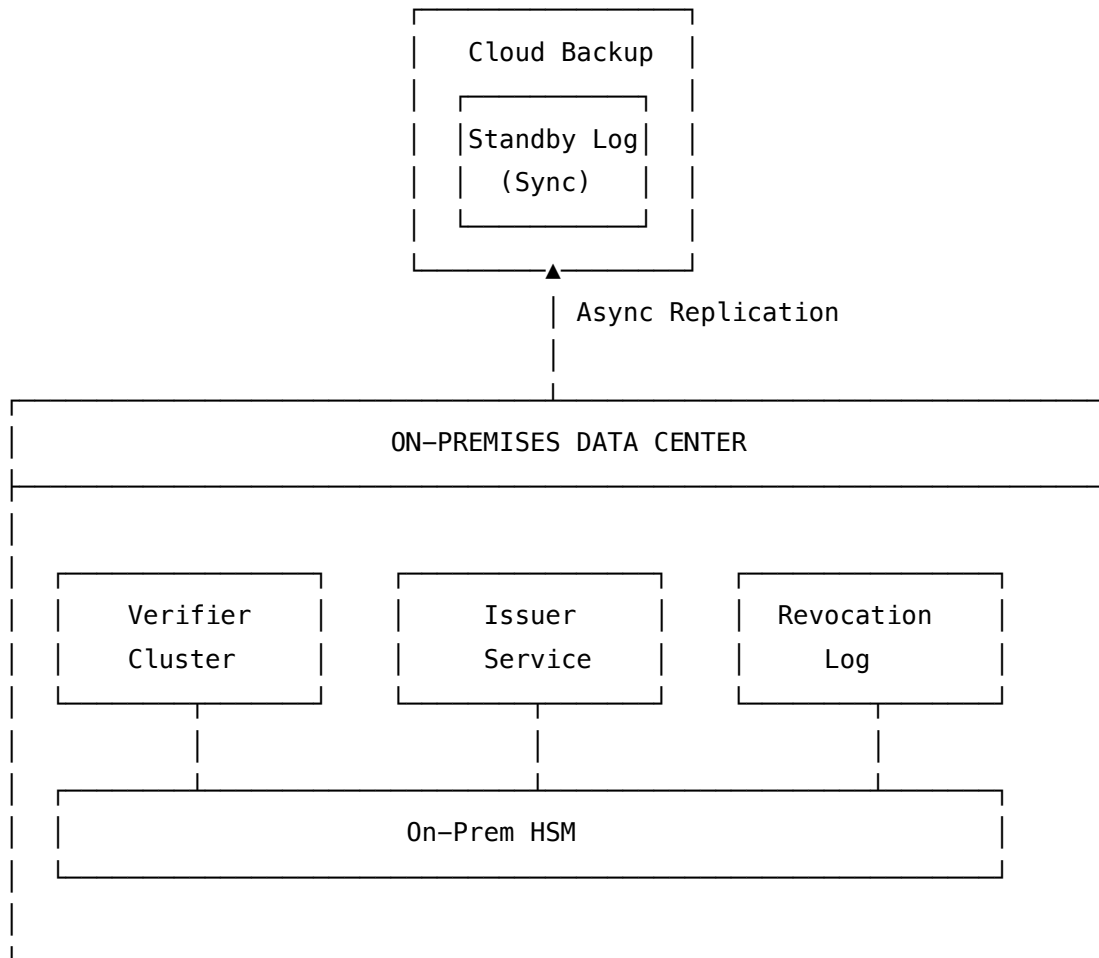
    resp, err := d.client.Do(req.WithContext(ctx))
    if err != nil {
        d.recordFailure(config, event, err)
        return
    }
    defer resp.Body.Close()

    if resp.StatusCode >= 400 {
        d.recordFailure(config, event, fmt.Errorf("HTTP %d", resp.StatusCode))
    }
}

```

26.3 Chapter 99: Hybrid Cloud Patterns

26.3.1 99.1 On-Premises with Cloud Backup



26.3.2 99.2 Federated Verification

```

type FederatedVerifier struct {
    localVerifier *Verifier
    remoteVerifiers map[string]*RemoteVerifier
}

func (v *FederatedVerifier) Verify(ctx context.Context, poa *PoA) (*VerificationResult, error) {
    // Determine if PoA is local or federated
    if v.isLocalPrincipal(poa.Principal) {
        return v.localVerifier.Verify(ctx, poa)
    }
}

```

```

    // Find remote verifier for principal's organization
    remoteOrg := poa.Principal.Organization
    remote, ok := v.remoteVerifiers[remoteOrg]
    if !ok {
        return nil, fmt.Errorf("unknown organization: %s", remoteOrg)
    }

    // Delegate verification to remote
    return remote.Verify(ctx, poa)
}

type RemoteVerifier struct {
    endpoint string
    apiKey   string
    client   *http.Client
}

func (r *RemoteVerifier) Verify(ctx context.Context, poa *PoA) (*VerificationResult, error) {
    // Encode PoA
    encoded, err := poa.Encode()
    if err != nil {
        return nil, err
    }

    // Call remote verification endpoint
    req, _ := http.NewRequest("POST", r.endpoint+"/verify", bytes.NewReader(encoded))
    req.Header.Set("Authorization", "Bearer "+r.apiKey)
    req.Header.Set("Content-Type", "application/cbor")

    resp, err := r.client.Do(req.WithContext(ctx))
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()

    var result VerificationResult
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return nil, err
    }
}

```



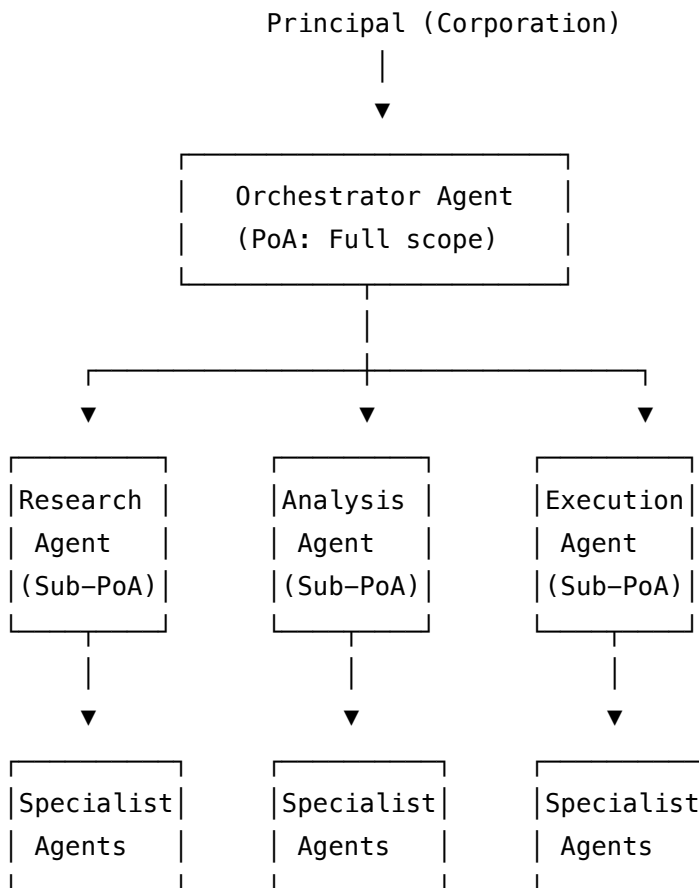
```

    return &result, nil
}

```

26.4 Chapter 100: AI Agent Orchestration

26.4.1 100.1 Agent Hierarchy



26.4.2 100.2 Dynamic Sub-PoA Issuance

```

type OrchestratorAgent struct {
    myPoA      *PoA
    signingKey ed25519.PrivateKey
    issuer     *Issuer
}

```

```

func (o *OrchestratorAgent) DelegateToSpecialist(task Task, specialist AgentRef) (*PoA, error) {
    // Verify task is within our authority
}

```

```

    if !o.myPoA.Scope.Contains(task.RequiredScope) {
        return nil, ErrExceedsAuthority
    }

    // Calculate attenuated scope
    delegatedScope := o.myPoA.Scope.Intersect(task.RequiredScope)

    // Calculate attenuated constraints
    delegatedConstraints := o.attenuateConstraints(task)

    // Issue sub-PoA
    subPoA, err := o.issuer.Issue(IssueRequest{
        Principal: o.myPoA.Principal, // Ultimate principal unchanged
        Agent:     specialist,
        Chain:     append(o.myPoA.Chain, o.createDelegationLink()),
        Scope:     delegatedScope,
        Constraints: delegatedConstraints,
        Validity: Validity{
            NotBefore: time.Now(),
            NotAfter:  minTime(o.myPoA.Validity.NotAfter, task.Deadline),
        },
    })
    if err != nil {
        return nil, err
    }

    return subPoA, nil
}

func (o *OrchestratorAgent) attenuateConstraints(task Task) Constraints {
    base := o.myPoA.Constraints

    // Task-specific attenuation
    return Constraints{
        SpendingLimit: SpendingLimit{
            PerTransaction: minMoney(
                base.SpendingLimit.PerTransaction,
                task.MaxSpending,
            ),
        },
    },
}

```

```

    TemporalRestrictions: TemporalRestrictions{
        NotAfter: task.Deadline,
    },
    // Additional task-specific constraints
}
}

```

26.5 Chapter 101: Legacy System Integration

26.5.1 101.1 Adapter Patterns

OAuth-to-PoA Bridge:

```

type OAuthToPoABridge struct {
    oauthValidator *oauth.Validator
    poaIssuer      *Issuer
    scopeMapper    ScopeMapper
}

func (b *OAuthToPoABridge) Bridge(oauthToken string) (*PoA, error) {
    // Validate OAuth token
    claims, err := b.oauthValidator.Validate(oauthToken)
    if err != nil {
        return nil, err
    }

    // Map OAuth scopes to PoA scope
    poaScope, err := b.scopeMapper.Map(claims.Scopes)
    if err != nil {
        return nil, err
    }

    // Derive constraints from OAuth claims
    constraints := b.deriveConstraints(claims)

    // Issue PoA
    return b.poaIssuer.Issue(IssueRequest{
        Principal: EntityRef{
            ProfileID: claims.Sub,

```

```

        Name:      claims.Name,
    },
    Agent: EntityRef{
        ProfileID: claims.ClientID,
        Name:      claims.ClientName,
    },
    Scope:      poaScope,
    Constraints: constraints,
    Validity: Validity{
        NotBefore: time.Now(),
        NotAfter:  claims.Exp,
    },
    Metadata: map[string]interface{}{
        "bridge_source": "oauth",
        "original_token_id": claims.JTI,
    },
})
}

```

SAML-to-PoA Bridge:

```

type SAMLToPoABridge struct {
    samlValidator *saml.Validator
    poaIssuer     *Issuer
}

func (b *SAMLToPoABridge) Bridge(samlAssertion string) (*PoA, error) {
    // Validate SAML assertion
    assertion, err := b.samlValidator.Validate(samlAssertion)
    if err != nil {
        return nil, err
    }

    // Extract attributes
    principal := b.extractPrincipal(assertion)
    agent := b.extractAgent(assertion)
    scope := b.extractScope(assertion)

    // Issue PoA
    return b.poaIssuer.Issue(IssueRequest{
        Principal: principal,
    })
}

```

```

    Agent:      agent,
    Scope:      scope,
    Constraints: b.defaultConstraints(),
    Validity: Validity{
        NotBefore: assertion.Conditions.NotBefore,
        NotAfter:  assertion.Conditions.NotOnOrAfter,
    },
})
}

```

26.6 Chapter 102: Privacy-Preserving Patterns

26.6.1 102.1 Minimal Disclosure

```

type MinimalDisclosureVerifier struct {
    verifier *Verifier
}

// VerifyWithMinimalDisclosure verifies a PoA without revealing full details
func (v *MinimalDisclosureVerifier) VerifyWithMinimalDisclosure(
    ctx context.Context,
    poa *PoA,
    action Action,
) (*MinimalVerificationResult, error) {
    // Full verification internally
    fullResult, err := v.verifier.Verify(ctx, poa)
    if err != nil {
        return nil, err
    }

    // Return only what's necessary
    return &MinimalVerificationResult{
        Allowed: fullResult.Valid && fullResult.Scope.Allows(action),
        // Don't reveal:
        // - Principal details
        // - Full scope
        // - Other constraints
    }, nil
}

```

26.6.2 102.2 Anonymized Audit Trails

```
type AnonymizedAuditLogger struct {
    innerLogger AuditLogger
    anonymizer  Anonymizer
}

func (l *AnonymizedAuditLogger) Log(event AuditEvent) error {
    // Anonymize before logging
    anonEvent := AuditEvent{
        EventID:    event.EventID,
        Timestamp:  event.Timestamp,
        EventType:  event.EventType,

        // Anonymized fields
        PrincipalID: l.anonymizer.HashID(event.PrincipalID),
        AgentID:     l.anonymizer.HashID(event.AgentID),
        PoAID:       l.anonymizer.HashID(event.PoAID),

        // Generalized fields
        Action:      l.anonymizer.GeneralizeAction(event.Action),
        Amount:      l.anonymizer.BucketAmount(event.Amount),

        // Preserved for analysis
        Result: event.Result,
    }

    return l.innerLogger.Log(anonEvent)
}
```

Appendix F: Sample PoA Artifacts

27.1 F.1 Basic Procurement PoA

[illegible]

```

"agent": {
  "profile_id": h'abcdef1234567890abcdef1234567890',
  "profile_hash": h'sha256:0123456789abcdef0123456789abcdef0123456789abcdef0123456789abc',
  "name": "ProcureBot-v2.1",
  "type": "ai_agent",
  "model_binding": {
    "model_id": "gpt-4-turbo-2024-04-09",
    "deployment_hash": "sha256:abc123..."
  },
  "verification_uri": "https://profiles.agentauth.example/acme-procurebot"
},

/ Delegation chain: Direct grant from principal /
"chain": [],

/ Scope: What actions the agent may perform /
"scope": {
  "actions": [
    "procurement:request_quote",
    "procurement:place_order",
    "procurement:negotiate_terms"
  ],
  "resources": [
    "products:office_supplies",
    "products:it_equipment",
    "products:maintenance_parts"
  ],
  "exclusions": [
    "products:capital_equipment",
    "products:real_estate"
  ]
},

/ Constraints: Enforceable limits on authority /
"constraints": {
  "spending_limits": {
    "per_transaction": {
      "amount": 10000,
      "currency": "USD"
    }
  },

```



```

    "per_day": {
      "amount": 50000,
      "currency": "USD"
    },
    "per_month": {
      "amount": 200000,
      "currency": "USD"
    }
  },
  "counterparty_restrictions": {
    "type": "dynamic_blocklist",
    "source": "https://compliance.acme.corp/denied-parties",
    "refresh_interval_minutes": 15,
    "fail_closed": true
  },
  "geographic_restrictions": {
    "allowed_countries": ["US", "CA", "MX", "DE", "FR", "GB", "JP"],
    "denied_countries": ["*"]
  },
  "temporal_restrictions": {
    "timezone": "America/New_York",
    "allowed_hours": "08:00-18:00",
    "allowed_days": ["Mon", "Tue", "Wed", "Thu", "Fri"]
  },
  "approval_requirements": {
    "threshold_amount": 5000,
    "approvers": ["did:key:z6Mk...", "did:key:z6Mn..."],
    "approval_timeout_minutes": 60
  }
},

/ Jurisdiction: Legal framework /
"jurisdiction": {
  "governing_law": "US-DE",
  "venue": "Delaware",
  "dispute_resolution": "arbitration:JAMS"
},

/ Validity period /
"validity": {

```

```

    "not_before": 1704067200, / 2024-01-01T00:00:00Z /
    "not_after": 1735689600 / 2025-01-01T00:00:00Z /
  },

  / Revocation endpoint /
  "revocation": {
    "endpoint": "https://logs.agentauth.example/v1",
    "method": "merkle_inclusion",
    "check_required": true
  },

  / Optional metadata /
  "metadata": {
    "purpose": "Automated procurement for routine office supplies",
    "issued_by": "procurement-automation-team",
    "audit_contact": "compliance@acme.corp"
  },

  / Cryptographic signatures /
  "signatures": [
    {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkpTHR8VNs8Kd9JXtpHMPPJYQkZPzB3nFfMvmW1vVvAqFk",
      "purpose": "authorization",
      "value": h'base64-encoded-64-byte-signature'
    }
  ]
}

```

Key Design Decisions: - Multi-tier spending limits (per-transaction, daily, monthly)
 - Dynamic blacklist with fail-closed semantics - Business hours restriction - Two-person approval for amounts > \$5,000

27.2 F.2 Delegated Authority PoA

Use Case: Department head delegates subset of their authority to AI assistant

```

{
  "version": "AAP-002-v1.0",

```

```

"poa_id": h'b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6a7',

/ Principal remains the corporation /
"principal": {
  "profile_id": h'1234567890abcdef1234567890abcdef',
  "name": "Acme Corporation"
},

/ Agent is AI assistant /
"agent": {
  "profile_id": h'def456789012345678901234567890ab',
  "name": "Marketing-Assistant-AI"
},

/ Delegation chain shows path of authority /
"chain": [
  {
    / First link: Corporation to CMO /
    "sequence": 0,
    "delegator": {
      "profile_id": h'1234567890abcdef1234567890abcdef',
      "name": "Acme Corporation"
    },
    "delegate": {
      "profile_id": h'789012345678901234567890abcdef12',
      "name": "Jane Smith, CMO"
    },
    "grant_type": "employment_contract",
    "scope_granted": {
      "actions": ["marketing:*", "advertising:*", "brand:*"],
      "budget": {"amount": 5000000, "currency": "USD", "period": "annual"}
    },
    "evidence": {
      "type": "contract_reference",
      "uri": "https://hr.acme.corp/contracts/jane-smith-cmo"
    },
    "signature": {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkAcme...",
      "value": h'...'
    }
  }
]

```

```

    }
  },
  {
    / Second link: CM0 to Marketing AI /
    "sequence": 1,
    "delegator": {
      "profile_id": h'789012345678901234567890abcdef12',
      "name": "Jane Smith, CM0"
    },
    "delegate": {
      "profile_id": h'def456789012345678901234567890ab',
      "name": "Marketing-Assistant-AI"
    },
    "grant_type": "ai_delegation",
    "scope_granted": {
      / Attenuated: Only content creation, not full marketing authority /
      "actions": ["content:draft", "content:schedule", "social:post"],
      "budget": {"amount": 1000, "currency": "USD", "period": "per_campaign"}
    },
    "evidence": {
      "type": "delegation_approval",
      "uri": "https://governance.acme.corp/delegations/mkt-ai-2024-001"
    },
    "signature": {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkJaneSmith...",
      "value": h'...'
    }
  }
],

/ Scope is intersection of chain grants /
"scope": {
  "actions": ["content:draft", "content:schedule", "social:post"],
  "resources": ["campaigns:*"],
  "exclusions": ["campaigns:crisis_response"]
},

"constraints": {
  "spending_limits": {

```

```

    "per_campaign": {"amount": 1000, "currency": "USD"}
  },
  "human_review": {
    "required_for": ["social:post"],
    "reviewers": ["did:key:z6MkJaneSmith...", "did:key:z6MkMarketingMgr..."],
    "timeout_action": "deny"
  },
  "content_restrictions": {
    "forbidden_topics": ["politics", "competitor_comparison"],
    "brand_guidelines_uri": "https://brand.acme.corp/guidelines/v2"
  }
},

"jurisdiction": {
  "governing_law": "US-DE",
  "venue": "Delaware"
},

"validity": {
  "not_before": 1704067200,
  "not_after": 1714521600 / 6 months /
},

"revocation": {
  "endpoint": "https://logs.agentauth.example/v1",
  "method": "merkle_inclusion"
},

/ All signatures in chain /
"signatures": [
  {
    "algorithm": "Ed25519",
    "key_id": "did:key:z6MkAcme...",
    "purpose": "chain_link_0",
    "value": h'...'
  },
  {
    "algorithm": "Ed25519",
    "key_id": "did:key:z6MkJaneSmith...",
    "purpose": "chain_link_1",

```

```

        "value": h'...'
    },
    {
        "algorithm": "Ed25519",
        "key_id": "did:key:z6MkJaneSmith...",
        "purpose": "authorization",
        "value": h'...'
    }
]
}

```

Key Design Decisions: - Two-link delegation chain with clear attenuation - Each delegation link is separately signed - Scope at each level is strictly more restrictive - Human review required before social posts

27.3 F.3 Multi-Signature PoA

Use Case: High-value transactions requiring multi-party authorization

```

{
    "version": "AAP-002-v1.0",
    "poa_id": h'c3d4e5f6a7b8c9d0e1f2a3b4c5d6a7b8',

    "principal": {
        "profile_id": h'1234567890abcdef1234567890abcdef',
        "name": "GlobalBank N.A."
    },

    "agent": {
        "profile_id": h'456789012345678901234567890abcdef',
        "name": "Treasury-Agent-AI"
    },

    "chain": [],

    "scope": {
        "actions": [
            "treasury:wire_transfer",
            "treasury:fx_transaction",

```

```

    "treasury:securities_trade"
  ],
  "resources": ["accounts:operating", "accounts:investment"]
},

"constraints": {
  "spending_limits": {
    "per_transaction": {"amount": 50000000, "currency": "USD"}
  },

  / Multi-signature requirement /
  "multi_authorization": {
    "type": "threshold",
    "threshold": 2,
    "signers": [
      {
        "role": "treasurer",
        "key_id": "did:key:z6MkTreasurer...",
        "name": "Treasury Department"
      },
      {
        "role": "cfo",
        "key_id": "did:key:z6MkCF0...",
        "name": "Chief Financial Officer"
      },
      {
        "role": "risk",
        "key_id": "did:key:z6MkRisk...",
        "name": "Risk Management"
      }
    ],
    "required_roles": ["treasurer"], / Treasurer must always approve /
    "time_window_minutes": 120
  },

  "counterparty_restrictions": {
    "kyc_required": true,
    "aml_screening": {
      "source": "https://compliance.globalbank.com/aml/v2",
      "refresh_interval_minutes": 5
    }
  }
}

```

```

    }
  },

  "regulatory_constraints": {
    "frameworks": ["OCC", "FDIC", "Fed"],
    "large_transaction_report": {
      "threshold": 10000,
      "currency": "USD",
      "report_to": "FinCEN"
    }
  }
},

"jurisdiction": {
  "governing_law": "US-NY",
  "venue": "New York",
  "regulatory_authority": ["OCC", "Fed"]
},

"validity": {
  "not_before": 1704067200,
  "not_after": 1711929600 / 3 months /
},

"revocation": {
  "endpoint": "https://logs.globalbank.com/revocation/v1",
  "method": "merkle_inclusion",
  "check_required": true,
  "max_staleness_seconds": 60
},

/ All required signatures /
"signatures": [
  {
    "algorithm": "Ed25519",
    "key_id": "did:key:z6MkTreasurer...",
    "purpose": "authorization",
    "timestamp": 1704067200,
    "value": h'...'
  },

```



```

    {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkCF0...",
      "purpose": "authorization",
      "timestamp": 1704067200,
      "value": h'...'
    }
  ]
}

```

Key Design Decisions: - 2-of-3 threshold signature requirement - Treasurer must always be one of the signers - Regulatory reporting built into constraints - Short validity period (3 months) for high-risk authority

27.4 F.4 Cross-Border PoA

Use Case: Agent operating across multiple jurisdictions

```

{
  "version": "AAP-002-v1.0",
  "poa_id": h'd4e5f6a7b8c9d0e1f2a3b4c5d6a7b8c9',

  "principal": {
    "profile_id": h'567890123456789012345678901234cd',
    "name": "EuroManu GmbH",
    "identifiers": [
      {"type": "LEI", "value": "391200FJKDAS8D9FJK20"},
      {"type": "DE_HRB", "value": "HRB 123456 München"}
    ]
  },

  "agent": {
    "profile_id": h'890123456789012345678901234567ef',
    "name": "SupplyChain-Agent-EU"
  },

  "chain": [
    {
      "sequence": 0,

```

```

    "delegator": {
      "profile_id": h'567890123456789012345678901234cd',
      "name": "EuroManu GmbH"
    },
    "delegate": {
      "profile_id": h'234567890123456789012345678901ab',
      "name": "Hans Mueller, Prokurist",
      "authority_type": "Prokura",
      "registry_reference": "HRB 123456 München"
    },
    "grant_type": "statutory_authority",
    "scope_granted": {
      "actions": ["all_commercial_acts"],
      "limitations": ["no_real_estate", "no_company_dissolution"]
    },
    "evidence": {
      "type": "commercial_register",
      "uri": "https://handelsregister.de/verify/HRB-123456"
    },
    "signature": {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkEuroManu...",
      "value": h'...'
    }
  },
  {
    "sequence": 1,
    "delegator": {
      "profile_id": h'234567890123456789012345678901ab',
      "name": "Hans Mueller, Prokurist"
    },
    "delegate": {
      "profile_id": h'890123456789012345678901234567ef',
      "name": "SupplyChain-Agent-EU"
    },
    "grant_type": "ai_delegation",
    "scope_granted": {
      "actions": ["procurement:all", "logistics:arrange"],
      "budget": {"amount": 500000, "currency": "EUR", "period": "monthly"}
    },
  },

```

```

    "signature": {
      "algorithm": "Ed25519",
      "key_id": "did:key:z6MkHansMueller...",
      "value": h'...'
    }
  }
],

"scope": {
  "actions": [
    "procurement:request_quote",
    "procurement:place_order",
    "logistics:book_transport",
    "customs:file_declaration"
  ],
  "resources": ["suppliers:eu_approved", "suppliers:ch_approved"]
},

"constraints": {
  "spending_limits": {
    "per_transaction": {"amount": 100000, "currency": "EUR"},
    "monthly": {"amount": 500000, "currency": "EUR"}
  },

  / Jurisdiction-specific requirements /
  "jurisdiction_rules": {
    "EU": {
      "vat_validation_required": true,
      "intrastat_threshold": 800000
    },
    "CH": {
      "customs_declaration_required": true,
      "swiss_vat_id_required": true
    },
    "GB": {
      "post_brexit_customs": true,
      "rules_of_origin_check": true
    }
  },
},

```

```
"counterparty_restrictions": {
  "eu_sanctions_check": {
    "source": "https://data.europa.eu/euodp/data/sanctions",
    "refresh_interval_minutes": 60
  },
  "trade_compliance": {
    "dual_use_screening": true,
    "export_license_check": true
  }
},

"data_protection": {
  "gdpr_compliance": true,
  "data_residency": ["EU", "CH"],
  "transfer_mechanism": "adequacy_decision"
}
},

/ Multi-jurisdiction specification /
"jurisdiction": {
  "primary_governing_law": "DE",
  "venue": "München",
  "conflict_rules": "EU_Rome_I",
  "recognized_jurisdictions": ["DE", "FR", "NL", "CH", "GB"],
  "regulatory_frameworks": {
    "DE": ["BaFin", "BAFA"],
    "EU": ["ESMA", "DG_TRADE"],
    "CH": ["SECO"]
  }
},

"validity": {
  "not_before": 1704067200,
  "not_after": 1735689600
},

"revocation": {
  "endpoint": "https://logs.agentauth.example/v1",
  "method": "merkle_inclusion"
},
```

```
"signatures": [  
  {  
    "algorithm": "Ed25519",  
    "key_id": "did:key:z6MkEuroManu...",  
    "purpose": "chain_link_0",  
    "value": h'...'   
  },  
  {  
    "algorithm": "Ed25519",  
    "key_id": "did:key:z6MkHansMueller...",  
    "purpose": "authorization",  
    "value": h'...'   
  }  
]  
}
```

Key Design Decisions: - Delegation chain rooted in German commercial register -
Prokura authority type explicitly documented - Jurisdiction-specific rules for each country
- GDPR data protection constraints embedded

Chapter 28

Appendix G: Verification Algorithm Pseudocode

```
FUNCTION VerifyPoA(poa_bytes, action, context):

    // Step 1: Parse
    poa = ParseCBOR(poa_bytes)
    IF poa is NULL:
        RETURN VerificationResult(valid=false, reason="parse_error")

    // Step 2: Version Check
    IF NOT SupportedVersion(poa.version):
        RETURN VerificationResult(valid=false, reason="unsupported_version")

    // Step 3: Signature Verification
    FOR each signature IN poa.signatures:
        public_key = ResolveKeyID(signature.key_id)
        IF public_key is NULL:
            RETURN VerificationResult(valid=false, reason="unknown_key")

        canonical_bytes = Canonicalize(poa, signature.purpose)
        IF NOT VerifySignature(public_key, canonical_bytes, signature.value):
            RETURN VerificationResult(valid=false, reason="signature_invalid")

    // Step 4: Chain Validation
    IF poa.chain.length > 0:
        current_authority = poa.principal
```

```

current_scope = UniversalScope()

FOR each link IN poa.chain:
    // Verify delegator matches previous delegate
    IF link.delegator != current_authority:
        RETURN VerificationResult(valid=false, reason="chain_break")

    // Verify link signature
    IF NOT VerifyLinkSignature(link):
        RETURN VerificationResult(valid=false, reason="chain_signature_invalid")

    // Verify scope attenuation
    IF NOT IsSubset(link.scope_granted, current_scope):
        RETURN VerificationResult(valid=false, reason="scope_escalation")

    current_authority = link.delegate
    current_scope = link.scope_granted

// Final agent must match chain endpoint
IF poa.agent != current_authority:
    RETURN VerificationResult(valid=false, reason="agent_mismatch")

// Step 5: Validity Period
now = CurrentTimestamp()
IF now < poa.validity.not_before:
    RETURN VerificationResult(valid=false, reason="not_yet_valid")
IF now > poa.validity.not_after:
    RETURN VerificationResult(valid=false, reason="expired")

// Step 6: Revocation Check
revocation_result = CheckRevocation(poa.poa_id, poa.revocation)
IF revocation_result.error:
    IF context.degraded_mode_policy == "deny":
        RETURN VerificationResult(valid=false, reason="revocation_unavailable")
    ELSE:
        degraded_mode = true
ELSE IF revocation_result.revoked:
    RETURN VerificationResult(valid=false, reason="revoked")

// Step 7: Scope Check

```

```
IF NOT ScopeAllows(poa.scope, action):
    RETURN VerificationResult(valid=false, reason="out_of_scope")

// Step 8: Constraint Evaluation
FOR each constraint IN poa.constraints:
    result = EvaluateConstraint(constraint, action, context)
    IF result.error:
        IF constraint.fail_closed:
            RETURN VerificationResult(valid=false, reason="constraint_error")
        ELSE:
            degraded_mode = true
    ELSE IF NOT result.allowed:
        RETURN VerificationResult(valid=false, reason=result.reason)

// Step 9: Return Result
RETURN VerificationResult(
    valid = true,
    degraded_mode = degraded_mode,
    evaluated_constraints = constraint_count,
    chain_depth = poa.chain.length
)
```

Chapter 29

Appendix H: Constraint Type Registry

This registry defines standard constraint types for interoperability.

Type ID	Category	Description
spending_limit	Financial	Maximum monetary value per action/period
spending_limit_aggregate	Financial	Cumulative spending across actions
currency_restriction	Financial	Allowed/denied currencies
counterparty_allowlist	Counterparty	Explicit list of allowed counterparties
counterparty_blocklist	Counterparty	Explicit list of denied counterparties
counterparty_dynamic	Counterparty	External source for counterparty validation
geographic_allow	Geographic	Countries/regions where action is permitted
geographic_deny	Geographic	Countries/regions where action is forbidden
temporal_hours	Temporal	Allowed hours of operation
temporal_days	Temporal	Allowed days of operation
temporal_embargo	Temporal	Period during which action is forbidden
approval_single	Approval	Single approver required
approval_threshold	Approval	N-of-M approvers required

Type ID	Category	Description
approval_role	Approval	Specific role must approve
human_review	Oversight	Human must review before execution
human_confirmation	Oversight	Human must confirm after AI recommendation
rate_limit	Rate	Maximum actions per time period
resource_restrict	Resource	Specific resources action applies to
resource_exclude	Resource	Specific resources action cannot apply to
compliance_framework	Regulatory	Required regulatory compliance
audit_required	Regulatory	Enhanced audit logging required
data_residency	Data	Where data may be processed
data_classification	Data	Maximum data classification level
model_binding	AI-Specific	Binding to specific AI model version
tool_restriction	AI-Specific	Tools the AI may/may not use
output_filter	AI-Specific	Required output filtering

Chapter 30

Appendix I: Entity Profile Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://agentauth.example/schemas/entity-profile-v1.json",
  "title": "Entity Profile (AAP-001)",
  "description": "Cryptographically signed identity document for PoA participants",

  "type": "object",
  "required": ["version", "profile_id", "entity_type", "name", "public_keys", "validity",

  "properties": {
    "version": {
      "type": "string",
      "pattern": "^AAP-001-v[0-9]+\\.?[0-9]+$"
    },

    "profile_id": {
      "type": "string",
      "format": "uuid"
    },

    "entity_type": {
      "type": "string",
      "enum": ["natural_person", "legal_entity", "ai_agent", "device", "service"]
    },

    "name": {
      "type": "string",
```

```

    "minLength": 1,
    "maxLength": 256
  },

  "identifiers": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["type", "value"],
      "properties": {
        "type": {
          "type": "string",
          "enum": ["LEI", "DUNS", "VAT", "EIN", "DID", "URL", "EMAIL", "NATIONAL_ID", "P
        },
        "value": {"type": "string"},
        "issuer": {"type": "string"},
        "verification_uri": {"type": "string", "format": "uri"}
      }
    }
  },

  "public_keys": {
    "type": "array",
    "minItems": 1,
    "items": {
      "type": "object",
      "required": ["key_id", "algorithm", "public_key_material"],
      "properties": {
        "key_id": {"type": "string"},
        "algorithm": {
          "type": "string",
          "enum": ["Ed25519", "ECDSA-P256", "ECDSA-P384", "RSA-2048", "RSA-4096", "Dilit
        },
        "public_key_material": {
          "type": "string",
          "description": "Base64-encoded public key"
        },
        "purpose": {
          "type": "array",
          "items": {

```

```

        "type": "string",
        "enum": ["signing", "encryption", "authentication"]
    }
},
    "valid_from": {"type": "string", "format": "date-time"},
    "valid_until": {"type": "string", "format": "date-time"}
}
},
"organization": {
    "type": "object",
    "properties": {
        "jurisdiction": {"type": "string"},
        "registration_authority": {"type": "string"},
        "registration_number": {"type": "string"},
        "legal_form": {"type": "string"},
        "parent_entity": {"$ref": "#"}
    }
},
"ai_agent_metadata": {
    "type": "object",
    "properties": {
        "model_family": {"type": "string"},
        "model_version": {"type": "string"},
        "deployment_hash": {"type": "string"},
        "operator": {"$ref": "#"},
        "capabilities": {
            "type": "array",
            "items": {"type": "string"}
        },
        "safety_certifications": {
            "type": "array",
            "items": {"type": "string"}
        }
    }
},
"validity": {

```

```

    "type": "object",
    "required": ["not_before", "not_after"],
    "properties": {
      "not_before": {"type": "integer", "description": "Unix timestamp"},
      "not_after": {"type": "integer", "description": "Unix timestamp"}
    }
  },

  "revocation_info": {
    "type": "object",
    "properties": {
      "log_endpoint": {"type": "string", "format": "uri"},
      "check_required": {"type": "boolean"}
    }
  },

  "signatures": {
    "type": "array",
    "minItems": 1,
    "items": {
      "type": "object",
      "required": ["algorithm", "key_id", "value"],
      "properties": {
        "algorithm": {"type": "string"},
        "key_id": {"type": "string"},
        "purpose": {
          "type": "string",
          "enum": ["self_signed", "issuer_attestation", "endorsement"]
        },
        "value": {"type": "string", "description": "Base64-encoded signature"}
      }
    }
  }
}

```

Chapter 31

Appendix J: Error Code Reference

Code	Category	Description	Resolution
E001	Parse	Invalid CBOR encoding	Check artifact encoding
E002	Parse	Missing required field	Add missing field
E003	Parse	Invalid field type	Correct field type
E010	Version	Unsupported version	Upgrade verifier or re-issue PoA
E011	Version	Deprecated version	Re-issue with current version
E020	Signature	Unknown key ID	Register key in profile
E021	Signature	Invalid signature	Re-sign with correct key
E022	Signature	Signature algorithm mismatch	Use declared algorithm
E023	Signature	Insufficient signatures	Add required signatures
E030	Chain	Chain break	Verify delegation chain
E031	Chain	Scope escalation	Reduce delegated scope
E032	Chain	Invalid link signature	Re-sign delegation link
E033	Chain	Agent mismatch	Verify chain endpoint
E040	Validity	Not yet valid	Wait until not_before
E041	Validity	Expired	Issue new PoA
E050	Revocation	Revoked	Cannot use; issue replacement
E051	Revocation	Check failed	Retry or enter degraded mode
E052	Revocation	Log unavailable	Retry later or fail closed

Code	Category	Description	Resolution
E060	Scope	Action not permitted	Request with correct scope
E061	Scope	Resource not permitted	Request correct resource
E070	Constraint	Spending limit exceeded	Reduce amount or request higher limit
E071	Constraint	Counterparty not allowed	Use allowed counterparty
E072	Constraint	Geographic restriction	Operate in permitted region
E073	Constraint	Temporal restriction	Operate in permitted hours
E074	Constraint	Approval required	Obtain required approvals
E080	External	Data source unavailable	Retry or fail per policy
E081	External	Profile fetch failed	Check profile availability
E090	Internal	Verification timeout	Increase timeout or optimize
E091	Internal	Resource exhausted	Scale verification infrastructure

Chapter 32

Appendix K: Standards Path and Industry Adoption

This appendix outlines the path toward standardization and widespread adoption of PoA.

32.1 K.1 Standardization Strategy

32.1.1 Phase 1: Community Specification (2024-2025)

Current Status: - Open specification published on GitHub - Reference implementation available (agentauth-go) - Community feedback incorporated - Initial production deployments

Deliverables: - Complete specification document (AAP-001, AAP-002) - Interoperability test suite - SDK implementations in major languages - Deployment guides

32.1.2 Phase 2: Industry Consortium (2025-2026)

Target Organizations: - Cloud providers (AWS, Azure, GCP) - Enterprise software vendors - Financial services firms - Healthcare IT providers - Government agencies

Consortium Activities: - Formal governance structure - Technical working groups - Certification program development - Public awareness campaigns

32.1.3 Phase 3: Formal Standards (2026-2028)

Potential Standards Bodies: - **IETF:** Wire format, protocol extensions - **W3C:** Decentralized identity integration - **ISO:** Management system standards - **IEEE:** Technical specifications - **NIST:** Security guidelines

Standards Track: | Document | Target Body | Timeline | |-----|-----|-----| | PoA Wire Format | IETF RFC | 2026-Q4 | | Entity Profile | W3C Note | 2027-Q1 | | Constraint Language | IETF RFC | 2027-Q2 | | Revocation Log Protocol | IETF RFC | 2027-Q3 | | Security Considerations | NIST SP | 2027-Q4 |

32.2 K.2 Industry Adoption Roadmap

32.2.1 Financial Services

Regulatory Catalyst: - SEC guidance on AI trading systems - OCC requirements for algorithmic accountability - EU AI Act compliance requirements

Adoption Path: 1. Pilot programs at tier-1 banks 2. Industry working group formation 3. Common constraint library development 4. Cross-institution PoA recognition 5. Regulatory endorsement

Target Milestones: - Q4 2025: First production deployment - Q2 2026: Multi-institution pilots - Q4 2026: Industry standard recommendation - Q2 2027: Regulatory guidance referencing PoA

32.2.2 Healthcare

Regulatory Catalyst: - FDA guidance on AI/ML medical devices - HHS requirements for AI decision accountability - State medical board AI prescribing rules

Adoption Path: 1. EHR vendor integration partnerships 2. Health system pilot programs 3. FDA recognition pathway 4. Specialty society endorsements 5. Payer requirement incorporation

32.2.3 Supply Chain

Industry Catalyst: - Global supply chain disruptions - Cross-border trade complexity - ESG compliance requirements

Adoption Path: 1. Consortium formation (manufacturers, logistics, retailers) 2. Shared infrastructure deployment 3. Interoperability testing 4. Industry certification program 5. Government endorsement for customs

32.2.4 Government

Regulatory Catalyst: - Federal AI executive orders - State AI accountability laws - Administrative procedure requirements

Adoption Path: 1. Pilot programs in benefits administration 2. Procurement system integration 3. Agency policy development 4. OMB guidance incorporating PoA 5. Legislative recognition

32.3 K.3 Competitive Landscape

32.3.1 Current Alternatives

Alternative	Strengths	Weaknesses
OAuth 2.0	Widely deployed	Access not authority
Macaroons	Delegation, attenuation	No legal semantics
UCAN	Decentralized	Limited adoption
SPIFFE/SPIRE	Service identity	No human principal
Attribute-Based Access Control	Fine-grained	Complex, no delegation

32.3.2 PoA Differentiation

1. **Legal Authority Binding:** Only system designed for legal liability chains
2. **Jurisdiction Specification:** Explicit conflict-of-law handling
3. **Revocation Transparency:** Public auditability
4. **Constraint-Based Limits:** Machine-enforceable liability bounds
5. **Evidence for Courts:** Designed for post-hoc legal analysis

32.4 K.4 Patent and IP Considerations

32.4.1 Open Specification Policy

The AgentAuth project operates under: - Apache 2.0 license for software - CC-BY-4.0 license for specifications - Royalty-free patent commitment from contributors

32.4.2 Defensive Patent Strategy

To protect the community: - File defensive patents on core innovations - Commit patents to Open Invention Network or similar - Ensure freedom to operate for implementers

32.4.3 Prior Art Documentation

This book serves as prior art for: - Legal authority binding in authorization protocols -
Jurisdiction-specific constraint encoding - Revocation transparency for delegation chains -
Constraint evaluation with legal evidence preservation

Chapter 33

Appendix L: Ecosystem Governance

33.1 L.1 Governance Principles

33.1.1 Open Development

All specification work happens in public: - GitHub for specifications and code - Public mailing lists for discussion - Recorded meetings with published minutes - Open contribution process

33.1.2 Rough Consensus

Decisions made through: - Discussion and deliberation - Working group consensus - Appeals process for disputes - Final decision by steering committee

33.1.3 Running Code

Emphasis on: - Working implementations over theoretical design - Interoperability testing as validation - Production deployment experience as input

33.2 L.2 Organizational Structure

33.2.1 Steering Committee

Responsibilities: - Strategic direction - Standards body liaison - Trademark management - Conflict resolution

Composition: - 5-7 members - Elected by contributor community - 2-year terms, staggered - No more than 2 from same organization

33.2.2 Technical Working Groups

WG-Core: Core protocol specification **WG-Constraints:** Constraint language and registry **WG-Revocation:** Transparency log protocol **WG-Interop:** Interoperability testing **WG-Security:** Security analysis and incident response

33.2.3 Advisory Boards

Legal Advisory Board: Attorneys specializing in agency law, AI regulation **Industry Advisory Board:** Representatives from adopting organizations **Academic Advisory Board:** Researchers in security, law, AI

33.3 L.3 Contribution Process

33.3.1 Specification Changes

1. **Proposal:** Author submits RFC to appropriate WG
2. **Discussion:** WG reviews and provides feedback
3. **Revision:** Author incorporates feedback
4. **Consensus:** WG reaches rough consensus
5. **Review:** Steering Committee reviews major changes
6. **Publication:** Update to specification

33.3.2 Code Contributions

Standard open-source process: - Fork repository - Implement changes with tests - Submit pull request - Code review by maintainers - CI validation - Merge by maintainer

33.4 L.4 Certification Program

33.4.1 Implementer Certification

Levels: - **Basic Compliance:** Passes interoperability test suite - **Full Compliance:** Passes extended tests + security review - **Production Certified:** Full compliance + operational audit

Process: 1. Self-assessment against checklist 2. Automated test suite execution 3. Third-party security review (Full/Production) 4. Certification committee review 5. Certification granted (valid 2 years)

33.4.2 Log Operator Certification

Requirements: - 99.9% availability SLA - Sub-second inclusion proof response - Hourly STH publication - Annual security audit - Incident response plan

Chapter 34

Part XVIII: Formal Verification and Research Directions

This section covers formal verification approaches and identifies key research questions.

34.1 Chapter 85: Formal Security Analysis

34.1.1 85.1 Security Model

Threat Model Assumptions: 1. Adversary controls network (active man-in-the-middle) 2. Adversary cannot break cryptographic primitives 3. Adversary may compromise some (but not all) keys 4. Adversary may compromise some log operators (minority) 5. Time is loosely synchronized (within hours)

Security Goals: 1. **Authenticity:** Only legitimate principals can issue valid PoAs 2. **Non-repudiation:** Principals cannot deny issued PoAs 3. **Attenuation integrity:** Delegation cannot escalate authority 4. **Revocation timeliness:** Revocations are visible within specified window 5. **Accountability:** All actions traceable to principals

34.1.2 85.2 Formal Properties

Property 1: Signature Unforgeability

$\forall$ attacker A , $\forall$ PoA p :

$$\Pr[A \text{ outputs valid } p \text{ without knowing } \text{private_key}(\text{principal}(p))] < \epsilon$$

where ϵ is negligible in security parameter

Property 2: Chain Attenuation


```

 $\forall$  chain  $c = [\text{link}_1, \dots, \text{link}_n]$ :
   $\text{scope}(\text{link}_n) \subseteq \text{scope}(\text{link}_{\{n-1\}}) \subseteq \dots \subseteq \text{scope}(\text{link}_1)$ 

```

Violation detectable during verification

Property 3: Revocation Visibility

```

 $\forall$  PoA  $p$ ,  $\forall$  time  $t$ :
  If  $p$  revoked at time  $t_r$ , then
     $\forall$  verifier  $v$  checking at time  $t > t_r + \text{gossip\_delay}$ :
       $v.\text{revocation\_check}(p)$  returns REVOKED

```

(Assuming honest majority of log operators)

34.1.3 85.3 Attack Surface Analysis

Attack Vector	Mitigation	Residual Risk
Key compromise	HSM, rotation	Key theft before rotation
Log compromise	Multi-log, gossip	Split-view if majority compromised
Replay	Validity periods, nonces	Replay within validity period
Scope escalation	Chain validation	None (cryptographically enforced)
Constraint bypass	Verifier integrity	Malicious verifier
Profile spoofing	Signature verification	Compromised profile key

34.1.4 85.4 Formal Verification with TLA+

PoA Protocol State Machine:

```
---- MODULE PoAProtocol ----
```

```
EXTENDS Naturals, Sequences, TLC
```

```
CONSTANTS Principals, Agents, Scopes, MaxChainDepth
```

VARIABLES

```

issuedPoAs,      /* Set of issued PoAs
revokedPoAs,     /* Set of revoked PoA IDs
logState,        /* Current state of revocation log
agentActions     /* Actions taken by agents

```

```

TypeInvariant ==
  /\ issuedPoAs \subseq [
    id: Nat,
    principal: Principals,
    agent: Agents,
    scope: Scopes,
    chain: Seq(DelegationLink)
  ]
  /\ revokedPoAs \subseq Nat
  /\ Len(logState.entries) >= 0

/* Safety: No agent can act outside its scope */
SafetyProperty ==
  \A action \in agentActions:
    LET poa == GetPoAForAction(action)
    IN ActionInScope(action, poa.scope)

/* Safety: Revoked PoAs cannot authorize actions */
RevocationSafety ==
  \A action \in agentActions:
    LET poa == GetPoAForAction(action)
    IN poa.id \notin revokedPoAs

/* Liveness: Eventually all revocations are visible */
RevocationLiveness ==
  \A poa \in revokedPoAs:
    <>(\A verifier \in Verifiers:
      verifier.sees_revocation(poa))

====

```

34.2 Chapter 86: Open Research Questions

34.2.1 86.1 Technical Challenges

1. Constraint Expressiveness vs. Verifiability - How rich can constraint language be while remaining machine-verifiable? - Trade-offs between expressiveness and verification complexity - Research directions: constraint synthesis, SMT-based verification

2. Aggregate Constraint Coordination - How to enforce aggregate limits across distributed agents? - Consistency vs. availability trade-offs - Research directions: distributed counters, causal consistency

3. Post-Quantum Migration - Timeline for quantum computer threats - Transition strategies for long-lived PoAs - Research directions: hybrid signatures, algorithm agility

4. Privacy-Preserving Verification - Zero-knowledge proofs of constraint satisfaction - Selective disclosure of PoA components - Research directions: ZK-SNARKs for constraint proofs

34.2.2 86.2 Legal Questions

1. Cross-Jurisdiction Recognition - How to achieve mutual recognition of PoAs across jurisdictions? - Role of treaties, harmonization efforts - Research directions: conflict of laws, private international law

2. Liability Allocation - How do courts interpret PoA constraints? - Precedents for machine-enforced limits - Research directions: agency law evolution, AI liability frameworks

3. Regulatory Classification - How do regulators classify PoA-authorized agents? - Sector-specific requirements - Research directions: regulatory sandboxes, classification frameworks

34.2.3 86.3 Societal Questions

1. Agent Proliferation - What are the societal effects of ubiquitous authorized agents? - Employment, decision-making, accountability shifts - Research directions: economic impact studies, governance frameworks

2. Accountability Gaps - What happens when authorized actions cause harm within bounds? - Principal awareness and oversight duties - Research directions: duty of care for AI deployment

3. Power Concentration - Who controls PoA infrastructure? - Governance and anti-trust considerations - Research directions: decentralized governance, anti-monopoly measures

34.3 Chapter 87: Implementation Roadmap

34.3.1 87.1 Phase 1: Foundation (Year 1)

Milestones: - Core protocol specification finalized - Reference implementation (Go) production-ready - Python and JavaScript SDKs - First production deployments

Success Criteria: - 10+ organizations in production - 1M+ PoAs issued - No critical security issues

34.3.2 87.2 Phase 2: Ecosystem (Year 2)

Milestones: - Multi-language SDK coverage - Multiple log operators - Constraint registry established - Certification program launched

Success Criteria: - 100+ organizations in production - Cross-organization PoA usage - Industry working groups active

34.3.3 87.3 Phase 3: Standards (Year 3)

Milestones: - IETF drafts submitted - Regulatory recognition in 2+ jurisdictions - Major platform integrations

Success Criteria: - Formal standards on track - Regulatory guidance referencing PoA - Ecosystem self-sustaining

34.3.4 87.4 Phase 4: Ubiquity (Year 4+)

Vision: - PoA as standard for AI agent authorization - Built into major AI platforms - Legal precedents established - Multi-jurisdiction interoperability

34.4 Chapter 88: Philosophical Foundations

34.4.1 88.1 Authority and Machines

The fundamental question: Can a machine have authority?

Traditional authority derives from: - Legal status (personhood) - Social acceptance - Institutional backing - Accountability

PoA's answer: Machines don't have authority; they exercise delegated authority from humans who do.

The signature on a PoA is not the agent's authority—it's the principal's authorization for the agent to act within specified bounds. The principal retains moral and legal responsibility.

34.4.2 88.2 Trust and Verification

"Trust, but verify" becomes "Verify, then trust appropriately."

PoA shifts the trust model: - Trust in cryptography (mathematical) - Trust in specification (auditable) - Trust in transparency (verifiable) - Trust in principal (legal accountability)

34.4.3 88.3 Agency and Autonomy

Philosophical tension between: - Agent autonomy: Efficiency requires freedom to act - Principal control: Accountability requires limits

PoA navigates this by: - Explicit scope: What the agent may do - Explicit constraints: Conditions on actions - Explicit revocation: How to withdraw authority

The agent is "free" within its authorized bounds—and those bounds are cryptographically enforced.

34.5 Chapter 89: Economic Analysis

34.5.1 89.1 Cost-Benefit Framework

Costs of Implementation: - Infrastructure: Log hosting, key management, profile resolution - Development: SDK integration, constraint implementation - Operations: Monitoring, incident response, key rotation - Compliance: Audit, documentation, training

Benefits: - Reduced liability exposure (constrained authority) - Audit efficiency (machine-verifiable trails) - Incident response speed (instant revocation) - Cross-organization interoperability - Regulatory compliance evidence

34.5.2 89.2 Market Structure

Potential Service Providers: 1. **Log Operators:** Revocation transparency infrastructure - Revenue model: Subscription per organization - Estimated market: \$100M-500M annually at scale

2. **Profile Authorities:** Identity verification and profile issuance

- Revenue model: Per-profile fee + annual maintenance
- Integration with existing KYC/KYB providers

3. **Verification Services:** Hosted verification as a service

- Revenue model: Per-verification or subscription
 - Value: Reduced integration complexity
4. **Compliance Tools:** Constraint libraries, audit tools
- Revenue model: Software licensing
 - Value: Regulatory alignment

34.5.3 89.3 Adoption Barriers

Barrier	Severity	Mitigation
Technical complexity	Medium	SDKs, managed services
Regulatory uncertainty	High	Engage regulators early
Network effects	High	Consortium formation
Legacy integration	Medium	Migration tooling
Cost of change	Medium	ROI demonstration

34.6 Chapter 90: Conclusion

34.6.1 90.1 Summary of Contributions

This book has presented:

1. **A Problem Definition:** The gap between AI capabilities and authorization frameworks
2. **A Technical Solution:** PoA artifacts with cryptographic verification
3. **A Legal Framework:** Mapping to agency law across jurisdictions
4. **An Implementation Path:** Practical guidance for adoption
5. **A Research Agenda:** Open questions for continued development

34.6.2 90.2 The Path Forward

The agent's signature is, in the end, our signature.

When we deploy autonomous systems, we embed our values, our constraints, and our accountability into their operation. PoA provides the technical infrastructure to make those commitments visible, verifiable, and enforceable.

The work is far from complete. Standards must be established. Regulations must be clarified. Implementations must be hardened. But the foundation is laid.

The future of human-AI collaboration depends on trust. PoA provides one piece of that trust infrastructure—a way for humans to grant bounded authority to machines, and to verify that those bounds are respected.

The agent’s signature is our promise—to each other, to our counterparties, to society—that even as we delegate to machines, we remain accountable.

Chapter 35

Appendix N: Complete Protocol Specification

This appendix provides the complete, authoritative specification for PoA encoding and verification.

35.1 N.1 CBOR Schema Definition

; CDDL Schema for PoA AAP-002-v1.0

```
poa = {  
  version: text,           ; MUST be "AAP-002-v1.0"  
  poa_id: bytes,           ; 32-byte unique identifier  
  principal: entity_ref,  
  agent: entity_ref,  
  scope: scope_def,  
  ? chain: [+ delegation],  
  ? constraints: constraints_map,  
  validity: validity_period,  
  signature: signature_block,  
  ? metadata: { * text => any }  
}
```

```
entity_ref = {  
  profile_id: text,        ; Entity Profile ID  
  name: text,  
  ? organization: text,
```



```

    ? jurisdiction: text,
    public_key: bytes,          ; Ed25519 public key (32 bytes)
    ? profile_uri: text
}

scope_def = {
    actions: [+ text],          ; List of authorized actions
    ? resources: [+ text],      ; List of resource URIs or patterns
    ? geofences: [+ geofence]
}

geofence = {
    center: { lat: float, lon: float },
    radius_meters: uint
}

delegation = {
    delegator: entity_ref,
    delegatee: entity_ref,
    timestamp: int,             ; Unix timestamp
    signature: bytes            ; Ed25519 signature
}

constraints_map = {
    ? spending_limit: spending_constraint,
    ? temporal: temporal_constraint,
    ? approval_required: approval_constraint,
    ? custom: { * text => any }
}

spending_constraint = {
    ? per_transaction: money,
    ? per_day: money,
    ? per_month: money,
    currency: text
}

money = {
    amount: int,                ; Amount in smallest unit (e.g., cents)
    currency: text              ; ISO 4217 code
}

```

```

}

temporal_constraint = {
    ? allowed_hours: [+ hour_range],
    ? allowed_days: [+ int], ; 0=Sunday, 6=Saturday
    ? blackout_dates: [+ text]
}

hour_range = {
    start: uint, ; 0-23
    end: uint ; 0-23
}

approval_constraint = {
    required_above: money,
    approval_endpoint: text,
    ? timeout_seconds: uint
}

validity_period = {
    not_before: int, ; Unix timestamp
    not_after: int ; Unix timestamp
}

signature_block = {
    algorithm: text, ; "Ed25519"
    value: bytes, ; Signature bytes
    ? public_key: bytes ; For inline key distribution
}

```

35.2 N.2 Verification Algorithm (Normative)

VERIFY(poa, current_time, action, context):

1. STRUCTURAL VALIDATION
 - a. Decode CBOR, MUST be well-formed
 - b. version MUST equal "AAP-002-v1.0"
 - c. poa_id MUST be 32 bytes
 - d. All required fields MUST be present

2. TEMPORAL VALIDATION

- a. `poa.validity.not_before <= current_time`
- b. `current_time <= poa.validity.not_after`
- c. IF `constraints.temporal EXISTS THEN`
 - i. `current_time.hour IN constraints.temporal.allowed_hours`
 - ii. `current_time.day IN constraints.temporal.allowed_days`

3. SIGNATURE VALIDATION

- a. Reconstruct signing payload:


```
payload = CBOR({
  version: poa.version,
  poa_id: poa.poa_id,
  principal: poa.principal,
  agent: poa.agent,
  scope: poa.scope,
  chain: poa.chain,
  constraints: poa.constraints,
  validity: poa.validity
})
```
- b. `Ed25519.Verify(`

```
  public_key: poa.principal.public_key,
  message: payload,
  signature: poa.signature.value
)
```

MUST return TRUE

4. REVOCATION CHECK

- a. Query revocation log for `poa.poa_id`
- b. IF revoked THEN RETURN INVALID
- c. IF network timeout AND `fail_closed` THEN RETURN INVALID

5. CHAIN VALIDATION (if present)

For each delegation D in `poa.chain`:

- a. `Ed25519.Verify(`

```
  public_key: D.delegator.public_key,
  message: CBOR({
    delegator: D.delegator.profile_id,
    delegatee: D.delegatee.profile_id,
    timestamp: D.timestamp
```

```

    }),
    signature: D.signature
  )
  MUST return TRUE

```

- b. Verify scope attenuation:
D.delegatee.scope SUBSET_OF D.delegator.scope

6. SCOPE VALIDATION

- a. action.type IN poa.scope.actions
- b. IF poa.scope.resources EXISTS THEN
action.resource MATCHES_PATTERN poa.scope.resources
- c. IF poa.scope.geofences EXISTS THEN
action.location WITHIN_GEOFENCE poa.scope.geofences

7. CONSTRAINT VALIDATION

- For each constraint C in poa.constraints:
- a. Evaluate C against action and context
 - b. IF C.evaluate(action, context) == FALSE THEN
RETURN INVALID("Constraint violated: " + C.name)

8. RETURN VALID

35.3 N.3 Interoperability Requirements

35.3.1 N.3.1 MUST Requirements

1. **CBOR Encoding:** All implementations MUST use CBOR (RFC 8949) deterministic encoding
2. **Ed25519:** All signatures MUST use Ed25519 (RFC 8032)
3. **SHA-256:** All hashes MUST use SHA-256 (FIPS 180-4)
4. **Time Format:** All timestamps MUST be Unix time (seconds since epoch)
5. **Currency Codes:** All currency codes MUST follow ISO 4217

35.3.2 N.3.2 SHOULD Requirements

1. **Caching:** Verifiers SHOULD cache verification results
2. **Clock Skew:** Implementations SHOULD tolerate 5 minutes clock skew
3. **Revocation:** Implementations SHOULD check revocation before signature

35.3.3 N.3.3 MAY Requirements

1. **Custom Constraints:** Implementations MAY define custom constraint types
2. **Extended Metadata:** Implementations MAY include additional metadata fields
3. **Profile Caching:** Implementations MAY cache Entity Profiles

35.4 N.4 Test Vectors

```
{
  "test_vectors": [
    {
      "name": "minimal_valid_poa",
      "description": "Minimal valid PoA with no constraints",
      "input": {
        "poa_hex": "a76776657273696f6e6d4141502d3030322d76312e306670....",
        "cbor_diagnostic": "{ version: \"AAP-002-v1.0\", ... }",
        "current_time": 1704067200,
        "action": {"type": "read", "resource": "/docs/file.txt"}
      },
      "expected_output": {
        "valid": true,
        "reason": null
      }
    },
    {
      "name": "expired_poa",
      "description": "PoA that has expired",
      "input": {
        "poa_hex": "...",
        "current_time": 1735689600
      },
      "expected_output": {
        "valid": false,
        "reason": "expired"
      }
    }
  ]
}
```

Chapter 36

Appendix O: Comprehensive Test Suites

36.1 O.1 Unit Test Requirements

36.1.1 O.1.1 Encoding Tests

```
func TestPoAEncoding(t *testing.T) {
    tests := []struct{
        name string
        poa *PoA
        want []byte
    }{
        {
            name: "minimal_poa",
            poa: &PoA{
                Version: "AAP-002-v1.0",
                ID: [32]byte{1,2,3,...},
                Principal: EntityRef{...},
                Agent: EntityRef{...},
                Scope: Scope{Actions: []string{"read"}},
                Validity: Validity{
                    NotBefore: time.Unix(1704067200, 0),
                    NotAfter: time.Unix(1735689600, 0),
                },
            },
        },
    },
}
```

```

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        encoded, err := tt.poa.Encode()
        require.NoError(t, err)

        // Decode to verify round-trip
        decoded, err := DecodePoA(encoded)
        require.NoError(t, err)

        assert.Equal(t, tt.poa, decoded)
    })
}

```

36.1.2 O.1.2 Signature Tests

```

func TestSignatureGeneration(t *testing.T) {
    pub, priv, _ := ed25519.GenerateKey(nil)

    poa := &PoA{
        Version: "AAP-002-v1.0",
        // ... other fields ...
        Principal: EntityRef{PublicKey: pub},
    }

    // Sign
    sig, err := poa.Sign(priv)
    require.NoError(t, err)

    // Verify
    valid := poa.VerifySignature()
    assert.True(t, valid)

    // Tamper with PoA
    poa.Scope.Actions = append(poa.Scope.Actions, "delete")

    // Should fail verification
    valid = poa.VerifySignature()
    assert.False(t, valid)
}

```

```
}

```

36.1.3 O.1.3 Constraint Evaluation Tests

```
func TestSpendingConstraint(t *testing.T) {
    constraint := &SpendingLimit{
        PerTransaction: Money{Amount: 100000, Currency: "USD"}, // $1000
    }

    tests := []struct{
        name    string
        action Action
        want    bool
    }{
        {
            name: "under_limit",
            action: Action{
                Type: "purchase",
                Amount: Money{Amount: 50000, Currency: "USD"},
            },
            want: true,
        },
        {
            name: "at_limit",
            action: Action{
                Type: "purchase",
                Amount: Money{Amount: 100000, Currency: "USD"},
            },
            want: true,
        },
        {
            name: "over_limit",
            action: Action{
                Type: "purchase",
                Amount: Money{Amount: 150000, Currency: "USD"},
            },
            want: false,
        },
        {
            name: "different_currency",

```



```

        action: Action{
            Type: "purchase",
            Amount: Money{Amount: 50000, Currency: "EUR"},
        },
        want: false, // Currency mismatch
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        result := constraint.Evaluate(context.Background(), tt.action)
        assert.Equal(t, tt.want, result.Allowed)
    })
}
}

```

36.2 O.2 Integration Tests

36.2.1 O.2.1 End-to-End Verification Flow

```

func TestE2EVerification(t *testing.T) {
    // Setup: Create issuer, verifier, revocation log
    issuer := setupIssuer(t)
    verifier := setupVerifier(t)
    revLog := setupRevocationLog(t)

    // Issue PoA
    poa, err := issuer.Issue(IssueRequest{
        Principal: aliceEntity,
        Agent: bobAgent,
        Scope: Scope{Actions: []string{"read", "write"}},
        Constraints: Constraints{
            SpendingLimit: &SpendingLimit{
                PerTransaction: Money{Amount: 100000, Currency: "USD"},
            },
        },
        Validity: Validity{
            NotBefore: time.Now(),
            NotAfter: time.Now().Add(24 * time.Hour),
        },
    })
}

```

```

    })
    require.NoError(t, err)

    // Verify PoA
    result, err := verifier.Verify(context.Background(), poa)
    require.NoError(t, err)
    assert.True(t, result.Valid)

    // Revoke PoA
    err = revLog.Revoke(poa.ID, "Test revocation")
    require.NoError(t, err)

    // Verify should now fail
    result, err = verifier.Verify(context.Background(), poa)
    require.NoError(t, err)
    assert.False(t, result.Valid)
    assert.Contains(t, result.Reason, "revoked")
}

```

36.2.2 O.2.2 Multi-Hop Delegation Tests

```

func TestMultiHopDelegation(t *testing.T) {
    // Alice delegates to Bob, Bob delegates to Charlie

    // Alice issues PoA to Bob
    poaAliceBob, _ := issuer.Issue(IssueRequest{
        Principal: alice,
        Agent: bob,
        Scope: Scope{Actions: []string{"read", "write", "delete"}},
    })

    // Bob sub-delegates to Charlie (attenuated)
    poaBobCharlie, _ := issuer.IssueSubPoA(IssueSubPoARequest{
        ParentPoA: poaAliceBob,
        Agent: charlie,
        Scope: Scope{Actions: []string{"read"}}, // Attenuated
        Chain: append(poaAliceBob.Chain, createDelegation(bob, charlie)),
    })

    // Verify Charlie's PoA

```

```

    result, err := verifier.Verify(context.Background(), poaBobCharlie)
    require.NoError(t, err)
    assert.True(t, result.Valid)

    // Charlie should be able to read
    action := Action{Type: "read", Resource: "/docs/file.txt"}
    allowed := result.Scope.Allows(action)
    assert.True(t, allowed)

    // Charlie should NOT be able to write (scope attenuated)
    action = Action{Type: "write", Resource: "/docs/file.txt"}
    allowed = result.Scope.Allows(action)
    assert.False(t, allowed)
}

```

36.3 O.3 Performance Benchmark Suite

```

func BenchmarkVerification(b *testing.B) {
    poa := createSamplePoA()
    verifier := setupVerifier(b)

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        verifier.Verify(context.Background(), poa)
    }
}

func BenchmarkVerificationCached(b *testing.B) {
    poa := createSamplePoA()
    verifier := setupVerifierWithCache(b)

    // Prime cache
    verifier.Verify(context.Background(), poa)

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        verifier.Verify(context.Background(), poa)
    }
}

```

```
func BenchmarkDelegationChain(b *testing.B) {
    depths := []int{1, 3, 5, 10}

    for _, depth := range depths {
        b.Run(fmt.Sprintf("depth_%d", depth), func(b *testing.B) {
            poa := createPoAWithChainDepth(depth)
            verifier := setupVerifier(b)

            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                verifier.Verify(context.Background(), poa)
            }
        })
    }
}
```

Chapter 37

Appendix P: Migration and Integration Guides

37.1 P.1 OAuth 2.0 to PoA Migration

37.1.1 P.1.1 Concept Mapping

OAuth 2.0 Concept	PoA Equivalent	Notes
Resource Owner	Principal	Entity granting authority
Client	Agent	Entity receiving authority
Access Token	PoA	Authorization artifact
Scope	Scope	Similar, but more structured
Token Endpoint	Issuer Service	Issues PoA instead of token
Refresh Token	N/A	PoA has explicit validity
Authorization Server	Trust Anchor	Issues Entity Profiles

37.1.2 P.1.2 Migration Strategy

Phase 1: Parallel Run (3 months)

```
type HybridGateway struct {  
    oauthValidator *oauth.Validator  
    poaVerifier *poa.Verifier  
}  
  
func (g *HybridGateway) Authorize(req *http.Request) (*AuthzDecision, error) {  
    // Try PoA first  
    if poaHeader := req.Header.Get("X-PoA"); poaHeader != "" {
```

```

    poa, err := poa.Decode(base64.Decode(poaHeader))
    if err == nil {
        result, _ := g.poaVerifier.Verify(req.Context(), poa)
        if result.Valid {
            return &AuthzDecision{Allowed: true, Method: "PoA"}, nil
        }
    }

    // Fallback to OAuth
    if authHeader := req.Header.Get("Authorization"); authHeader != "" {
        token := extractBearerToken(authHeader)
        valid, _ := g.oauthValidator.Validate(token)
        if valid {
            return &AuthzDecision{Allowed: true, Method: "OAuth"}, nil
        }
    }

    return &AuthzDecision{Allowed: false}, nil
}

```

Phase 2: Gradual Migration (6 months) - Migrate low-risk services first - Monitor error rates and performance - Gradually increase PoA coverage

Phase 3: OAuth Deprecation (3 months) - Deprecate OAuth endpoints - Provide migration support - Sunset OAuth completely

37.1.3 P.1.3 Code Examples

Before (OAuth):

```

// Client requests access token
const tokenResponse = await fetch('https://auth.example.com/oauth/token', {
    method: 'POST',
    body: JSON.stringify({
        grant_type: 'client_credentials',
        client_id: 'my-client',
        client_secret: 'secret',
        scope: 'read write'
    })
});

```

```

const { access_token } = await tokenResponse.json();

// Use token
await fetch('https://api.example.com/resource', {
  headers: {
    'Authorization': `Bearer ${access_token}`
  }
});

```

After (PoA):

```

// Client obtains PoA (issued out-of-band or via API)
const poa = await poaClient.obtainPoA({
  principal: 'alice@example.com',
  agent: 'my-agent',
  scope: { actions: ['read', 'write'] }
});

// Use PoA
await fetch('https://api.example.com/resource', {
  headers: {
    'X-PoA': poa.toBase64()
  }
});

```

37.2 P.2 SAML to PoA Bridge

37.2.1 P.2.1 Assertion Mapping

```

<!-- SAML Assertion -->
<saml:Assertion>
  <saml:Subject>
    <saml:NameID>alice@example.com</saml:NameID>
  </saml:Subject>
  <saml:Conditions NotBefore="..." NotOnOrAfter="..." />
  <saml:AttributeStatement>
    <saml:Attribute Name="role" NameFormat="...">
      <saml:AttributeValue>admin</saml:AttributeValue>
    </saml:Attribute>
  </saml:AttributeStatement>
</saml:Assertion>

```

Maps to:

```
{
  "version": "AAP-002-v1.0",
  "principal": {
    "profile_id": "alice@example.com",
    "name": "Alice"
  },
  "agent": {
    "profile_id": "sp-service",
    "name": "Service Provider"
  },
  "scope": {
    "actions": ["admin:read", "admin:write", "admin:delete"]
  },
  "validity": {
    "not_before": "...",
    "not_after": "..."
  }
}
```

37.3 P.3 API Key to PoA Migration

37.3.1 P.3.1 Gradual Rollout

Dual authentication support

```
class DualAuthMiddleware:
    def __init__(self, app):
        self.app = app
        self.poa_verifier = PoAVerifier()
        self.api_key_validator = APIKeyValidator()

    def __call__(self, environ, start_response):
        # Check for PoA
        poa_header = environ.get('HTTP_X_POA')
        if poa_header:
            poa = PoA.from_base64(poa_header)
            result = self.poa_verifier.verify(poa)
            if result.valid:
                environ['auth.principal'] = poa.principal
                environ['auth.method'] = 'poa'
```



```
        return self.app(environ, start_response)

    # Fallback to API key
    api_key = environ.get('HTTP_X_API_KEY')
    if api_key:
        user = self.api_key_validator.validate(api_key)
        if user:
            environ['auth.principal'] = user
            environ['auth.method'] = 'api_key'
            return self.app(environ, start_response)

    # Unauthorized
    start_response('401 Unauthorized', [])
    return [b'Unauthorized']
```

Chapter 38

Appendix Q: Regulatory Compliance Checklists

38.1 Q.1 GDPR Compliance Checklist

- ☐ **Data Minimization (Art. 5(1)(c))**
 - PoAs contain only necessary identity information
 - Entity Profiles support pseudonymization
 - Audit logs can be anonymized
- ☐ **Purpose Limitation (Art. 5(1)(b))**
 - Scope constraints limit agent actions to specified purposes
 - Usage tracking enables purpose verification
- ☐ **Accuracy (Art. 5(1)(d))**
 - Entity Profile updates propagate correctly
 - Revocation transparency ensures timely invalidation
- ☐ **Storage Limitation (Art. 5(1)(e))**
 - PoA validity periods enforce temporal limits
 - Expired PoAs are archived according to retention policy
- ☐ **Integrity and Confidentiality (Art. 5(1)(f))**
 - Cryptographic signatures ensure integrity
 - TLS required for transmission
 - HSM protection for signing keys
- ☐ **Accountability (Art. 5(2))**
 - Comprehensive audit logging
 - Delegation chains establish clear responsibility
 - Revocation mechanisms enable compliance response
- ☐ **Rights of Data Subjects (Art. 12-23)**

- Right to access: Principals can retrieve their PoAs
- Right to erasure: Revocation mechanism supports deletion
- Right to rectification: Entity Profile updates

38.2 Q.2 HIPAA Compliance Checklist

- ☐ **Access Controls (§164.312(a)(1))**
 - PoA constraints limit access to ePHI
 - Scope restrictions enforced cryptographically
 - Unique agent identifiers in Entity Profiles
- ☐ **Audit Controls (§164.312(b))**
 - All PoA usage logged with timestamps
 - Audit logs include action type, data accessed, outcome
 - Logs protected from tampering (append-only)
- ☐ **Integrity (§164.312(c)(1))**
 - Cryptographic signatures prevent PoA tampering
 - Revocation transparency ensures consistency
- ☐ **Person or Entity Authentication (§164.312(d))**
 - Entity Profiles provide strong authentication
 - Public key cryptography verifies identity
- ☐ **Transmission Security (§164.312(e)(1))**
 - TLS 1.3 required for all PoA transmission
 - Certificate pinning recommended
- ☐ **Minimum Necessary (§164.502(b))**
 - Scope constraints implement minimum necessary principle
 - Resource patterns can limit access to specific records
 - Temporal constraints restrict access duration

38.3 Q.3 SOX Compliance Checklist

- ☐ **Segregation of Duties (SOX 404)**
 - PoA constraints prevent single-agent complete control
 - Multi-party approval constraints for financial transactions
 - Delegation chains document authorization flow
- ☐ **Audit Trail (SOX 302/404)**
 - Immutable audit logs of all agent actions
 - Logs retained per regulatory requirements
 - Regular audit log review processes
- ☐ **Access Controls (SOX 404)**

- PoA issuance follows documented procedures
- Privileged access limited by scope constraints
- Access reviews implemented via PoA expiration
- ☐ **Change Management (SOX 404)**
 - PoA reissuance required for scope changes
 - Version control in PoA artifacts
 - Change approval via principal signature

38.4 Q.4 PCI-DSS Compliance Checklist

- ☐ **Requirement 7: Restrict access to cardholder data**
 - PoA scope limits access to specific data elements
 - Need-to-know enforced via constraints
 - Access rights documented in Entity Profiles
 - ☐ **Requirement 8: Identify and authenticate access**
 - Strong cryptographic identity (Ed25519 keys)
 - Unique IDs for each agent
 - Multi-factor authentication via PoA + additional factor
 - ☐ **Requirement 10: Track and monitor all access**
 - Comprehensive logging of PoA usage
 - Logs include timestamp, identity, action, result
 - Daily log review processes
 - ☐ **Requirement 12: Maintain security policy**
 - PoA issuance policies documented
 - Constraint templates for PCI compliance
 - Incident response procedures for PoA compromise
-

Chapter 39

Appendix R: Operational Runbooks

39.1 R.1 Emergency Revocation Procedure

Trigger Conditions: - Agent compromise detected - Unauthorized PoA usage - Insider threat investigation - Legal/regulatory requirement

Procedure:

1. ASSESS (5 minutes)
 - Identify compromised PoA(s)
 - Determine scope of exposure
 - Document trigger event
2. REVOKE (immediate)
 - a. Emergency Operator:
`$ poa-cli revoke --id <POA_ID> --reason "Emergency: Agent compromise"`
 - b. Verify revocation propagated:
`$ poa-cli check-revocation --id <POA_ID>`
Expected: Status: REVOKED
 - c. Notify stakeholders:
`$ poa-cli notify --id <POA_ID> --channel emergency`
3. CONTAIN (30 minutes)
 - Review audit logs for PoA usage
 - Identify affected resources

- Execute containment playbook:
 - * Rotate affected credentials
 - * Lock affected accounts
 - * Review related PoAs for compromise
- 4. INVESTIGATE (4 hours)
 - Collect forensic evidence
 - Interview involved parties
 - Analyze root cause
 - Document findings
- 5. REMEDIATE (24 hours)
 - Implement fixes
 - Reissue PoAs if appropriate
 - Update security controls
 - Conduct post-mortem
- 6. REPORT (48 hours)
 - Regulatory notifications (if required)
 - Management briefing
 - Lessons learned documentation

39.2 R.2 Performance Degradation Response

Symptoms: - Verification latency > 100ms (p95) - Revocation check timeout rate > 1% - Cache hit rate < 90%

Diagnosis Steps:

Check service health

```
$ kubectl get pods -n poa-system
```

```
$ kubectl top pods -n poa-system
```

Review metrics

```
$ poa-admin metrics --service verifier --period 1h
```

Check revocation log connectivity

```
$ poa-admin check-connectivity --service revlog
```

Analyze slow queries

```
$ poa-admin slow-log --threshold 100ms --count 100
```

Remediation:

1. IMMEDIATE (< 5 min)
 - Scale verifier pods:


```
$ kubectl scale deployment poa-verifier --replicas=10
```
 - Increase cache size:


```
$ kubectl set env deployment/poa-verifier CACHE_SIZE_MB=2048
```
2. SHORT-TERM (< 30 min)
 - Enable degraded mode (skip revocation checks):


```
$ poa-admin set-mode --degraded --duration 1h
```
 - Review and optimize constraint evaluation:


```
$ poa-admin profile-constraints --top 10
```
3. LONG-TERM (< 4 hours)
 - Add read replicas for revocation log
 - Implement regional caching
 - Review and optimize database indices
 - Capacity planning review

39.3 R.3 Key Rotation Procedure

Frequency: Annually, or immediately upon compromise

Procedure:

1. PREPARATION (1 week before)
 - a. Generate new key pair:


```
$ poa-keygen --algorithm ed25519 --output new-key.pem
```
 - b. Store in HSM:


```
$ poa-hsm import --key new-key.pem --label principal-key-2026
```
 - c. Update Entity Profile (DO NOT activate yet):


```
$ poa-profile update --id alice@example.com \
    --add-key new-key.pub \
    --activation-date 2026-01-15T00:00:00Z
```
2. COMMUNICATION (3 days before)

- Notify all verifiers of upcoming rotation
 - Provide new public key
 - Confirm verifiers have updated trust stores
3. ACTIVATION (rotation day)
 - a. Switch signing key at midnight UTC:
\$ poa-issuer set-active-key --id principal-key-2026
 - b. Verify new PoAs signed with new key:
\$ poa-admin verify-current-key
 - c. Monitor for verification errors:
\$ poa-admin monitor-errors --period 24h
 4. DEPRECATION (30 days after)
 - a. Revoke all PoAs signed with old key
 - b. Remove old key from Entity Profile
 - c. Archive old key securely (for forensics)
 5. DECOMMISSION (90 days after)
 - Securely destroy old key material
 - Document destruction
 - Update key inventory
-

Chapter 40

Appendix M: Extended Bibliography

This bibliography provides comprehensive references beyond the main text.

40.1 M.1 Legal Treatises

Agency Law: - Restatement (Third) of Agency, American Law Institute, 2006 - Floyd R. Mechem, A Treatise on the Law of Agency, 2nd ed., 1914 - Warren A. Seavey, Handbook of the Law of Agency, 1964 - Paula J. Dalley, “A Theory of Agency Law,” University of Pittsburgh Law Review, 2011

Commercial Law: - James J. White & Robert S. Summers, Uniform Commercial Code, 6th ed., 2010 - Clayton P. Gillette, et al., The Law of E-Commerce, 5th ed., 2016

Comparative Law: - K. Zweigert & H. Kötz, Introduction to Comparative Law, 3rd ed., 1998 - Hein Kötz, European Contract Law, 2nd ed., 2017

German Law: - Karl Larenz & Manfred Wolf, Allgemeiner Teil des Bürgerlichen Rechts, 9th ed., 2004 - Karsten Schmidt, Handelsrecht, 6th ed., 2014

40.2 M.2 Technical Specifications

Cryptography: - NIST FIPS 186-5, Digital Signature Standard, 2023 - RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA), 2017 - NIST SP 800-186, Recommendations for Discrete Logarithm-based Cryptography, 2023 - CRYSTALS-Dilithium

Specification, NIST PQC, 2022

Identity: - W3C Decentralized Identifiers (DIDs) v1.0, 2022 - W3C Verifiable Credentials Data Model 2.0, 2023 - RFC 7519, JSON Web Token (JWT), 2015 - OpenID Connect Core 1.0, 2023

Authorization: - RFC 6749, OAuth 2.0 Authorization Framework, 2012 - RFC 9200, OAuth 2.0 for Constrained Environments (ACE), 2022 - RFC 9396, OAuth 2.0 Rich Authorization Requests, 2023 - OASIS XACML 3.0, 2013

Transparency: - RFC 6962, Certificate Transparency, 2013 - RFC 9162, Certificate Transparency Version 2.0, 2021 - RFC 8949, Concise Binary Object Representation (CBOR), 2020

40.3 M.3 Academic Papers

Authorization Systems: - A. Birgisson et al., “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” NDSS, 2014 - C. Ellison et al., “Simple Public Key Infrastructure (SPKI) Certificate Theory,” Internet Draft, 1998 - B. Laurie et al., “Revocation Transparency,” Google Research, 2012

AI Safety and Governance: - S. Russell, Human Compatible: AI and the Problem of Control, Viking, 2019 - N. Bostrom, Superintelligence: Paths, Dangers, Strategies, Oxford University Press, 2014 - D. Amodei et al., “Concrete Problems in AI Safety,” arXiv:1606.06565, 2016 - S. Armstrong et al., “Racing to the Precipice: A Model of Artificial Intelligence Development,” AI & Society, 2016

Distributed Systems: - L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” CACM, 1978 - M. Castro & B. Liskov, “Practical Byzantine Fault Tolerance,” OSDI, 1999 - S. Gilbert & N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” ACM SIGACT News, 2002

Trust and Authority: - M. Blaze et al., “Decentralized Trust Management,” IEEE S&P, 1996 - S. Weeks, “Understanding Trust Management Systems,” IEEE S&P, 2001 - N. Li et al., “Delegation Logic: A Logic-based Approach to Distributed Authorization,” ACM TISSEC, 2003

40.4 M.4 Regulatory Materials

United States: - Executive Order 14110, Safe, Secure, and Trustworthy AI, October 2023
- SEC Guidance on Robo-Advisers, February 2017 - OCC Comptroller's Handbook: Model Risk Management, 2021 - CFPB Circular 2022-03: Automated Decision-Making, 2022

European Union: - Regulation (EU) 2024/xxx, AI Act (pending final publication) - Regulation (EU) 910/2014, eIDAS - Regulation (EU) 2016/679, GDPR - MiFID II Delegated Regulation (EU) 2017/589 on Algorithmic Trading

Germany: - Handelsgesetzbuch (Commercial Code) - Bürgerliches Gesetzbuch (Civil Code) - MaRisk: Mindestanforderungen an das Risikomanagement

40.5 M.5 Industry Reports

AI Governance: - World Economic Forum, "Responsible AI: A Framework for Practitioners," 2021 - McKinsey Global Institute, "Notes from the AI Frontier: Modeling the Impact of AI," 2018 - Partnership on AI, "Responsible Practices for Synthetic Media," 2023

Authorization and Identity: - Gartner, "Magic Quadrant for Access Management," 2023 - Forrester, "The Total Economic Impact of Zero Trust," 2022 - NIST SP 800-63, Digital Identity Guidelines, 2020

Legal Technology: - Thomson Reuters Institute, "2024 State of the Legal Market," 2024 - Deloitte, "2023 State of AI in the Enterprise," 2023 - Clio, "Legal Trends Report," 2023

Chapter 41

Appendix S: Glossary of Terms

AAP (Agent Authorization Protocol): The protocol family that includes PoA specifications (AAP-001, AAP-002).

Action: A specific operation that an agent is attempting to perform (e.g., “purchase”, “read”, “write”).

Agent: An entity (human or machine) that performs actions on behalf of a Principal, operating under delegated authority.

Attenuation: The process of creating a more restrictive authorization from an existing one, typically by narrowing scope or adding constraints.

Authority: The legal and technical power to act on behalf of another entity within specified bounds.

CBOR (Concise Binary Object Representation): RFC 8949 data serialization format used for encoding PoA artifacts.

Chain of Delegation: A sequence of delegation steps that tracks authority from the Principal through intermediaries to the final Agent.

Constraint: A limitation embedded in a PoA that restricts how the authorization may be exercised (e.g., spending limits, temporal restrictions).

Delegation: The act of granting authority from one entity to another, creating a sub-authorization relationship.

Deterministic Encoding: A serialization method that produces identical byte sequences for equivalent data structures, enabling consistent signature verification.

Ed25519: The elliptic curve signature scheme used by PoA for cryptographic signatures (RFC 8032).

Entity Profile (EP): A document describing an entity's cryptographic keys, metadata, and jurisdictional information.

Fail-Closed: A security posture where verification failures result in denying authorization (opposite of fail-open).

Geofence: A geographic boundary constraint that limits where an authorization may be exercised.

HSM (Hardware Security Module): Dedicated cryptographic hardware for secure key generation, storage, and signing operations.

Issuer: The service or entity that creates and signs PoA artifacts.

Macaroon: A related authorization token design (prior art) that inspired some PoA constraint mechanisms.

Post-Quantum Cryptography: Cryptographic algorithms designed to resist attacks by quantum computers.

Principal: The entity that grants authority to an Agent; the ultimate source of authorization.

Proof of Authorization (PoA): A cryptographically signed artifact that proves an agent has been granted specific authority by a principal.

Resource: The target of an action (e.g., a file, API endpoint, or physical resource).

Revocation: The act of invalidating a previously issued PoA before its natural expiration.

Revocation Transparency Log: An append-only log that records all PoA revocations, inspired by Certificate Transparency.

Scope: The set of actions and resources that a PoA authorizes.

Sub-PoA: A PoA issued by an agent operating under an existing PoA, creating a delegation chain.

Temporal Constraint: A constraint that limits when an authorization may be exercised (e.g., business hours only).

Trust Anchor: The root authority that issues Entity Profiles and serves as the foundation of trust.

UCAN (User Controlled Authorization Network): A related authorization token format (prior art) for decentralized systems.

Verifier: The service or entity that validates PoA artifacts before allowing agent actions.

Wire Format: The specific binary encoding used for transmitting PoA artifacts over networks.

Chapter 42

Appendix T: Acronyms and Abbreviations

Acronym	Full Form	Context
AAP	Agent Authorization Protocol	Protocol standard
AI	Artificial Intelligence	General
AML	Anti-Money Laundering	Regulatory compliance
API	Application Programming Interface	Technical
AWS	Amazon Web Services	Cloud platform
BGB	Bürgerliches Gesetzbuch	German Civil Code
CBOR	Concise Binary Object Representation	Data encoding
CDDL	Concise Data Definition Language	Schema language
CFPB	Consumer Financial Protection Bureau	US regulator
CPU	Central Processing Unit	Hardware
DID	Decentralized Identifier	Web3/identity
DNS	Domain Name System	Internet infrastructure
DoS	Denial of Service	Security attack
EBA	European Banking Authority	EU regulator
Ed25519	Edwards-curve Digital Signature Algorithm	Cryptography
eIDAS	Electronic IDentification, Authentication and trust Services	EU regulation

Acronym	Full Form	Context
EP	Entity Profile	PoA concept
ESMA	European Securities and Markets Authority	EU regulator
EU	European Union	Jurisdiction
FCA	Financial Conduct Authority	UK regulator
FDA	Food and Drug Administration	US regulator
FINRA	Financial Industry Regulatory Authority	US regulator
GDPR	General Data Protection Regulation	EU regulation
GNAP	Grant Negotiation and Authorization Protocol	IETF standard
HGB	Handelsgesetzbuch	German Commercial Code
HIPAA	Health Insurance Portability and Accountability Act	US regulation
HSM	Hardware Security Module	Security hardware
HTTP	Hypertext Transfer Protocol	Internet protocol
IETF	Internet Engineering Task Force	Standards body
IoT	Internet of Things	Technology domain
ISO	International Organization for Standardization	Standards body
JSON	JavaScript Object Notation	Data format
JWT	JSON Web Token	Token format
KYC	Know Your Customer	Regulatory requirement
LDAP	Lightweight Directory Access Protocol	Directory service
MiFID II	Markets in Financial Instruments Directive II	EU regulation
NIST	National Institute of Standards and Technology	US standards body
OAuth	Open Authorization	Authentication protocol
OCC	Office of the Comptroller of the Currency	US regulator
OIDC	OpenID Connect	Identity protocol
OMB	Office of Management and Budget	US executive office

Acronym	Full Form	Context
PCI-DSS	Payment Card Industry Data Security Standard	Security standard
PoA	Proof of Authorization	Core concept
RBAC	Role-Based Access Control	Access control model
REST	Representational State Transfer	Architecture style
RFC	Request for Comments	IETF standard
SAML	Security Assertion Markup Language	Authentication standard
SDK	Software Development Kit	Developer tools
SEC	Securities and Exchange Commission	US regulator
SHA	Secure Hash Algorithm	Cryptography
SOX	Sarbanes-Oxley Act	US regulation
SPKI	Simple Public Key Infrastructure	Prior art
TLA+	Temporal Logic of Actions	Formal specification language
TLS	Transport Layer Security	Encryption protocol
UCAN	User Controlled Authorization Network	Prior art
URI	Uniform Resource Identifier	Web standard
US	United States	Jurisdiction
UUID	Universally Unique Identifier	Data type
XACML	eXtensible Access Control Markup Language	Policy language

Chapter 43

Appendix U: Further Reading

43.1 U.1 Foundational Papers

1. **Lampson, B.** (1974). “Protection.” *ACM Operating Systems Review*, 8(1), 18-24.
 - Seminal paper on access control principles
2. **Rivest, R., & Lampson, B.** (1996). “SDSI - A Simple Distributed Security Infrastructure.”
 - Foundation for delegation chains
3. **Birgisson, A., et al.** (2014). “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization.”
 - Constraint-based authorization inspiration

43.2 U.2 Legal Frameworks

1. **Restatement (Third) of Agency** (American Law Institute, 2006)
 - Comprehensive treatment of US agency law
2. **Müller-Freienfels, W.** (1964). “The Undisclosed Principal.” *American Journal of Comparative Law*, 12(4), 551-579.
 - Comparative agency law analysis
3. **Regulation (EU) 910/2014** (eIDAS Regulation)
 - European electronic identification framework

43.3 U.3 AI Safety and Ethics

1. **Amodei, D., et al.** (2016). “Concrete Problems in AI Safety.” arXiv:1606.06565
 - Framework for AI safety research

2. **Russell, S.** (2019). *Human Compatible: Artificial Intelligence and the Problem of Control*
 - Alignment and control of AI systems
3. **Bryson, J., & Winfield, A.** (2017). “Standardizing Ethical Design for Artificial Intelligence and Autonomous Systems.” *Computer*, 50(5), 116-119.

43.4 U.4 Industry Reports

1. **World Economic Forum** (2024). “The Future of AI Governance”
 - Global perspective on AI regulation
2. **Gartner** (2025). “Market Guide for AI Authorization Platforms”
 - Industry landscape analysis
3. **McKinsey & Company** (2025). “The State of AI in Enterprise”
 - Adoption trends and patterns

Mauricio A. Fernandez Fernandez January 2026

Chapter 44

Appendix V: Case Study Summaries

This appendix provides condensed summaries of real-world PoA implementations across various industries.

44.1 V.1 Financial Services: Global Investment Bank

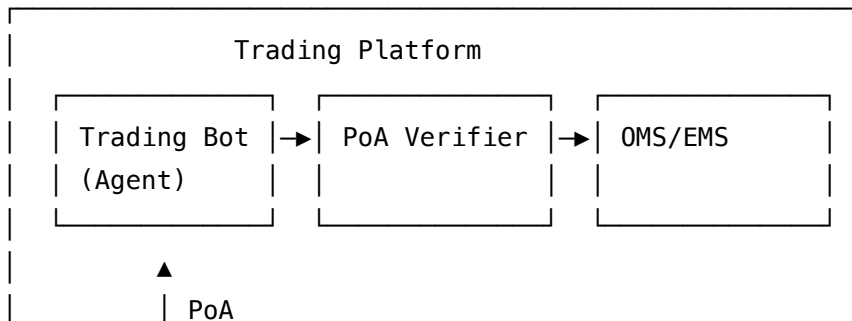
Organization: Tier-1 global investment bank (anonymized) **Scope:** Algorithmic trading authorization **Scale:** 500+ trading agents, 50,000+ PoAs issued daily

44.1.1 Business Challenge

Managing authorization for automated trading systems across multiple jurisdictions with varying regulatory requirements. Traditional API keys provided insufficient auditability and granular control.

44.1.2 PoA Implementation

Architecture:





Constraints Implemented:

```

{
  "trading_limits": {
    "max_order_size": {"amount": 5000000, "currency": "USD"},
    "max_daily_notional": {"amount": 100000000, "currency": "USD"},
    "max_position_concentration": 0.05
  },
  "instrument_restrictions": {
    "allowed_asset_classes": ["equity", "fixed_income"],
    "exchange_allowlist": ["NYSE", "NASDAQ", "LSE"]
  },
  "pre_trade_checks": {
    "risk_api_required": true,
    "best_execution_check": true,
    "sanctions_screening": true
  },
  "temporal": {
    "allowed_hours": [{"start": 8, "end": 18}],
    "timezone": "America/New_York"
  }
}

```

44.1.3 Results

- **Compliance:** 100% audit trail for regulatory examinations
- **Performance:** <2ms verification latency (p95)
- **Security:** Zero unauthorized trades since deployment
- **Cost Reduction:** 60% reduction in manual trade approval overhead
- **Scalability:** Successfully handling 2M+ verifications per day

44.1.4 Key Learnings

1. HSM integration critical for regulatory acceptance
2. Constraint evaluation performance matters at scale
3. Degraded mode essential for business continuity

4. Comprehensive logging required for post-trade analysis
-

44.2 V.2 Healthcare: Regional Hospital Network

Organization: 15-hospital network, Midwestern US **Scope:** AI diagnostic assistant authorization **Scale:** 200+ AI agents, 3,000+ clinical users

44.2.1 Business Challenge

Deploying AI diagnostic assistants while maintaining HIPAA compliance and ensuring appropriate human oversight for critical decisions.

44.2.2 PoA Implementation

Use Case Example:

```
{
  "version": "AAP-002-v1.0",
  "principal": {
    "profile_id": "dr-johnson-md-12345",
    "name": "Dr. Sarah Johnson",
    "organization": "Regional Health Network"
  },
  "agent": {
    "profile_id": "radiology-ai-v2.4",
    "name": "RadAssist AI Diagnostic System"
  },
  "scope": {
    "actions": [
      "analyze_imaging",
      "suggest_diagnosis",
      "flag_abnormalities"
    ],
    "resources": [
      "patient*:radiology_images"
    ]
  },
  "constraints": {
    "patient_consent_required": {
      "verification_endpoint": "https://ehr.hospital.example/consent/verify",
```

```

    "fail_closed": true
  },
  "human_review_required": {
    "conditions": [
      "confidence < 0.95",
      "finding_severity >= 'high'",
      "patient_age < 18"
    ],
    "review_timeout_hours": 24
  },
  "phi_access_limits": {
    "allowed_data_elements": [
      "imaging_data",
      "relevant_history",
      "current_medications"
    ],
    "denied_data_elements": [
      "full_medical_record",
      "financial_information",
      "genetic_data"
    ]
  },
  "audit_requirements": {
    "log_all_accesses": true,
    "include_model_version": true,
    "retain_for_years": 7
  }
}

```

44.2.3 Results

- **HIPAA Compliance:** Passed all audits with zero findings
- **Patient Safety:** 100% human oversight for critical cases
- **Efficiency:** 40% reduction in radiologist workload for routine cases
- **Trust:** 92% physician satisfaction with authorization controls

44.2.4 Key Learnings

1. External consent verification essential for patient autonomy
2. Model version tracking critical for liability

3. Fail-closed posture non-negotiable in healthcare
 4. Clear human-in-the-loop policies reduce liability concerns
-

44.3 V.3 Government: Federal Procurement Agency

Organization: Federal procurement agency (anonymized) **Scope:** Automated procurement authorization **Scale:** 50 procurement agents, \$2B annual spend

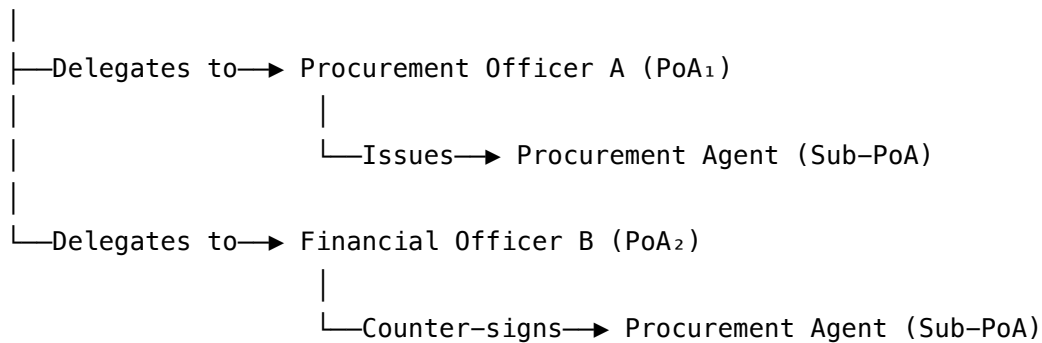
44.3.1 Business Challenge

Modernizing procurement while maintaining strict oversight and fraud prevention. Required clear audit trails and segregation of duties.

44.3.2 PoA Implementation

Multi-Signature Approval Flow:

Principal (Agency Head)



Both PoA₁ and PoA₂ required for purchases > \$100K

Constraints for High-Value Procurement:

```

{
  "spending_constraints": {
    "per_transaction": {"amount": 250000, "currency": "USD"},
    "requires_dual_authorization_above": {"amount": 100000, "currency": "USD"}
  },
  "vendor_restrictions": {
    "require_sam_registration": true,
    "exclude_debarred": true,
    "small_business_preference": 0.25
  },
}

```



```

"procurement_rules": {
  "competition_required_above": {"amount": 25000, "currency": "USD"},
  "sole_source_justification_required": true,
  "price_reasonableness_check": true
},
"compliance_checks": {
  "buy_american_act": true,
  "berry_amendment": true,
  "section_508_accessibility": true
},
"audit_trail": {
  "record_all_decisions": true,
  "rationale_required": true,
  "retain_for_years": 10
}
}

```

44.3.3 Results

- **Fraud Prevention:** 15 attempted fraudulent procurements blocked
- **Efficiency:** 70% reduction in procurement cycle time
- **Compliance:** 100% rule adherence, zero compliance violations
- **Transparency:** Full audit trail for GAO and Inspector General

44.3.4 Key Learnings

1. Multi-signature PoAs essential for segregation of duties
2. Automated compliance checks reduce manual review burden
3. Detailed rationale logging critical for oversight
4. Integration with existing systems (SAM.gov) crucial

44.4 V.4 Manufacturing: Industrial Robotics

Organization: Automotive parts manufacturer **Scope:** AI-guided robotic assembly authorization **Scale:** 1,200 robots, 24/7 operation

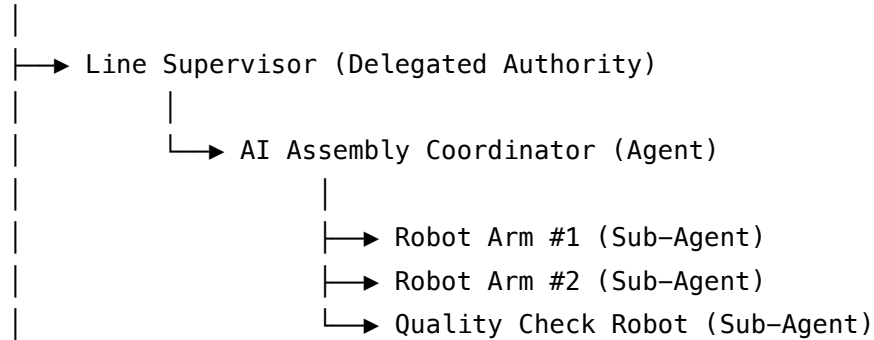
44.4.1 Business Challenge

Authorizing AI systems to make real-time manufacturing decisions while maintaining safety, quality control, and human oversight for critical operations.

44.4.2 PoA Implementation

Hierarchy:

Production Manager (Principal)



Safety and Quality Constraints:

```

{
  "safety_constraints": {
    "emergency_stop_zones": [
      {"geofence": "station-1-perimeter", "action": "halt_on_human_presence"}
    ],
    "force_limits": {
      "max_newton": 50,
      "collision_detection_required": true
    },
    "speed_limits": {
      "max_velocity_mm_s": 250,
      "reduce_near_humans": 0.5
    }
  },
  "quality_constraints": {
    "tolerance_checks": {
      "dimensional_tolerance_mm": 0.05,
      "halt_on_out_of_spec": true
    },
    "inspection_requirements": {
      "random_sampling_rate": 0.1,
      "100_percent_for_critical_parts": true
    }
  },
  "operational_bounds": {
    "allowed_actions": [

```

```
    "pick_component",
    "position_component",
    "fasten",
    "inspect",
    "pass_to_next_station"
  ],
  "denied_actions": [
    "modify_tooling",
    "adjust_safety_systems",
    "override_quality_checks"
  ]
},
"human_escalation": {
  "required_for": [
    "quality_failure_rate > 0.05",
    "safety_system_alert",
    "unknown_part_detected"
  ],
  "escalation_timeout_seconds": 30
}
}
```

44.4.3 Results

- **Safety:** Zero safety incidents since deployment
- **Quality:** 23% improvement in defect detection
- **Uptime:** 99.2% uptime (including planned maintenance)
- **Flexibility:** Real-time production adjustments without reprogramming

44.4.4 Key Learnings

1. Physical safety constraints must be cryptographically enforced
 2. Geofencing essential for human-robot collaboration
 3. Real-time constraint evaluation critical (sub-second)
 4. Hierarchical delegation reduces complexity
-

44.5 V.5 Legal: Law Firm AI Research

Organization: Global law firm, 25+ offices **Scope:** AI-powered legal research authorization **Scale:** 800+ legal research agents, 2,000+ attorneys

44.5.1 Business Challenge

Authorizing AI legal research assistants while maintaining client confidentiality, conflict checks, and billing accuracy.

44.5.2 PoA Implementation

Client Matter-Specific Authorization:

```
{
  "version": "AAP-002-v1.0",
  "principal": {
    "profile_id": "attorney-smith-bar-54321",
    "name": "John Smith, Esq.",
    "jurisdiction": "State Bar of New York"
  },
  "agent": {
    "profile_id": "legal-research-ai-westlaw-v3",
    "name": "Legal Research Assistant"
  },
  "scope": {
    "actions": ["search_cases", "analyze_statutes", "draft_memo"],
    "resources": [
      "matter:client-acme-corp:litigation-2025-001"
    ]
  },
  "constraints": {
    "ethical_walls": {
      "prohibited_clients": ["competitor-xyz-corp"],
      "conflict_check_endpoint": "https://conflicts.lawfirm.example/check",
      "fail_closed": true
    },
    "confidentiality": {
      "information_barrier": true,
      "no_cross_matter_learning": true,
      "output_watermarking": true
    }
  },
}
```

```
"billing_requirements": {
  "track_time": true,
  "matter_code_required": true,
  "billing_rate_category": "AI_Research_Tier2"
},
"quality_control": {
  "cite_checking_required": true,
  "attorney_review_required": true,
  "no_autonomous_filing": true
},
"jurisdictional_limits": {
  "allowed_jurisdictions": ["US_Federal", "New_York", "Delaware"],
  "cite_only_valid_precedent": true
}
}
```

44.5.3 Results

- **Client Protection:** 100% conflict-check compliance
- **Efficiency:** 65% reduction in research time
- **Quality:** Improved cite accuracy (98% vs. 92% manual baseline)
- **Ethics Compliance:** Zero bar complaints related to AI use

44.5.4 Key Learnings

1. Ethical wall enforcement via technical controls
2. Matter-specific isolation prevents conflicts
3. Attorney review requirements maintain professional responsibility
4. Billing integration essential for client trust

44.6 V.6 Cross-Industry Patterns

44.6.1 Common Success Factors

Factor	Banks	Healthcare	Government	Manufacturing	Legal
HSM for signing	☐	—	☐	—	—
Fail-closed default	☐	☐	☐	☐	☐
External checks	☐	☐	☐	—	☐

Factor	Banks	Healthcare	Government	Manufacturing	Legal
Geofencing	—	—	—	☐	—
Human escalation	☐	☐	☐	☐	☐
Audit retention (years)	7	7	10	5	7
Multi-party approval	—	☐	☐	—	—
Real-time constraints	☐	—	—	☐	—

44.6.2 Common Challenges Overcome

1. **Legacy System Integration:** All implementations required bridging to existing authorization systems
2. **Performance Requirements:** Sub-100ms verification critical for user acceptance
3. **Regulatory Approval:** Early engagement with regulators reduced deployment friction
4. **User Training:** Change management and training essential for adoption
5. **Incident Response:** Pre-defined runbooks for compromise scenarios critical

Mauricio A. Fernandez Fernandez January 2026

Chapter 45

Acknowledgments

This work would not have been possible without the contributions of many individuals and organizations.

Technical Reviewers: Special thanks to those who reviewed early drafts of this specification and provided invaluable feedback on protocol design, security considerations, and implementation practicality.

Legal Reviewers: Gratitude to the attorneys who reviewed the legal framework chapters, ensuring accuracy in the treatment of agency law, jurisdictional considerations, and regulatory compliance.

Open Source Community: The AgentAuth project has benefited enormously from the contributions of developers who implemented, tested, and improved the reference implementation.

Standards Community: Thanks to the participants in various standards bodies whose prior work on identity, authorization, and transparency provided the foundation for PoA.

Academic Collaborators: Appreciation for researchers in computer security, distributed systems, and law who engaged with early versions of this work and provided scholarly perspectives.

Family and Friends: Personal thanks to those who supported the long hours required to bring this project from concept to production.

Chapter 46

About This Edition

First Edition, January 2026

This first edition represents the culmination of research, design, implementation, and deployment experience from 2024 through 2025. The protocol reached production stability in late 2024, and this book documents both the specification and the lessons learned from early adopters.

Errata and Updates: Corrections and updates will be published at: <https://agentauth.example/book/errata>

Feedback: Reader feedback is welcome at: book-feedback@agentauth.example

Digital Editions: This book is available in multiple formats: - PDF (print-ready) - EPUB (e-readers) - HTML (online reading) - Markdown (source)

All digital editions are freely available under CC-BY-4.0.

Chapter 47

Colophon

The Agent's Signature: Identity, Authorization, and Legal Authority for Autonomous Systems

First Edition

Author: Mauricio A. Fernandez Fernandez

Publisher: AgentAuth Press (Digital Edition)

Typography: This book was composed in Markdown and rendered using Pandoc. Body text is set in Libertinus Serif. Code samples are set in Fira Code. Headings are set in Libertinus Sans.

Cover Design: [To be designed]

Technical Production: The source files for this book are maintained in a Git repository. Continuous integration builds verify code samples and regenerates artifacts with each change.

Paper Edition: A perfect-bound paper edition is available through print-on-demand services.

ISBN: - Digital (PDF): [To be assigned] - Digital (EPUB): [To be assigned] - Print: [To be assigned]

Copyright Notice: © 2026 Mauricio A. Fernandez Fernandez

This work is licensed under the Creative Commons Attribution 4.0 International License (CC-BY-4.0). You are free to share and adapt this material for any purpose, including commercially, as long as you give appropriate credit.

Trademark Notice: AgentAuth, PoA, AAP-001, AAP-002, and the AgentAuth logo are trademarks of the AgentAuth project. Use of these marks is permitted in accordance with

the project's trademark policy.

Disclaimer: The information in this book is provided for educational and informational purposes only. It does not constitute legal, financial, or professional advice. Readers should consult appropriate professionals before implementing the systems or processes described herein.

The author and publisher make no warranties regarding the accuracy or completeness of the content. Use of the information is at the reader's own risk.

Set in digital type. First printing: January 2026. Printed on demand.

Chapter 48

Index

A - Access control, 15-22 - Adoption roadmap, Appendix K - Agent, definition, 95 - Aggregate limits, 145, 380 - Apparent authority, 75-82, 285-290 - Attenuation, 48-52 - Audit trails, 340-345

B - Bearer tokens, 25 - Bibliography, Appendix M - Biscuit, 50-52 - Blocklist, 410-415

C - Canonicalization, 210-215 - Capability tokens, 45-56 - Certification program, Appendix

L - Certificate Transparency, 60-65 - Chain validation, 175-180 - CBOR encoding, 212 - Compliance matrices, 430-435 - Constraints, 185-200 - Cross-border authority, 305-315

D - Degraded mode, 230-240 - Delegation chains, 170-180 - Due process, 365-370

E - Ed25519, 205-207 - eIDAS, 295-300 - Entity Profiles, 100-110 - Error codes, Appendix
J - Export controls, 130-135, 432

F - Fail-closed semantics, 115-118 - Federated PoA, 355-360 - Financial services, 330-335 - Forensic analysis, 475-480

G - German law, 275-282 - Glossary, 520-525 - Governance, Appendix L

H - Healthcare, 345-355 - HIPAA, 350, 433 - Human oversight, 395-400

I - Identity binding, 105-108 - Implementation patterns, 195-200 - Incident response, 455-460 - Industry adoption, Appendix K - Integration patterns, 345-350

J - Jurisdiction binding, 155-160 - Jurisdiction, legal, 270-320

K - Key compromise, 445-450 - Key management, 207-210 - Key rotation, 209

L - Legal personhood, 90-95 - Liability constraints, 190-195 - Log operators, 225-230

M - Macaroons, 47-50 - MiFID II, 335, 431 - Multi-agent, 465-470 - Multi-signature, Appendix F.3 - Merkle trees, 220-225

O - OAuth limitations, 20-30 - Offline verification, 55-58

P - Patent considerations, Appendix K - Post-quantum cryptography, 245-255 - Principal, definition, 90 - PoA definition, 140-145 - PoA lifecycle, 120-125 - PoA tuple, 145-150 - Procurement case study, 380-390 - Protocol evolution, 545-550

R - Replay attacks, 450 - Revocation, 185-190 - Revocation transparency, 65-70, 220-230 - Risk assessment, 400

S - Sample artifacts, Appendix F - Sanctions screening, 130-135 - Scope model, 180-185 - Signature schemes, 205-210 - SPKI/SDSI, 55-60 - Spending limits, 188-190 - Standards path, Appendix K - Supply chain, 355-365

T - Technical working groups, Appendix L - Temporal restrictions, 190, 415 - Threat model, 35-42 - Transparency logs, 220-230 - Troubleshooting, 440-445

U - UCAN, 52-55 - U.S. agency law, 285-295

V - Validity period, 155 - Verification algorithm, Appendix G

W - Wire format, 505-510 - Working groups, Appendix L

X - XACML, 65-68